Prompts y respuestas IA

Álvaro Arcís Reverte y Alejandro Nicolás Navarro

Máster Universitario en Inteligencia Artificial
Curso 2026–2027

Prompt

Prepara un archivo README inicial, teniendo en cuenta lo pedido.

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
# Predicción Multiclase de Riesgo Cardiovascular en Población Adulta
> **Máster Universitario en Inteligencia Artificial - Universidad de
Murcia (UMU)**
> **Asignatura:** Machine Learning | **Curso:** 2026/2027
> **Colaboración clínica:** Hospital Morales Meseguer

---

##   Descripción del Proyecto

Las enfermedades cardiovasculares representan una de las principales
causas de morbimortalidad a nivel global. Este proyecto tiene como
objetivo diseñar, entrenar y evaluar modelos predictivos de Machine
Learning capaces de clasificar con alta precisión y fiabilidad clínica el
nivel de riesgo cardiovascular en una cohorte adulta de **2.254
pacientes**.

La variable objetivo (`riesgo_cardiovascular`) contempla tres niveles de
estratificación:
* **Clase 0:** Bajo Riesgo
* **Clase 1:** Riesgo Moderado
* **Clase 2:** Alto Riesgo

El repositorio recoge el flujo de trabajo end-to-end: desde el análisis
exploratorio de datos (EDA) y la mitigación de problemáticas de calidad
del dato (imputación, outliers, desbalanceo, escalado), hasta el ajuste
de hiperparámetros, validación cruzada y comparación estadística de
clasificadores base y ensambles.

---

##   Integrantes del Equipo

| Nombre y Apellidos | Usuario de GitHub | Correo Institucional |
| :--- | :--- | :--- |
| *[Nombre Miembro 1]* | `@usuario1` | `usuario1@um.es` |
| *[Nombre Miembro 2]* | `@usuario2` | `usuario2@um.es` |
| *[Nombre Miembro 3]* | `@usuario3` | `usuario3@um.es` |

---

##   Estructura del Repositorio

El proyecto se organiza bajo una estructura modular orientada a la
reproducibilidad científica:
```

```text
  data/
      raw/                          # Dataset original
(enfermedad_cardiovascular.csv)
      processed/                    # Datasets limpios o transformados (si
aplica)
  docs/                             # Documentación técnica y entregables
      memoria_tecnica.pdf           # Memoria técnica maquetada (máx. 15
páginas)
      uso_etico_ia.pdf              # Registro de prompts y reflexión
crítica de IA
      video_link.txt                # Enlace al vídeo explicativo (máx. 7
min)
  notebooks/                        # Jupyter Notebooks de desarrollo y
análisis
      01_eda_calidad_datos.ipynb # Análisis exploratorio y detección de
problemáticas
      02_preprocesamiento.ipynb  # Transformaciones, codificación e
imputación
      03_modelado_evaluacion.ipynb# Entrenamiento, ajuste de
hiperparámetros y métricas
      04_test_estadisticos.ipynb # Comparación estadística de modelos
(opcional)
  src/                              # Código fuente refactorizado y
reutilizable
      __init__.py
      data_loader.py                # Carga y comprobación de integridad
      preprocessing.py             # Pipelines de preprocesamiento
(Scikit-Learn)
      models.py                     # Definición y pipelines de algoritmos
      evaluation.py                 # Cálculo de métricas y matrices de
confusión
  .gitignore                        # Filtro de ficheros innecesarios
  environment.yml                   # Configuración del entorno Conda
(alternativa)
  requirements.txt                  # Fichero de dependencias fijadas
(versiones exactas)
  README.md                         # Documento principal del repositorio
```

Análisis

README inicial preparado, incluyendo descripción del proyecto, integrantes, estructura de carpetas y archivos. Se rellena con los datos de los miembros y la estructura del proyecto se cambiará.

Prompt

```
lee el proyecto y recomiendame una estructura para los archivos, crea los
archivos y las carpetas necesarias, pero ni se te ocurra resolverlos, e
incluye esta interacción al Uso_IA
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
Tras leer el enunciado, las instrucciones de entrega, el dataset y el
README, propuso y creó la siguiente estructura (solo esqueletos vacíos,
sin resolver ningún apartado):

- data/raw/            -> se movió aquí enfermedad_cardiovascular.csv
(git mv)
- data/processed/      -> vacío (.gitkeep)
- notebooks/           -> 01_eda, 02_preprocesamiento, 03_modelado,
04_evaluacion, 05_test_estadisticos (.ipynb), cada uno solo con títulos
de sección y celdas "# TODO"
- src/                 -> __init__, config, data_loader, preprocessing,
models, evaluation, statistical_tests (.py), solo con un docstring que
describe su responsabilidad y "TODO"
- results/figures/, results/tables/, models/ -> vacíos (.gitkeep)
- docs/decisiones.md   -> tabla para registrar decisiones de diseño y
avances
- docs/memoria/, docs/uso_etico_ia/ -> vacíos (.gitkeep)
- docs/video.md        -> plantilla para el enlace del vídeo
- requirements.txt     -> vacío, con instrucciones para rellenarlo con
pip freeze
- .gitignore           -> entornos virtuales, cachés, checkpoints,
modelos y auxiliares de LaTeX
- README.md            -> se rellenó la sección "Estructura del
Repositorio" y se añadió una sección "Reproducción" pendiente
```

Análisis

Se prepara la estructura de carpetas y archivos del proyecto, incluyendo los esqueletos vacíos para cada sección para facilitar la organización y el desarrollo posterior.

Conversación Alejandro para el EDA:
[Ver conversación interactiva completa (HTML renderizado)]

Análisis

En el enlace se sitúa la conversación mantenida con el LLM para la resolución de problemáticas de calidad del dato, incluyendo prompts, respuestas resumidas y análisis crítico.

Prompt

en visual studio code (.ipynb) cuando muestro, por ejemplo, un dataframe, se cortan las columnas o las filas a mitad y lo mismo ocurre con los print largos. Cómo puedo ver la salida completa?

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Este problema ocurre por dos razones: las limitaciones propias de Pandas (para
no colapsar la memoria) y las limitaciones de la interfaz de VS Code (para no
ralentizar el editor).

Aquí tienes las soluciones para ambos casos:

1. Para DataFrames de Pandas (Filas y Columnas cortadas)

Pandas viene configurado por defecto para truncar tablas grandes. Puedes cambiar
esta configuración ejecutando esto en una celda al principio de tu notebook:

```
import pandas as pd

# Mostrar todas las columnas
pd.set_option('display.max_columns', None)

# Mostrar todas las filas (¡Cuidado si tienes millones de filas!)
pd.set_option('display.max_rows', None)

# Mostrar el contenido completo de cada celda (que no corte el texto largo)
pd.set_option('display.max_colwidth', None)
```

Consejo Pro para VS Code:
VS Code tiene un visor de datos integrado excelente.

1. Haz clic en el botón "Variables" en la barra superior del notebook (al lado
   de "Clear All Outputs").
2. Busca tu DataFrame y haz clic en el icono pequeño que parece una tabla a la
   izquierda del nombre de la variable.
   Se abrirá una pestaña interactiva donde puedes ordenar, filtrar y ver todos
   los datos sin recortar.

2. Para salidas de texto largas (print())

```
Si haces un print() muy largo, VS Code a veces corta el texto o le pone
una
barra de desplazamiento molesta.

Opción A: Desactivar el scroll (Hacer que se vea todo hacia abajo)

1.  Pasa el cursor por el margen izquierdo de la celda de salida (al lado
de
    donde aparece el texto).
2.  Haz clic en los tres puntos (...) que aparecen o haz clic derecho
sobre la
    salida.
3.  Selecciona "Toggle Output Scrolling" (Alternar desplazamiento de
salida).
    Esto expandirá la celda para que muestre todo el texto en lugar de
    recortarlo en una ventanita pequeña.

Opción B: Cambiar el límite de líneas en la configuración de VS Code

Por seguridad, VS Code limita el texto plano a unas 100 líneas por celda
para no
trabarse. Puedes aumentarlo:

1.  Ve a Configuración: Ctrl + , (en Windows/Linux) o Cmd + , (en Mac).
2.  En la barra de búsqueda escribe: notebook text line limit.
3.  Busca la opción: Notebook > Output: Text Line Limit.
4.  Aumenta el número (por defecto suele ser 100, puedes poner 500, 1000
o más).

3. Solución rápida en código para un print puntual

Si solo quieres imprimir un texto muy largo o un DataFrame como texto
completo
una sola vez sin cambiar la configuración global:

# Para DataFrames:
print(df.to_string())

# Para textos gigantes (evitar que la UI se congele guardándolo en un
archivo temporal):
with open("salida.txt", "w", encoding="utf-8") as f:
    f.write(str(tu_variable_gigante))
# Y luego lo abres directamente en VS Code con Ctrl+O
```

Análisis

Se acepta el código para poder visualizar los dataframes que pandas corta por defecto, dejándolo como código para descomentar si se necesita.

Prompt

cómo se lee una matriz de correlación? hay diferencia a la hora de analizar correlacion entre variables y entre la variable target?

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Para entender una matriz de correlación sin perderte, la regla de oro es
dividirla en dos zonas completamente distintas:

1.  La relación entre tus variables predictoras y el TARGET (la fila o
columna
    de la variable a predecir).
2.  La relación entre las variables predictoras ENTRE SÍ (el resto de
celdas de
    la matriz).

Lo que en una zona es excelente, en la otra puede ser un problema grave.

Lo básico: ¿Qué significa el número? (Coeficiente de -1 a 1)

  - Cercano a 1 (Positivo fuerte): Se mueven en la misma dirección. Si el
  valor
    de una variable sube, el de la otra también tiende a subir.
  - Cercano a -1 (Negativo fuerte): Se mueven en direcciones opuestas. Si
  el
    valor de una variable sube, el de la otra tiende a bajar (relación
    inversa).
  - Cercano a 0: No hay relación lineal/monótona. Lo que le ocurra a una
    variable no nos dice absolutamente nada sobre el comportamiento de la
    otra.

Zona 1: Variables Predictoras vs. TARGET

Aquí buscas responder: ¿Esta variable me sirve para predecir lo que
quiero
estimar?

  - ¿Qué buscas aquí? Quieres números lo más alejados de 0 posible (tanto
    fuertemente positivos como fuertemente negativos).
      - Un valor alto positivo indica un factor de riesgo o acelerador
      directo.
      - Un valor alto negativo indica un factor protector o freno directo.
      Ambos
        son igual de útiles para el algoritmo.
  - ¿Cuándo es "malo" aquí?
      - Cuando el valor está muy cerca de 0.00.
      - Significa que esa variable, de forma aislada, apenas tiene poder
      de
        asociación con el target. Si todas las variables de un dataset
        tuvieran
```

```
        correlación cercana a 0 con el objetivo, el modelo difícilmente
        aprendería un patrón útil.

Zona 2: Variables Predictoras ENTRE SÍ

Aquí buscas responder: ¿Tengo variables repetidas que aportan la misma
información?

  - ¿Qué buscas aquí? En general, en Machine Learning, quieres números
  cercanos
    a 0 (variables independientes entre sí). Esto garantiza que cada
    columna
    aporta un dato nuevo y distinto al conjunto.
  - ¿Cuándo es "malo" aquí?
      - Cuando dos variables predictoras tienen una correlación muy alta
      entre
        sí (típicamente superior a 0.80 o inferior a -0.80). Este
        fenómeno se
        conoce como Multicolinealidad.
      - ¿Por qué es un problema la multicolinealidad?
        1.  Confunde a ciertos modelos (especialmente lineales como
        Regresión
            Lineal o Logística): Los algoritmos no pueden distinguir a
            cuál de
            las dos variables atribuirle el peso en la predicción,
            volviendo los
            coeficientes matemáticos muy inestables.
        2.  Introduce redundancia innecesaria: Estás duplicando
        información sin
            ganar capacidad predictiva, aumentando el coste computacional
            del
            modelo.

Resumen visual: ¿Qué buscar en cada parte de la matriz?

| Zona de la Matriz         | Lo ideal (Lo que quieres ver)
| Lo problemático (Lo que debes vigilar)
|
| :------------------------ |
:-------------------------------------------------------------------------------------
|
:-------------------------------------------------------------------------------------
|
| **Variable vs. Target**   | **Lejos de 0** (valores altos positivos o
negativos, ej. $> 0.25$ o $< -0.25$). Indican buena capacidad para
predecir el objetivo. | **Cerca de 0** (ej. entre $-0.05$ y $0.05$). La
variable no aporta información relevante de forma individual para
predecir el target. |
```

```
| **Variable vs. Variable** | **Cerca de 0** (habitualmente entre $-0.30$
y $0.30$). Cada variable mide aspectos distintos e independientes.
| **Muy cerca de 1 o -1** (magnitudes $> 0.80$). Hay variables
redundantes (**Multicolinealidad**) que pueden desestabilizar el modelo.
|
```

Análisis

Se acepta la explicación de la tabla de correlación y la explicación de valores a tener en cuenta entre variables vs variables y variables vs target.

Prompt

```
# ------------------------------------------------------------------------------
# 1. MATRIZ DE CORRELACIÓN DE SPEARMAN (NUMÉRICAS + TARGET)
# ------------------------------------------------------------------------------
plt.figure(figsize=(18, 14))
corr_matrix = df[NUM_COLS + [TARGET_COL]].corr(method="spearman")
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

sns.heatmap(
    corr_matrix,
    mask=mask,
    cmap="vlag",
    center=0,
    annot=True,
    fmt=".2f",
    annot_kws={"size": 8},
    linewidths=0.5,
    cbar_kws={"shrink": 0.8},
)
plt.title(
    "Matriz de Correlación de Spearman (Variables Numéricas y Target)",
    fontsize=14,
    fontweight="bold",
    pad=15,
)
plt.show()

# ------------------------------------------------------------------------------
# 2. TOP VARIABLES NUMÉRICAS CON MAYOR PODER DISCRIMINATORIO EMPÍRICO
# ------------------------------------------------------------------------------
# Calculamos la correlación de Spearman de cada variable numérica con el target
# y seleccionamos las 8 de mayor magnitud absoluta (positivas o protectoras/negativas)
corr_with_target = df[NUM_COLS].corrwith(df[TARGET_COL], method="spearman")
top_features = corr_with_target.abs().sort_values(ascending=False).head(8).index

print("=" * 70)
print(
    "TOP 8 VARIABLES NUMÉRICAS MÁS ASOCIADAS EMPÍRICAMENTE AL RIESGO CARDIOVASCULAR"
)
print("=" * 70)
for feat in top_features:
    r_val = corr_with_target[feat]
    tipo = "Factor de Riesgo (+)" if r_val > 0 else "Factor Protector (-)"
    print(f" • {feat:28s}: Spearman r = {r_val:+.3f}  ({tipo})")

# Visualización mediante Boxplots estratificados por nivel de riesgo
fig, axes = plt.subplots(2, 4, figsize=(18, 11))
axes = axes.flatten()

class_labels_short = ["0 (Bajo)", "1 (Moderado)", "2 (Alto)"]

for idx, col in enumerate(top_features):
```

```
    r_val = corr_with_target[col]
    sns.boxplot(
        x=TARGET_COL,
        y=col,
        data=df,
        ax=axes[idx],
        palette="Blues",
        hue=TARGET_COL,
        legend=False,
    )
    axes[idx].set_title(
        f"{col}\n(Spearman r = {r_val:+.2f})", fontsize=11, fontweight="bold"
    )
    axes[idx].set_xlabel("Nivel de Riesgo Cardiovascular")
    axes[idx].set_xticklabels(class_labels_short)
    axes[idx].set_ylabel(col)

plt.suptitle(
    "Distribución de las 8 Variables Numéricas más Correlacionadas empíricamente",
    y=1.02,
    fontsize=14,
    fontweight="bold",
)
plt.tight_layout()
plt.show()
```

No es un poco innecesario este codigo por volver a calcular las correlacion de spearman si ya tienes la matriz de correlación de antes? o es mejor porque es más preciso?

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Tienes toda la razón: es completamente innecesario y redundante. No es más
preciso en absoluto; el cálculo matemático y el resultado numérico son exactamente los mismos.

¿Por qué es redundante?

En el paso 1 ya calculaste la matriz completa:

```
corr_matrix = df[NUM_COLS + [TARGET_COL]].corr(method="spearman")
```

Dentro de corr_matrix, la columna (o fila) correspondiente a TARGET_COL ya
contiene exactamente la correlación de Spearman de cada variable numérica con el
target.

Volver a ejecutar df[NUM_COLS].corrwith(...) en el paso 2 obliga a pandas a
volver a ordenar (rankear) todos los datos y recalcular las operaciones matemáticas desde cero, lo cual:

1. Desperdicia tiempo de cómputo (Spearman requiere ordenar los datos, lo que
   en datasets medianos/grandes es costoso).

```
2.  Viola el principio DRY (Don't Repeat Yourself).

Cómo simplificarlo

Puedes reemplazar esa línea extrayendo directamente la serie de la matriz
que ya
calculaste:

#
------------------------------------------------------------------------------
# 2. TOP VARIABLES NUMÉRICAS CON MAYOR PODER DISCRIMINATORIO EMPÍRICO
#
------------------------------------------------------------------------------
# En lugar de recalcular con .corrwith():
# corr_with_target = df[NUM_COLS].corrwith(df[TARGET_COL],
method="spearman")

# Simplemente extraes la columna del target para las variables numéricas:
corr_with_target = corr_matrix.loc[NUM_COLS, TARGET_COL]

top_features =
corr_with_target.abs().sort_values(ascending=False).head(8).index

El resto de tu código funcionará exactamente igual, pero de forma más
limpia,
idiomática y eficiente.
```

Análisis

Al notar que la correlación se calculaba dos veces y dudar de si aportaba mayor precisión, se confirmó con esta consulta que el segundo cálculo era redundante e innecesario, pasando a extraer los valores directamente de la matriz previa para simplificar el código.

Prompt

```python
for idx, col in enumerate(top_features):
    r_val = corr_with_target[col]
    sns.boxplot(
        x=TARGET_COL,
        y=col,
        data=df,
        ax=axes[idx],
        palette= "Blues",
        hue=TARGET_COL,
        legend=False,
    )
```

Haz que la paleta sea roja si la correlación es positiva y azul si es negativa.

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Aquí tienes el bucle modificado. Puedes asignar dinámicamente la paleta ("Reds" si es positiva o "Blues" si es negativa) evaluando el valor de r_val:

```
for idx, col in enumerate(top_features):
    r_val = corr_with_target[col]

    # Paleta dinámica: Rojo si aumenta el riesgo (+), Azul si es
    protector (-)
    palette_color = "Reds" if r_val > 0 else "Blues"

    sns.boxplot(
        x=TARGET_COL,
        y=col,
        data=df,
        ax=axes[idx],
        palette=palette_color,
        hue=TARGET_COL,
        legend=False,
    )
    axes[idx].set_title(
        f"{col}\n(Spearman r = {r_val:+.2f})", fontsize=11,
        fontweight="bold"
    )
    axes[idx].set_xlabel("Nivel de Riesgo Cardiovascular")
    axes[idx].set_xticklabels(class_labels_short)
    axes[idx].set_ylabel(col)
```

¿Qué hace este cambio?

- Si r > 0 (Factor de Riesgo): utiliza la gama "Reds", donde los niveles de
  riesgo van subiendo en tonos de rojo.
- Si r \le 0 (Factor Protector): utiliza la gama "Blues", manteniendo la
  escala de azules.

Análisis

Se acepta la sugerencia, pero sin hacer la variable auxiliar.

Prompt

Tengo una base de datos sobre Predicción Multiclase de Riesgo Cardiovascular en
Población Adulta para clasificar a los pacientes en tres niveles de riesgo
cardiovascular (Clase 0: Bajo Riesgo, Clase 1: Riesgo Moderado y Clase 2: Alto
Riesgo). Las variables son:

edad (Numérico): Edad del paciente expresada en años cumplidos.

presion_sistolica (Numérico): Presión arterial sistólica medida en reposo (mmHg).

presion_diastolica (Numérico): Presión arterial diastólica medida en reposo
(mmHg).

colesterol_total (Numérico): Nivel de colesterol total en suero (mg/dL).

colesterol_hdl (Numérico): Nivel de colesterol de alta densidad o 'colesterol
bueno' (mg/dL).

colesterol_ldl (Numérico): Nivel de colesterol de baja densidad o 'colesterol
malo' (mg/dL).

trigliceridos (Numérico): Concentración sérica de triglicéridos (mg/dL).

glucosa_ayunas (Numérico): Nivel de glucosa en sangre tras ayuno nocturno (mg/dL).

indice_masa_corporal (Numérico): Índice de masa corporal calculado como peso/talla^2 (kg/m^2).

frecuencia_cardiaca_reposo (Numérico): Frecuencia cardíaca medida en reposo
(latidos por minuto).

horas_sueno_diarias (Numérico): Promedio de horas de descanso nocturno declaradas por día.

consumo_alcohol_semanal (Numérico): Unidades de bebida estándar de alcohol
consumidas semanalmente.

anos_fumador (Numérico): Tiempo acumulado de exposición al tabaco expresado en
años.

proteina_c_reactiva (Numérico): Concentración de proteína C reactiva ultrasensible (mg/L).

acido_urico (Numérico): Nivel sérico de ácido úrico (mg/dL).

clearance_creatinina (Numérico): Tasa de filtrado glomerular / depuración de
creatinina (mL/min).

```
frecuencia_ejercicio_semanal (Numérico): Días a la semana con actividad
física
moderada o intensa.

perimetro_cintura (Numérico): Circunferencia abdominal medida en
centímetros.

hemoglobina_glicosilada (Numérico): Porcentaje de hemoglobina glicosilada
HbA1c
en sangre (%).

fumador_actual (Booleano): Indica el hábito tabáquico activo (1: Sí, 0:
No).

hipertension_previa (Booleano): Diagnóstico previo confirmado de
hipertensión
arterial (1: Sí, 0: No).

diabetes_mellitus (Booleano): Diagnóstico previo de diabetes mellitus
tipo 2 (1:
Sí, 0: No).

tratamiento_estatinas (Booleano): Uso habitual de fármacos
hipolipemiantes /
estatinas (1: Sí, 0: No).

uso_antihipertensivos (Booleano): Uso habitual de medicamentos para el
control
presivo (1: Sí, 0: No).

tipo_alimentacion (Nominal): Patrón dietético principal del paciente.
Valores
posibles: 'Mediterránea', 'Occidental', 'Vegetariana', 'Baja en
carbohidratos'.

nivel_actividad (Nominal): Grado habitual de actividad física diaria.
Valores
posibles: 'Sedentario', 'Ligeramente activo', 'Moderadamente activo',
'Muy
activo'.

antecedentes_familiares_tipo (Nominal): Presencia de eventos
cardiovasculares en
familiares de primer grado. Valores posibles: 'Ninguno', 'Materno',
'Paterno',
'Ambos'.

riesgo_cardiovascular (Objetivo): Variable target de clasificación en 3
categorías (0: Bajo riesgo, 1: Riesgo moderado, 2: Alto riesgo).
```

Quiero que me digas valores en los que suelen estar estas variables y qué errores sería conveniente revisar.

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Para preparar adecuadamente esta base de datos antes de entrenar un modelo de
Machine Learning, es fundamental validar tanto los rangos clínicos plausibles
como la coherencia lógica entre variables.

A continuación, tienes una guía detallada dividida en dos partes: rangos de
referencia y errores/inconsistencias clave a revisar.

1. Rangos de Valores Típicos y Plausibles

A continuación se presentan los valores divididos en rangos normales, rangos
habituales en la práctica clínica y límites extremos que indicarían un posible
error de captura:

| Variable | Rango Normal / Óptimo
| Rango Típico en Adultos | Alerta: Posible Error de Datos
|
| :---------- | :----------
| :---------- |
:---------- |
| **edad** | 18 - 65 años
| 18 - 95 años | < 18 (si es solo adultos) o > 110
|
| **presion_sistolica** | 90 - 120 mmHg
| 100 - 180 mmHg | < 70 (shock) o > 240 mmHg
|
| **presion_diastolica** | 60 - 80 mmHg
| 60 - 110 mmHg | < 40 o > 140 mmHg
|
| **colesterol_total** | < 200 mg/dL
| 130 - 320 mg/dL | < 80 o > 500 mg/dL
|
| **colesterol_hdl** | > 40 (hombres) / > 50
(mujeres) | 25 - 85 mg/dL | < 15 o > 120 mg/dL
|
| **colesterol_ldl** | < 100 mg/dL
| 70 - 220 mg/dL | < 30 o > 350 mg/dL
|

```
| **trigliceridos**                  | $< 150$ mg/dL
| 60 - 350 mg/dL          | $< 30$ o $> 1000$ mg/dL
|
| **glucosa\_ayunas**                | 70 - 99 mg/dL
| 70 - 250 mg/dL          | $< 40$ (hipoglucemia severa) o $> 400$ mg/dL
|
| **indice\_masa\_corporal**         | 18.5 - 24.9 kg/m²
| 18 - 45 kg/m²           | $< 13$ o $> 65$ kg/m²
|
| **frecuencia\_cardiaca\_reposo**   | 60 - 100 lpm
| 50 - 110 lpm            | $< 35$ o $> 180$ lpm
|
| **horas\_sueno\_diarias**          | 7 - 9 horas
| 4 - 10 horas            | $< 2$ o $> 16$ horas
|
| **consumo\_alcohol\_semanal**      | 0 - 7 (mujeres) / 0 - 14 (hombres)
| 0 - 30 UBEs\*           | $< 0$ o $> 80$ UBEs semanales
|
| **anos\_fumador**                  | 0 años
| 0 - 50 años             | $< 0$ o mayor que `edad - 10`
|
| **proteina\_c\_reactiva (PCR)**    | $< 1.0$ mg/L (bajo riesgo)
| 0.1 - 10 mg/L           | $> 20$ mg/L (suele indicar infección aguda,
no riesgo vascular basal) |
| **acido\_urico**                   | 3.5 - 7.2 mg/dL
| 2.5 - 10 mg/dL          | $< 1.5$ o $> 15$ mg/dL
|
| **clearance\_creatinina**          | 90 - 130 mL/min
| 30 - 140 mL/min         | $< 5$ o $> 200$ mL/min
|
| **frecuencia\_ejercicio\_semanal** | 3 - 5 días
| 0 - 7 días              | $< 0$ o $> 7$ días
|
| **perimetro\_cintura**             | $< 80$ (mujeres) / $< 94$
(hombres) | 65 - 130 cm             | $< 45$ o $> 180$ cm
|
| **hemoglobina\_glicosilada**       | 4.0% - 5.6%
| 4.5% - 11%              | $< 3.5\%$ o $> 16\%$
|

*UBE = Unidad de Bebida Estándar (aprox. 1 caña de cerveza, 1 copa de
vino o 1
chupito).

2. Errores de Coherencia Cruzada (Cruciales para Limpieza)

En variables médicas, los datos pueden estar individualmente en rango,
pero ser
fisiológicamente imposibles o contradictorios al compararlos entre sí:
```

```
A. Presión Arterial

  - Error crítico: presion_diastolica >= presion_sistolica. La sistólica
  siempre
    debe ser superior.
  - Presión de pulso no plausible: La diferencia (presion_sistolica -
    presion_diastolica) rara vez es menor a 20 mmHg o mayor a 100 mmHg en
    reposo.

B. Coherencia en el Perfil Lipídico

  - Ecuación de Friedewald: En condiciones normales, el colesterol total
  se
    aproxima con:
    \text{Colesterol Total} \approx \text{LDL} + \text{HDL} +
    (\text{Triglicéridos} / 5)
    Si la suma de \text{LDL} + \text{HDL} supera ampliamente al
    colesterol_total, hay un error de registro.

C. Tabaco y Edad

  - anos_fumador >= edad: Imposible.
  - anos_fumador > (edad - 10): Muy poco probable (indicaría que empezó a
  fumar
    con menos de 10 años).
  - Contradicción de hábito: Pacientes con fumador_actual == 0 pero
  anos_fumador
    > 0 representan ex-fumadores (esto es correcto, pero conviene
    definirlo);
    sin embargo, fumador_actual == 1 con anos_fumador == 0 suele ser un
    error (a
    menos que haya empezado hace menos de un año).

D. Diabetes y Glucemia

  - Si diabetes_mellitus == 0, pero hemoglobina_glicosilada >= 6.5% y
    glucosa_ayunas >= 126 mg/dL, es probable que el paciente sea un
    diabético no
    diagnosticado o haya un error en la etiqueta binaria.
  - Si diabetes_mellitus == 1, pero tiene glucosa_ayunas y
    hemoglobina_glicosilada en rangos extremadamente bajos sin
    medicación,
    convendría revisarlo.

E. Hipertensión y Medicación

  - Pacientes con uso_antihipertensivos == 1 que tienen
  hipertension_previa == 0
    (raro, salvo uso off-label para otras patologías como migraña o
    insuficiencia cardíaca).
```

```
F. Actividad Física

  - Inconsistencia entre nivel_actividad == 'Sedentario' y
    frecuencia_ejercicio_semanal >= 5.
  - Inconsistencia entre nivel_actividad == 'Muy activo' y
    frecuencia_ejercicio_semanal == 0.

G. Antropometría (IMC vs. Cintura)

  - Un indice_masa_corporal > 35 (obesidad moderada/severa) acompañado de
  un
    perimetro_cintura < 70 cm indica casi con seguridad un error en las
    unidades
    (por ejemplo, registrar la cintura en pulgadas en lugar de
    centímetros).

3. Buenas Prácticas adicionales para tu Modelo

1.  Codificación de Variables Nominales:
      - antecedentes_familiares_tipo y tipo_alimentacion deben tratarse
      con
        One-Hot Encoding.
      - nivel_actividad tiene un orden intrínseco (Sedentario \to
      Ligeramente
        \to Moderadamente \to Muy activo), por lo que un Ordinal Encoding
        (0, 1, 2, 3) preservará mejor esa jerarquía.
2.  Distribución del Target (riesgo_cardiovascular):
      - Comprueba el desbalance de clases. En salud, la Clase 2 (Alto
      Riesgo)
        suele tener muchas menos muestras que la Clase 0. Si hay
        desbalance
        severo, evalúa métricas como Macro F1-Score o PR-AUC multiclase
        en lugar
        de Accuracy, y utiliza técnicas como SMOTE o ponderación de
        clases
        (class_weight='balanced').
3.  Valores Nulos Encubiertos:
      - Busca números como -1, 999 o 0 en variables como
      clearance_creatinina,
        horas_sueno_diarias o presion_sistolica, que a veces se
        introducen en
        bases de datos hospitalarias para indicar "no registrado".
```

Análisis

En cuanto a esta propuesta inicial de la IA para detectar problemáticas y establecer algunos rangos para detectar valores erróneos o atípicos, hay opinión dividida. Por una parte, se aceptan y se toman en cuenta algunas propuestas sobre valores y rangos a comprobar. Algunos ya los habíamos tenido en cuenta, como puede ser

comprobar si hay menores de edad en los ejemplos o comprobar que el consumo de alcohol semanal no es negativo, pero algunos otros son propuestas interesantes que no se habían tenido en cuenta, aunque falta comprobar si comprobación tienen sentido. También revela una fórmula por la cual se relacionan los dos tipos de colesterol y el colesterol total, revelando que también la columna *trigliceridos* actúa. Sin embargo, se descartan algunas propuestas como marcar ciertos valores extremos como errores, ya que eliminar valores atípicos pueden ser valores valiosos para poder aprender a detectar el riesgo alto, y más si es un problema en el ámbito de la medicina. Habrá que determinar qué valores atípicos conservar.

Prompt

Muchas de las variables que me has indicado como una alerta de posible error,si estoy haciendo un modelo para predecir el riesgo cardiovascular, no crees que son interesantes mantenerlas porque esos atípicos podrían ayudar a ser el detonante para detectar el riesgo?

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Es una observación completamente acertada y muy perspicaz desde el punto
de
vista clínico. En medicina, los eventos graves (infartos, ictus) suelen
ocurrir
precisamente en la "cola" de la distribución: el paciente que tiene 210
mmHg de
tensión sistólica o 9.5% de hemoglobina glicosilada es exactamente el que
debe
clasificarse como Alto Riesgo (Clase 2).

Eliminar estos valores a ciegas ("tratar los outliers" de forma estándar)
arruinaría la capacidad predictiva del modelo.

Sin embargo, hay una diferencia crítica entre dos tipos de valores
extremos. El
objetivo no es borrar datos por ser altos, sino distinguir el riesgo
extremo del
"ruido biológicamente imposible" o del "ruido de confusión".

Aquí tienes cómo diferenciar lo que debes conservar de lo que debes
corregir o
descartar:

1. Lo que DEBES mantener (Atípicos Clínicos / Señales de Alto Riesgo)

Son valores muy elevados, pero compatibles con la vida y directamente
causantes
de daño cardiovascular. No los elimines ni los recortes
(capping/winsorizing) si
```

```
vas a usar modelos basados en árboles:

  - Presión sistólica de 190 - 220 mmHg: Es una crisis o urgencia
  hipertensiva.
    Es un predictor brutal de riesgo cardiovascular inminente.
  - Triglicéridos de 500 - 1000 mg/dL: Se asocia a hipertrigliceridemia
  severa
    (y riesgo de pancreatitis además de cardiovascular).
  - Glucosa en ayunas de 250 - 350 mg/dL o HbA1c de 11 - 13%: Diabetes
    gravemente descompensada; genera aterosclerosis acelerada.
  - Clearance de creatinina de 15 - 30 mL/min: Insuficiencia renal
  avanzada; la
    enfermedad renal crónica es uno de los mayores multiplicadores de
    riesgo
    cardiovascular conocidos.
  - IMC de 45 - 55: Obesidad mórbida grado III, factor de riesgo masivo.

2. Lo que DEBES corregir o eliminar (Errores que engañan al algoritmo)

Aquí el valor atípico no representa "más riesgo", sino un fallo en la
toma o
registro del dato. Si dejas estos valores, el modelo aprenderá patrones
absurdos:

  - Incompatibilidad física o biológica:
      - Una presion_sistolica de 320 mmHg o 30 mmHg en un paciente
      ambulatorio.
        A 320 revienta una arteria cerebral al instante; a 30 la persona
        está en
        parada cardiorrespiratoria, no haciéndose un chequeo de
        prevención.
      - presion_diastolica (p. ej., 110) mayor que la sistolica (p. ej.,
      90):
        Error humano al teclear la barra espaciadora o intercambiar las
        columnas.
  - Confusión de escalas/unidades (Muy común):
      - glucosa_ayunas con valor 6.5: El médico anotó el dato en mmol/L
      (usado
        en Europa/EE.UU.) en lugar de mg/dL (debería ser ~117 mg/dL). El
        modelo
        interpretará que el paciente tiene glucosa 6.5 mg/dL (estaría
        muerto de
        hipoglucemia).
      - perimetro_cintura de 36: Medido en pulgadas, no en centímetros
      (36
        pulgadas son ~91 cm).
  - Datos "centinela" o nulos camuflados:
      - clearance_creatinina = -1 o 999. Si el algoritmo lo toma como
      número
```

```
        continuo, un 999 parecerá un riñón superhumano cuando en realidad
        era un
        dato faltante.

3. El caso especial de la PCR (Proteína C Reactiva): El "Ruido de
Confusión"

La proteina_c_reactiva es el mejor ejemplo de por qué hay que tener
cuidado:

  - En riesgo cardiovascular se utiliza la PCR ultrasensible (PCR-us):
      - Bajo riesgo: < 1 mg/L
      - Riesgo moderado: 1 - 3 mg/L
      - Riesgo alto: > 3 mg/L (hasta unos 10 mg/L)
  - ¿Qué pasa si un paciente tiene PCR de 80 mg/L? No significa que tenga
  8
    veces más riesgo cardíaco que uno de 10. Significa que ese día tenía
    una
    neumonía, una infección bacteriana o una apendicitis. Si dejas ese
    valor sin
    contexto, el modelo puede asumir que una inflamación sistémica aguda
    es la
    causa primaria de riesgo cardiovascular a largo plazo, distorsionando
    el
    peso de la variable. Clínicamente, ante una PCR > 20 mg/L, la
    analítica
    cardiovascular suele descartarse y repetirse semanas después.

¿Cómo manejar esto con Machine Learning?

1.  Usa modelos robustos a valores extremos: Si mantienes estos valores
atípicos
    reales (como 210 mmHg de tensión o glucosas de 300), no utilices
    Regresión
    Logística o Redes Neuronales sin escalar, ya que los extremos
    desestabilizan
    los gradientes y coeficientes. Utiliza modelos basados en árboles de
    decisión (XGBoost, LightGBM, Random Forest), ya que a estos modelos
    solo les
    importa el orden de partición (>180 ya es alto, da igual si es 185 o
    240,
    caerán en la misma rama).
2.  Tu filtro no debe ser estadístico (Z\text{-score} > 3), debe ser de
    plausibilidad fisiológica: No quites un dato solo porque se aleje de
    la
    campana de Gauss. Quítalo únicamente si el valor es incompatible con
    que el
    paciente haya llegado caminando a la consulta.
```

Análisis

La información que aporta no es suficiente para detectar valores erroneos ni tomar decisiones en el dataset, es mejor investigar y consultar individualmente cada variable. Sin embargo, la consulta sirve para confirmar que no se debe eliminar los valores atípicos de forma estándar porque arruinaría la capacidad predictiva del modelo.

Prompt

colesterol hdl valores puede ser negativo?

Respuesta IA `Gemini Flash-Lite 3.5 en Gemini`

No, el valor del colesterol HDL (conocido como colesterol "bueno") no puede ser negativo.
• Valores mínimos lógicos: Los niveles de colesterol HDL en la sangre se miden en miligramos por decilitro (mg/dL) y el límite físico inferior es de 0 mg/dL, ya que no existe una cantidad menor de cero.
• Valores considerados bajos:
• En hombres: menos de 40 mg/dL (y se considera muy bajo por debajo de 35 mg/dL).
• En mujeres: menos de 50 mg/dL (y se considera muy bajo por debajo de 40 mg/dL).
• Si has visto un número negativo o extraño en una analítica: Generalmente se debe a un error de transcripción, un fallo en la lectura del equipo del laboratorio, un formato de celda alterado o un código de error informático, y no a que tu organismo tenga un nivel negativo de lipoproteínas.
Consulta con tu médico o con el laboratorio que realizó el análisis para que repitan la lectura o te aclaren la cifra exacta de tu perfil lipídico. Si te han entregado unos análisis recientes, dime:
• ¿Qué valor numérico exacto te aparece?
• ¿Hay otros valores alterados (como los triglicéridos o el colesterol LDL)?
Así podré orientarte mejor sobre lo que significa tu resultado.

Análisis

Al encontrarse este valor negativo y dudar de su validez, se aceptó la afirmación de que el colesterol HDL no puede ser biológicamente negativo, estableciendo correctamente el límite inferior en 0 mg/dL.

Prompt

y los triglicéridos pueden ser negativos?

Respuesta IA Gemini Flash-Lite 3.5 en Gemini

No, los triglicéridos no pueden ser negativos. En una analítica de sangre, los resultados miden la cantidad de una sustancia real presente en el organismo, por lo que el valor mínimo posible es cero (0 mg/dL), aunque un nivel de cero absoluto tampoco es biológicamente realista. Si ves un signo de resta o un valor negativo en tu informe, generalmente se debe a una de las siguientes razones:

- Es un guion o símbolo de separación: A veces, el sistema del laboratorio utiliza un guion antes del número o para separar el nombre de la prueba de su resultado (por ejemplo: Triglicéridos - 85 mg/dL), lo que puede confundirse con un signo "menos".
- Un error en el informe impreso: Puede tratarse de una errata tipográfica o un fallo de formato en la visualización del documento.
- Error de laboratorio: En casos extremadamente raros, si el cálculo se realiza de forma automatizada mediante ciertas fórmulas indirectas y otros valores están muy descompensados, el sistema podría arrojar un error técnico.

Para ayudarte a salir de dudas, ¿podrías transcribir exactamente cómo aparece escrito el valor en tu papel, o indicarme el resto de los resultados (como el colesterol total, LDL y HDL)?

Análisis

Al encontrarse este valor negativo y dudar de su validez, se aceptó la afirmación de que los triglicéridos no pueden ser biológicamente negativos, estableciendo correctamente el límite inferior en 0 mg/dL. La IA plantea que podría haberse colado un signo menos. Esta hipótesis se podría tener en cuenta a la hora de decidir qué estrategia aplicr con las variables cuyos valores no podían ser negativos, si se determina que realmente ha sido un error al escribir los datos.

Prompt

rango típico de presión sistólica y presión dialótica

Respuesta IA Gemini Flash-Lite 3.5 en Gemini

El rango típico o normal de la presión arterial en adultos se sitúa por debajo de 120/80 mm Hg (milímetros de mercurio).

- Presión sistólica (el número superior): Representa la fuerza de la sangre contra las paredes de las arterias cuando el corazón se contrae. Un valor normal o saludable es menor de 120 mm Hg (típicamente entre 90 y 120 mm Hg).
- Presión diastólica (el número inferior): Mide la presión en las arterias cuando el corazón se relaja entre latidos. Un valor normal o saludable es menor de 80 mm Hg (típicamente entre 60 y 80 mm Hg).

Clasificación general (según guías clínicas como la AHA/ACC)

- Normal: Menor de 120 / Menor de 80 mm Hg
- Elevada: Sistólica entre 120 y 129 y diastólica menor de 80 mm Hg • Hipertensión (Etapa 1): Sistólica entre 130 y 139 o diastólica entre 80 y 89 mm Hg
- Hipertensión (Etapa 2): Sistólica de 140 mm Hg o más, o diastólica de 90 mm Hg o más

• Crisis hipertensiva: Valores superiores a 180 / 120 mm Hg (requiere atención médica inmediata)
Puedes consultar más detalles y herramientas informativas en la guía oficial de MedlinePlus en español o revisar la tabla explicativa del Manual MSD.
Si lo deseas, dime:
• ¿Es una consulta para un adulto joven, de mediana edad o mayor?
• ¿Hay síntomas (como mareos o dolor de cabeza) o medicación de por medio?
Puedo darte información más específica sobre cómo se interpretan las lecturas.

Análisis

Los datos que aporta son correctos dadas las fuentes de las que lo ha sacado, pero no me despeja la duda suficiente para saber si los valores mínimos y máximos en estas dos presiones se podrían dar en la realidad.

Prompt

podría ocurrir que al medir la presión sistólica de un paciente los valores fuesen 52 o 231? y que la presión distólica fuese 30 o 122? O son valores que no se pueden dar en la realidad? Están en mmHg.

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

Sí, todos esos valores son posibles en la realidad y representan situaciones clínicas reales de extrema gravedad (tanto por ser peligrosamente bajas como críticamente altas). No son meros errores de medición, aunque siempre se deben confirmar.
La explicación médica para cada uno de estos extremos en mm Hg es la siguiente:
Valores de Presión Sistólica
• 52 mm Hg (Posible y crítico): Este valor representa una hipotensión extrema. En la práctica médica, se asocia directamente con un estado de shock (como el shock séptico, cardiogénico o hemorrágico), donde el corazón no tiene la fuerza suficiente o no hay suficiente volumen de sangre para mantener la presión. Es una emergencia médica de código rojo, ya que los órganos vitales no reciben suficiente oxígeno.
• 231 mm Hg (Posible y crítico): Este valor representa una crisis hipertensiva severa. Ocurre en pacientes con hipertensión grave descontrolada o durante eventos médicos agudos (como un ictus o una disección aórtica). Valores tan altos ponen al paciente en riesgo inminente de daño orgánico (infarto, derrame cerebral o fallo renal) y requieren atención médica inmediata para bajar la presión de forma controlada.
Valores de Presión Diastólica
• 30 mm Hg (Posible y crítico): Una presión diastólica tan baja se puede observar en casos de insuficiencia aórtica severa (donde la válvula del corazón no cierra bien y la sangre regresa), en estados de vasodilatación extrema (como en una reacción alérgica grave o anafilaxia) o en shock séptico.
• 122 mm Hg (Posible y crítico): Este valor se sitúa firmemente dentro de una crisis hipertensiva (cualquier cifra diastólica mayor de 120 mm Hg lo es). Indica que las arterias están soportando una resistencia y una tensión masivas incluso cuando el corazón está en reposo, lo que fatiga gravemente el sistema cardiovascular.

¿Qué se hace en la realidad ante estos valores?
Cuando un tensiómetro digital o manual muestra cifras tan extremas, el protocolo médico exige:
1. Verificar el estado del paciente: Si el paciente está inconsciente, mareado o tiene dolor en el pecho, el valor suele ser real.
2. Repetir la medición: Se vuelve a medir (a menudo en el otro brazo o con un equipo manual calibrado) para descartar que el brazalete estuviera mal colocado, que el paciente se moviera o que el aparato tuviera la batería baja. Si estás analizando un caso concreto, cuéntame:
• ¿El paciente presenta síntomas actuales como mareos, desmayos, dolor de cabeza o dolor en el pecho?
• ¿Se trata de un ejercicio académico/simulación o de una lectura real? Así podré darte información más orientada a lo que necesitas.

Análisis

Al hacer la consulta sobre los valores extremos, la IA confirmó que dichas cifras son biológicamente viables en escenarios clínicos críticos, por lo que se confirma que no se descarta ningún valor para la presión sistólica y diastólica.

Prompt

rango de colesterol hdl y colesterol ldl, cuando es preocupante o imposible

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

Los rangos de colesterol en sangre se miden en miligramos por decilitro (mg/dL).
Colesterol LDL (el ”malo”)
• Nivel óptimo: Menos de 100 mg/dL.
• Casi óptimo o aceptable: 100 – 129 mg/dL.
• Límite alto: 130 – 159 mg/dL.
• Alto: 160 – 189 mg/dL.
• Muy alto / Preocupante: 190 mg/dL o más (requiere atención médica inmediata y a menudo tratamiento con estatinas, ya que eleva drásticamente el riesgo cardiovascular).
• Metas estrictas: En personas con alto riesgo cardiovascular o que ya han sufrido un infarto, los médicos buscan tener un LDL por debajo de 70 mg/dL o incluso menor de 55 mg/dL.
Colesterol HDL (el ”bueno”)
• Bajo (preocupante):
• Menos de 40 mg/dL en hombres.
• Menos de 50 mg/dL en mujeres.
• Aceptable / Normal: 40–59 mg/dL en hombres y 50–59 mg/dL en mujeres.
• Nivel protector (óptimo): 60 mg/dL o más (actúa como factor protector contra enfermedades del corazón).
• Nota sobre extremos: Aunque el HDL es ”bueno”, niveles extremadamente altos (por encima de 90-100 mg/dL de forma patológica) no necesariamente otorgan más

protección y a veces se asocian con alteraciones genéticas raras.
¿Cuándo es preocupante o ïmposible¿
• Cuándo es preocupante:
• Un LDL muy elevado (190 mg/dL) o un HDL muy bajo combinado con factores de riesgo como tabaquismo, hipertensión, diabetes o antecedentes familiares de infarto precoz.
• Un LDL moderadamente alto que no baja a pesar de mejorar la dieta y hacer ejercicio.
• ¿Existen valores ïmposibles¿
• Cero de LDL: No es imposible biológicamente; de hecho, los recién nacidos tienen niveles muy bajos de LDL (alrededor de 30-40 mg/dL) y el cuerpo puede funcionar con niveles reducidos si se toman fármacos potentes (como inhibidores de PCSK9 o estatinas), aunque en la práctica clínica un LDL de 0 no es un objetivo natural ni habitual.
• Cero de HDL: Es incompatible con la vida a largo plazo, ya que el HDL cumple funciones esenciales de transporte de lípidos y protección vascular; tener HDL indetectable o cero absoluto indicaría un trastorno genético gravísimo (como la enfermedad de Tangier, aunque en esos casos raros quedan vestigios mínimos).
• Cifras desorbitadas de LDL (por ejemplo, superiores a 400 o 500 mg/dL) suelen deberse a hipercolesterolemia familiar homocigota o heterogénea, una condición genética seria que requiere tratamiento especializado urgente.
Puedes consultar más detalles oficiales en la guía de MedlinePlus en español.
Si lo deseas, puedes comentarme:
• ¿Tienes a la mano un análisis de sangre reciente con tus valores?
• ¿Tienes antecedentes familiares de problemas del corazón o colesterol alto?
Con esos datos, puedo explicarte de forma más personalizada qué significan tus números.

Análisis

Los datos que aporta son correctos dadas las fuentes de las que lo ha sacado. Se confirma que los valores en la base de datos no son tan dispares como para que no se puedan dar en la realidad, salvo el valor negativo, que ya se sabía que era un error, aunque añade también el cero como un error.

Prompt

rango de trigliceridos, cuando es preocupante o imposible

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

Los niveles de triglicéridos miden la cantidad de grasa en la sangre. Un nivel superior a 150 mg/dL se considera elevado, y superar los 500 mg/dL es preocupante debido al riesgo inmediato de pancreatitis.
A continuación se detallan los rangos estándar y los niveles críticos:
Rangos de triglicéridos en sangre
Nivel de Triglicéridos Clasificación Significado y Riesgos
Menos de 150 mg/dL Normal Nivel óptimo para la salud cardiovascular.

150 a 199 mg/dL Límite alto Comienza a aumentar el riesgo cardiovascular a largo plazo.
200 a 499 mg/dL Alto Requiere cambios de estilo de vida y posible tratamiento médico.
500 mg/dL o más Muy alto (Preocupante) Riesgo elevado de pancreatitis aguda, una inflamación grave del páncreas.
¿Cuándo es preocupante?
Es preocupante en dos escenarios principales:
• Mayor a 200 mg/dL de forma sostenida: Aumenta notablemente el riesgo de sufrir un infarto, un accidente cerebrovascular (ACV) o desarrollar endurecimiento de las arterias (arteriosclerosis).
• Mayor a 500 mg/dL (Hipertrigliceridemia grave): Es una emergencia médica latente. A este nivel, el riesgo principal ya no es solo el corazón, sino que el páncreas sufra un ataque inflamatorio agudo (pancreatitis), lo cual requiere hospitalización inmediata. En casos extremos, algunas personas pueden registrar valores de 1,000 a 5,000 mg/dL o más.
¿Cuándo es ïmposible¿
En medicina no existe un límite numérico superior ïmposible", pero se consideran valores extraordinarios o errores de laboratorio bajo ciertas condiciones:
• Cifras inhumanas: Valores que superen los 10,000 mg/dL son extremadamente raros y suelen estar vinculados a mutaciones genéticas graves (como el síndrome de quilomicronemia familiar). A ese nivel, la sangre adquiere un aspecto lechoso o cremoso.
• Cero absoluto (0 mg/dL): Es biológicamente imposible tener 0 mg/dL de triglicéridos, ya que el cuerpo los necesita constantemente para almacenar y transportar energía. Un valor extremadamente bajo (por debajo de 30 mg/dL) es muy raro y suele indicar desnutrición severa, malabsorción o una condición genética llamada abetalipoproteinemia.
Si cuentas con un resultado de laboratorio reciente, cuéntame: ¿qué valor exacto obtuviste y si venías guardando un ayuno de 9 a 12 horas? Así podré darte una orientación más precisa.

Análisis

Los datos que aporta son correctos dadas las fuentes de las que lo ha sacado. Se confirma que los valores en la base de datos no son tan dispares como para que no se puedan dar en la realidad, salvo el valor negativo, que ya se sabía que era un error, aunque añade también el cero como un error.

Prompt

rango glucosa en ayunas, cuando es preocupante o imposible

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

Los niveles de glucosa en sangre en ayunas determinan el estado metabólico de una persona. Un nivel en ayunas entre 70 y 99 mg/dL se considera normal, mientras

que valores persistentemente elevados indican prediabetes o diabetes.

Rangos de glucosa en ayunas

Nivel de glucosa (mg/dL) Clasificación Significado

Menos de 70 mg/dL Hipoglucemia Azúcar bajo en sangre; requiere atención inmediata.

70 a 99 mg/dL Normal Rango óptimo y saludable.

100 a 125 mg/dL Prediabetes Alerta de resistencia a la insulina; requiere cambios de estilo de vida.

126 mg/dL o más Diabetes Criterio de diagnóstico para diabetes (si se repite en dos pruebas distintas).

Cuándo es preocupante

Un valor es preocupante cuando entra en rangos extremos que ponen en riesgo inmediato la salud o indican una patología descontrolada:

- Por debajo de 55 mg/dL (Hipoglucemia severa): Es una urgencia médica inmediata. Puede provocar desmayos, convulsiones, confusión y pérdida del conocimiento.
- Por encima de 250 - 300 mg/dL (Hiperglucemia severa): Es sumamente peligroso, especialmente si se acompaña de síntomas como sed extrema, visión borrosa o náuseas. En personas con diabetes tipo 1 puede desencadenar cetoacidosis diabética, y en tipo 2, un estado hiperosmolar, ambas condiciones potencialmente mortales.

Qué se considera ïmposible.º un error de medición

En la práctica clínica o con el uso de glucómetros caseros, existen valores que son biológicamente incompatibles con la vida si se mantienen, o que sugieren un error del dispositivo:

- 0 mg/dL o valores cercanos a cero: El cerebro consume glucosa constantemente; un valor de cero es biológicamente imposible en una persona viva y consciente. Indica un fallo del glucómetro o una tira reactiva dañada.
- Por encima de 600 - 999 mg/dL: Aunque existen casos extremos de shock hiperosmolar donde se han registrado más de 1000 mg/dL, la mayoría de los glucómetros comerciales están programados para no medir más allá de 500 o 600 mg/dL (mostrando la palabra ”HI.º error). Si sale este valor en una persona sin síntomas graves, suele ser por restos de azúcar o comida en el dedo al pincharse.

Si estás analizando tus propios resultados, ¿has obtenido algún valor específico que te preocupe o estás experimentando algún síntoma como mareos, sed excesiva o cansancio extremo?

Análisis

Los datos que aporta son correctos dadas las fuentes de las que lo ha sacado. Se confirma que los valores en la base de datos no son tan dispares como para que no se puedan dar en la realidad.

Prompt

rango IMC, cuando es preocupante o imposible

Respuesta IA Gemini Flash Lite 3.5 en Gemini

El Índice de Masa Corporal (IMC) es un indicador de la relación entre el peso y la talla. El rango normal de IMC es de 18.5 a 24.9.
A continuación se detallan los rangos oficiales de la Organización Mundial de la Salud (OMS), cuándo empieza a ser preocupante y qué límites se consideran biológicamente imposibles.
Rangos oficiales del IMC (Adultos)
Clasificación Rango de IMC
Bajo peso Menos de 18.5
Peso normal 18.5 – 24.9
Sobrepeso 25.0 – 29.9
Obesidad Clase I (Moderada) 30.0 – 34.9
Obesidad Clase II (Severa) 35.0 – 39.9
Obesidad Clase III (Mórbida) 40.0 o más
Cuándo es preocupante
El IMC se vuelve preocupante cuando se entra en los extremos, ya que aumenta drásticamente el riesgo de mortalidad y enfermedades crónicas:
• Por debajo de 16.0 (Delgadez severa): Es preocupante porque compromete el sistema inmune, causa pérdida de masa ósea, debilidad muscular y puede indicar trastornos de la conducta alimentaria (como la anorexia) o enfermedades subyacentes graves.
• Por encima de 35.0 (Obesidad severa y mórbida): Es preocupante debido al alto riesgo de sufrir eventos cardiovasculares (infartos, hipertensión), diabetes tipo 2, apnea del sueño, problemas articulares severos y ciertos tipos de cáncer.
Cuándo es ïmposible.º un error de cálculo
Dado que el IMC es una fórmula matemática (Peso en kg / Estatura en metros^2), se pueden introducir números que den cualquier resultado, pero biológicamente existen límites para la supervivencia humana:
• IMC menor a 10-12: Es prácticamente imposible sobrevivir en este rango. Un IMC tan bajo significa que el cuerpo ha consumido casi todo su tejido graso y muscular, lo que provoca un fallo multiorgánico fatal.
• IMC mayor a 100-120: Aunque se han registrado casos excepcionales en la historia de la medicina de personas con obesidad extrema que han alcanzado un IMC cercano a 100 o poco más (pesando más de 400-500 kg), un valor significativamente superior a este rango suele deberse a un error al introducir los datos en la calculadora.
Limitación importante del IMC
El IMC no distingue entre grasa y músculo. Una persona muy musculosa (como un culturista) puede tener un IMC superior a 30 (clasificado como .ºbesidad") sin tener un exceso de grasa corporal. Por ello, los profesionales de la salud suelen complementarlo con la medición del perímetro de la cintura o el porcentaje de grasa corporal.
Si lo deseas, puedo ayudarte a calcular un caso específico si me proporcionas:
• El peso (en kilogramos o libras)
• La estatura (en centímetros o pies/pulgadas)
• Si es para un adulto o un menor de edad (ya que en niños se usan percentiles)

Análisis

Los datos que aporta son correctos dadas las fuentes de las que lo ha sacado. El límite máximo de la muestra (50.60) se encuentra dentro de los márgenes biológicos de la obesidad mórbida. Sin embargo, el límite inferior de la muestra presenta un valor mínimo de 3.6 de IMC, el cual está significativamente por debajo del límite sugerido por la IA (12-13). Asimismo, se evalúa la recomendación de la IA basada en los criterios de la OMS para discretizar la variable IMC en clases, cuya viabilidad se determinará según cuánto beneficie al modelo escogido.

Prompt

la presión sistólica debe de ser mayor que la presión diastólica siempre?

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

Sí, la presión sistólica (el número superior) siempre debe ser mayor que la presión diastólica (el número inferior) en condiciones normales.

- Presión sistólica: Mide la fuerza de la sangre contra las paredes de las arterias cuando el corazón se contrae (late) para bombear sangre. Como el corazón está haciendo su máximo esfuerzo de expulsión, esta cifra siempre es más alta (lo normal es menor de 120 mmHg).
- Presión diastólica: Mide la presión que queda en las arterias cuando el corazón se relaja entre un latido y otro para llenarse de sangre. Esta cifra siempre es menor (lo normal es menos de 80 mmHg).

¿Qué pasa si las presiones "se juntan.º se invierten?

- Diferencia muy pequeña (presiones juntas): Si la diferencia entre la sistólica y la diastólica es muy reducida (por ejemplo, 120/100 mmHg), indica un problema médico que requiere evaluación, ya que la sangre no fluye con la diferencia de presión adecuada entre la contracción y el reposo del corazón.
- Presión invertida (diastólica mayor que la sistólica): En la práctica clínica normal no es posible que la diastólica supere a la sistólica; si un monitor de presión arterial muestra un valor así, suele deberse a un error de medición (como un brazalete mal colocado, movimiento o un fallo en el aparato) y se debe repetir la toma en reposo. Si los síntomas persisten o te sientes mal, consulta a un profesional de la salud. Puedes revisar más detalles sobre las lecturas en la guía de MedlinePlus.

Si lo deseas, dime:

- ¿Qué valores específicos estás observando o te preocupa interpretar?
- ¿Se trata de una medición aislada o te han diagnosticado hipertensión?

Análisis

En el último prompt de Google AI Studio que comentaba varios errores a corregir o eliminar, sugiría comprobar que la presión diastólica no fuese mayor que la sistólica. Con esta consulta, y comprobando las fuentes, se confirma explícitamente que en la práctica clínica normal no es posible que se supere y que por tanto suele deberse a un error de medición.

Prompt

rango frecuencia cardiaca en reposo, cuando es preocupante o imposible

Respuesta IA Gemini Flash Lite 3.5 en Gemini

El rango normal de la frecuencia cardíaca en reposo (FCR) para adultos es de 60 a 100 latidos por minuto (lpm).
Rangos de la Frecuencia Cardíaca en Reposo
Frecuencia (lpm) Estado Qué significa
Menos de 60 Bradycardia (Baja) Normal en atletas bien entrenados o durante el sueño profundo.
60 a 100 Normal El corazón bombea de manera eficiente en un adulto promedio en reposo.
Más de 100 Taquicardia (Alta) El corazón late demasiado rápido. Puede deberse a estrés, fiebre, deshidratación o problemas de salud.
Cuándo es preocupante
Debes prestar atención o consultar a un médico en las siguientes situaciones:
• Frecuencia en reposo constantemente >100 lpm: Puede indicar problemas tiroideos, anemia o arritmias.
• Frecuencia en reposo <50-60 lpm (si no eres atleta): Especialmente si te sientes débil.
• Acompañada de síntomas: Cualquier número fuera de rango es alarmante si va junto con:
• Mareos o vértigo
• Dificultad para respirar
• Dolor o presión en el pecho
• Desmayos o síncope
Cuándo es ïmposible"(Incompatible con la vida)
Bajo condiciones normales de reposo absoluto y sin soporte médico artificial, existen límites biológicos extremos:
• 0 lpm (Asistolia): El corazón se detiene por completo. El cerebro sufre daños irreversibles en 4-6 minutos sin reanimación (RCP).
• Menos de 20-30 lpm: Un corazón humano difícilmente puede mantener el conocimiento o la vida a este ritmo de forma prolongada sin un marcapasos artificial, ya que el cerebro no recibe suficiente oxígeno.
• Más de 220-250 lpm en reposo absoluto: Frecuencias tan extremadamente altas impiden que las cámaras del corazón se llenen de sangre entre latidos. Esto provoca un colapso circulatorio inmediato (frecuente en la fibrilación ventricular), lo que equivale a un paro cardíaco.
Para darte una respuesta más precisa, ¿podrías decirme si estás midiendo tus propios latidos, qué número exacto obtuviste y si sientes algún síntoma como mareos o fatiga?

Análisis

Los datos que aporta son correctos dadas las fuentes de las que lo ha sacado. Se confirma que el límite superior (112.70) es un valor clínicamente viable, pero no

queda claro si el límite inferior (25.80), que está en el rango de los 20-30 lmp, es simplemente un valor crítico o si no se puede dar en la realidad.

Prompt

entonces, un valor entre 20-30 lpm es un valor que se le podría registrar a un paciente o es un error?

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

Un valor entre 20 y 30 lpm sí se le puede registrar a un paciente real, pero representa una bradicardia extrema y una emergencia médica absoluta. No debe asumirse automáticamente que es un error de medición, aunque siempre debe confirmarse de inmediato con un electrocardiograma (ECG) clínico o la toma manual del pulso periférico y central.

En la práctica clínica, cuando un monitor o dispositivo registra esta cifra, se consideran dos escenarios principales:

1. Es una condición médica real (Patológica u Fisiológica Extrema)

Si la lectura es real, el paciente se encuentra en una situación crítica en la que su cerebro y órganos no están recibiendo suficiente oxígeno:

• Bloqueos cardíacos graves o enfermedad del nodo sinusal: El sistema eléctrico del corazón está fallando. En un bloqueo auriculoventriculo (AV) completo de tercer grado, las aurículas y los ventrículos se desconectan y el corazón activa un ritmo de escape"ventricular que late de forma natural a solo 20-40 lpm para mantener al paciente con vida temporalmente.

• Intoxicaciones por medicamentos: Sobredosis accidentales de fármacos como los betabloqueantes o la digoxina pueden deprimir el ritmo cardíaco hasta estos niveles.

• Límite humano fisiológico (Casos excepcionales): A nivel mundial, se han documentado de forma oficial casos aislados de atletas de élite en disciplinas de resistencia extrema (como el ciclista Miguel Induráin con 28 lpm) o variantes genéticas saludables excepcionales donde el pulso en reposo profundo ha llegado a 26-27 lpm sin síntomas de enfermedad. Sin embargo, esto ocurre en menos del 0.4 % de la población mundial y nunca en una persona sedentaria o enferma.

2. Es un error técnico del dispositivo (Artefacto)

En entornos domésticos con relojes inteligentes o pulsómetros de dedo (oxímetros), un registro de 20-30 lpm a menudo es un error de lectura provocado por:

• Mala perfusión sanguínea (manos frías): Si el paciente tiene los dedos helados o hipotensión, el sensor óptico no detecta bien el pulso y puede "saltarse"latidos reales, dividiendo la cifra real a la mitad (ej. registrar 25 lpm cuando el pulso real es de 50 lpm).

• Movimiento o sudor: El desplazamiento del sensor confunde al software del dispositivo.

• Arritmias específicas (Bigeminismo): Si el corazón late con un patrón de "latido fuerte - latido débil muy pegado", el monitor electrónico puede contar únicamente los latidos fuertes, mostrando la mitad de la frecuencia real en la pantalla.

¿Qué hacer ante un registro de 20-30 lpm?

La regla médica de oro es evaluar al paciente, no a la máquina:

1. Comprobar los síntomas: Si el paciente presenta desmayos (síncopes), mareos intensos, piel pálida/fría, confusión o falta de aire, la lectura es altamente compatible con una crisis real y requiere llamar a emergencias de inmediato. Generalmente, estas situaciones se solucionan en el hospital mediante medicación intravenosa de urgencia o la implantación de un marcapasos definitivo.
2. Confirmación manual: Palpar directamente el pulso carotídeo (en el cuello) durante un minuto completo para verificar si coincide con los latidos del monitor.
Si estás viendo esta medición en este momento, infórmame:
- ¿El paciente presenta mareos, desmayos, palidez o dificultad para respirar?
- ¿La medición proviene de un reloj/oxímetro de casa o de un monitor de hospital?
- ¿Se ha intentado tomar el pulso de forma manual en el cuello o la muñeca?

Análisis

Con esta consulta se confirma que el límite inferior se trata de un valor crítico en un paciente y no de un error.

Prompt

rango proteína c reactiva, cuando es preocupante o imposible

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

La proteína C reactiva (PCR) es una proteína producida por el hígado que aumenta en la sangre cuando hay inflamación o infección en el cuerpo.
El rango normal general de la PCR en adultos es menor a 8 o 10 mg/L (o menos de 0.8 a 1.0 mg/dL, dependiendo del laboratorio).
A continuación se detalla cuándo los niveles empiezan a ser preocupantes y qué valores se consideran biológicamente imposibles.
Niveles de PCR y su significado clínico
Nivel de PCR Interpretación Causas comunes
Menor a 10 mg/L Normal Estado saludable o inflamación insignificante.
10 a 50 mg/L Elevación leve a moderada Infecciones locales (bronquitis, cistitis), traumatismos menores, tabaquismo, obesidad, embarazo o estrés severo.
50 a 100 mg/L Elevación moderada a alta Infecciones bacterianas sistémicas, brotes de enfermedades autoinmunes (artritis reumatoide, lupus) o infarto de miocardio reciente.
Mayor a 100 mg/L Elevación muy alta Infecciones bacterianas graves (sepsis), quemaduras extensas, traumatismos mayores o vasculitis sistémicas.
¿Cuándo es preocupante?
Un nivel de PCR se vuelve preocupante bajo los siguientes criterios: • Mayor a 50 mg/L: Sugiere un proceso inflamatorio activo que requiere evaluación médica para identificar la causa (usualmente una infección bacteriana o un brote autoinmune).
• Mayor a 100 mg/L: Es un criterio de alarma médica inmediata. Suele indicar infecciones bacterianas agudas y potencialmente mortales como la sepsis o una neumonía grave.
• Elevación crónica menor (PCR ultrasensible): Si se mide con la prueba de

PCR ultrasensible (PCR-us) y se mantiene entre 1 y 3 mg/L o más, no indica una emergencia médica, pero sí un riesgo elevado de padecer enfermedades cardiovasculares (infartos o accidentes cerebrovasculares) a largo plazo.

¿Cuándo es imposible o un error de laboratorio?

En la biología humana existen límites físicos para la producción de esta proteína:

- Valores superiores a 500 - 600 mg/L: Son extremadamente raros, aunque pueden verse de forma excepcional en casos críticos de sepsis grave o shock séptico masivo.
- Valores superiores a 1000 mg/L: Se consideran prácticamente imposibles en un contexto clínico real. Si un reporte de laboratorio indica una cifra cercana o superior a los 1000 mg/L, lo más probable es que se trate de un error de transcripción, un fallo de calibración del analizador o una interferencia analítica (como la presencia de anticuerpos heterófilos o factor reumatoide en la muestra que alteran la medición).

En este escenario, la conducta estándar es repetir la toma de muestra de inmediato. Si tienes un resultado de PCR en la mano y deseas interpretarlo con mayor precisión, puedes compartirme el valor exacto, las unidades del laboratorio (mg/L o mg/dL) y si presentas síntomas como fiebre, dolor o fatiga.

Análisis

Al leer la consulta, volvió a aparecer el concepto de PCR ultrasensible para enfermedades cardiovasculares, el cual había salido también en el último prompt de Google AI Studio. Tras regresar al enunciado de la práctica, se confirmó que el atributo correspondía a la variante ultrasensible, por lo que se descartan los límites dados en esta consulta.

Prompt

rango proteína c reactiva ultrasensible para detección de riesgo cardiovascular, cuando es preocupante o imposible

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

La proteína C reactiva ultrasensible (PCR-us o hs-CRP) mide niveles mínimos de inflamación sistémica en la sangre para evaluar el riesgo de enfermedades cardiovasculares (como infartos o accidentes cerebrovasculares).

Rangos de riesgo cardiovascular (en mg/L)

- Menos de 1 mg/L: Riesgo cardiovascular bajo (perfil inflamatorio controlado).
- 1 a 3 mg/L: Riesgo cardiovascular moderado o intermedio (zona gris o de atención en hábitos de vida).
- Más de 3 mg/L: Riesgo cardiovascular alto (alerta de inflamación crónica o actividad en los vasos sanguíneos).

¿Cuándo es preocupante?

- Un valor superior a 3 mg/L indica un mayor riesgo de eventos coronarios y requiere revisar factores como tabaquismo, sedentarismo, resistencia a la insulina o el perfil lipídico.
- Si el resultado es mayor a 10 mg/L, no suele reflejar riesgo cardiovascular crónico, sino un proceso agudo en curso (como una infección bacteriana o viral,

un traumatismo, una lesión reciente o un brote inflamatorio/reumatológico).
• Qué hacer: En estos casos de valores >10 mg/L, los médicos generalmente recomiendan repetir la prueba tras 2 o 3 semanas (cuando la persona se haya recuperado) para saber si el valor alto era transitorio o si persiste una inflamación de fondo.
¿Cuándo puede dar un resultado alterado o ïmposible"de interpretar de forma directa?
Existen situaciones clínicas y fisiológicas que elevan la PCR-us sin que exista una enfermedad cardiaca primaria, lo cual debe ser tenido en cuenta por el médico:
• Infecciones agudas o recientes (resfriados, gripe, infecciones urinarias, etc.).
• Embarazo (los niveles aumentan de forma natural).
• Uso de estrógenos o anticonceptivos orales.
• Enfermedades inflamatorias crónicas o autoinmunes (artritis reumatoide, lupus), donde la inflamación articular eleva el marcador por motivos ajenos al riesgo vascular directo.
• Traumatismos recientes, cirugías o ejercicio físico muy intenso realizado justo antes de la extracción.
Si lo deseas, puedes comentarme:
• Cuál fue el valor exacto que obtuviste (y si estaba en mg/L o mg/dL).
• Si tenías algún síntoma, infección reciente o tomas medicación.
Así podremos revisar qué contexto médico podría estar influyendo en ese resultado.

Análisis

La IA sugiere descartar los valores que superen los 10 mg/L porque no suele reflejar riesgo cardiovascular crónico y los médicos generalmente suelen recomendar repetir la prueba a las 2-3 semanas. Este mismo límite aparecía también en la última consulta de Google AI Studio. Al consultar las fuentes, se confirma que este umbral realmente se aplica en medicina, por lo que se acepta la sugerencia de la IA.

Prompt

rango nivel sérico de ácido úrico, cuando es preocupante o imposible

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

El ácido úrico es un producto de desecho que se forma cuando el cuerpo descompone unas sustancias llamadas purinas. Los niveles en sangre se miden habitualmente mediante un análisis de sangre básico.
A continuación, te detallo los rangos de referencia generales, cuándo se consideran preocupantes y qué significan cifras extremas o «imposibles» de interpretar clínicamente.
—
1. Rangos normales de nivel sérico de ácido úrico
Los valores de referencia estándar en laboratorios suelen ser:
* **Hombres:** Entre **3.4 y 7.0 mg/dL** (miligramos por decilitro).
* **Mujeres:** Entre **2.4 y 6.0 mg/dL** (los valores suelen ser ligeramente más

bajos antes de la menopausia).
* *Nota:* Los rangos exactos pueden variar ligeramente según el laboratorio o el equipo de medición utilizado.

—

2. ¿Cuándo es preocupante? (Hiperuricemia)
Se habla de **hiperuricemia** cuando los niveles superan los límites superiores mencionados (más de 6.0 mg/dL en mujeres o más de 7.0 mg/dL en hombres). Sin embargo, un nivel elevado no siempre causa síntomas inmediatos. Se vuelve preocupante o requiere atención médica en los siguientes escenarios:
* **Riesgo de Gota:** Cuando el ácido úrico se eleva de forma sostenida, puede cristalizar en las articulaciones, provocando ataques de gota (artritis inflamatoria muy dolorosa, comúnmente en el dedo gordo del pie).
* **Formación de Piedras en el Riñón (Nefrolitiasis):** Los niveles altos aumentan la probabilidad de desarrollar cálculos renales, lo que causa cólicos renales intensos y dolor en la espalda baja o el flanco.
* **Daño Renal o Metabólico:** Niveles muy altos de manera crónica pueden asociarse con problemas renales, hipertensión, resistencia a la insulina o síndrome metabólico.
* *Asintomático:* Si un análisis muestra valores ligeramente altos pero nunca has tenido síntomas ni antecedentes de gota, por lo general no se medica de inmediato, aunque sí se aconsejan cambios en el estilo de vida (dieta, hidratación, control de peso).

—

3. Valores «imposibles» o clínicamente incoherentes
Desde una perspectiva médica y fisiológica, existen situaciones donde un valor de ácido úrico carece de sentido o es técnicamente imposible en sangre:
* **Valores negativos:** El ácido úrico no puede ser un número negativo ($<$ 0 mg/dL).
* **Valores absurdamente altos sin clínica extrema (Errores de laboratorio o transcripción):** Cifras extremadamente elevadas (por ejemplo, 30, 50 o 100 mg/dL) en una persona que camina por la calle sin estar hospitalizada suelen deberse a un error de transcripción, una interferencia en la muestra de sangre o una mala toma de la muestra.
* **El límite biológico en humanos:** En pacientes con insuficiencia renal severa o en procesos agudos de destrucción celular masiva (como durante la quimioterapia por ciertos cánceres, conocido como *síndrome de lisis tumoral*), los valores pueden subir drásticamente, pero cifras superiores a 15-20 mg/dL de forma aislada sin una enfermedad crítica subyacente grave son extremadamente raras y suelen requerir revisión urgente.
¿Te gustaría saber qué alimentos o hábitos ayudan a mantener el ácido úrico en niveles saludables?

Análisis

Los datos que aporta son correctos dadas las fuentes de las que lo ha sacado. Se confirma que los valores en la base de datos no son tan dispares como para que no se puedan dar en la realidad.

Prompt

rango tasa de filtrado glomerular / depuración de creatinina, cuando es preocupante o imposible

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

La evaluación de la función renal se realiza principalmente a través de la Tasa de Filtrado Glomerular (TFG o GFR) o mediante la Prueba de Aclaramiento/Depuración de Creatinina (que estima dicho filtrado a través de muestras de sangre y orina de 24 horas). Ambos valores indican cuántos mililitros de sangre purifican los riñones por minuto.
Los rangos de referencia, las cifras de alarma y los límites biológicos se clasifican de la siguiente manera:
1. Rangos normales y de referencia
• TFG / Aclaramiento Normal: 90 mL/min/1.73 m^2 o superior se considera el estado óptimo y saludable.
• Disminución leve: Entre 60 y 89 mL/min. En muchos casos (como en adultos mayores) puede ser un declive fisiológico normal por la edad o una señal temprana si existen factores como diabetes, hipertensión o proteínas en la orina (albuminuria).
2. Cuándo es preocupante (Criterios Médicos)
Un descenso en estos valores empieza a ser clínicamente relevante bajo dos escenarios:
• Preocupante / Sospecha de Enfermedad Renal Crónica (menor a 60 mL/min): Un valor de filtrado de entre 30 y 59 mL/min que persiste durante 3 o más meses es el criterio clínico universal para diagnosticar Enfermedad Renal Crónica (ERC). Requiere seguimiento por medicina interna o nefrología para frenar la progresión del daño.
• Crítico / Emergencia Médica (menor a 15 mL/min): Cualquier resultado por debajo de 15 mL/min representa una insuficiencia renal terminal (Fallo renal Estadio 5). A estos niveles, los riñones han perdido casi toda su capacidad funcional, acumulando toxinas peligrosas en el cuerpo. Requiere atención médica inmediata y suele hacer necesaria la terapia de reemplazo renal (diálisis o trasplante de riñón).
3. Cuándo un valor es ïmposible.º biológicamente incompatible con la vida
• Filtrado Glomerular de 0 mL/min: Fisiológicamente, un filtrado de cero significa el cese total de la función renal. No se puede sobrevivir de forma prolongada con un filtrado cercano a cero sin soporte artificial (diálisis), ya que la acumulación extrema de potasio, fluidos y desechos metabólicos (uremia) provocaría fallos cardíacos y sistémicos fatales en pocos días.
• Valores artificialmente sobreestimados o errores de laboratorio: En el otro extremo, valores exageradamente altos (por ejemplo, superiores a 150-180 mL/min en adultos que no están embarazadas o en fases iniciales muy específicas de nefropatía diabética) suelen ser fisiológicamente imposibles. Generalmente se deben a errores en la recolección exacta de la orina de 24 horas (recoger menos volumen o calcular mal el tiempo) o a una masa muscular extremadamente baja que altera las fórmulas basadas en la creatinina sérica.
Nota: Un resultado alterado de forma aislada no define una enfermedad crónica. La deshidratación severa, el uso de ciertos medicamentos (como antiinflamatorios) o

el ejercicio extenuante pueden bajar el filtrado o subir la creatinina temporalmente. Siempre se deben repetir las pruebas para confirmar tendencias.
Para ayudarte a interpretar mejor la situación, ¿podrías indicarme:
• ¿Cuál fue el resultado exacto de la TFG o aclaramiento en los análisis?
• ¿Aparece también el valor de la creatinina en sangre y la presencia de proteínas/albúmina en la orina?
• ¿El paciente padece de condiciones crónicas como diabetes o hipertensión?

Análisis

Los datos que aporta son correctos dadas las fuentes de las que lo ha sacado. Se confirma que los valores en la base de datos no son tan dispares como para que no se puedan dar en la realidad.

Prompt

rango del porcentaje de hemoglobina glicosilada HbA1c en sangre, cuando es preocupante o imposible

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

El porcentaje de hemoglobina glicosilada (HbA1c) mide el nivel promedio de azúcar en la sangre durante los últimos 2 a 3 meses.
El rango normal de HbA1c en adultos sanos es inferior al 5.7 %, mientras que un valor a partir del 6.5 % confirma el diagnóstico de diabetes.
Rangos estándar de HbA1c
• Normal: Menos de 5.7 %
• Prediabetes: 5.7 % a 6.4 %
• Diabetes: 6.5 % o más
¿Cuándo es preocupante?
Un nivel de HbA1c es preocupante tanto por ser demasiado alto como por ser críticamente bajo:
• Preocupante por niveles altos (8.0 % o más): Si una persona con diabetes supera el 8.0 %, indica un control deficiente de la glucemia. Valores por encima del 10 % al 12 % son críticamente altos y elevan de forma drástica el riesgo de complicaciones agudas (como cetoacidosis diabética o síndrome hiperosmolar) y daños a largo plazo en ojos, riñones, nervios y corazón.
• Preocupante por niveles bajos (<4.0 %): Un valor extremadamente bajo suele ser peligroso en pacientes medicados para la diabetes, ya que indica episodios frecuentes y graves de hipoglucemia (azúcar en sangre peligrosamente bajo), lo que puede provocar desmayos, convulsiones o coma.
¿Cuándo es biológicamente ïmposible¿
En la práctica médica, existen límites físicos y biológicos para este porcentaje: • Límite superior (Imposible >20 % - 25 %): Es prácticamente imposible ver valores superiores al 20 % o 25 %. Para alcanzar un 20 %, el azúcar en sangre promedio tendría que superar de forma sostenida los 500-600 mg/dL durante meses, lo cual es incompatible con la vida a mediano plazo debido a la toxicidad y la deshidratación extrema.

• Límite inferior (Imposible <2.5 % - 3.0 %): Los glóbulos rojos siempre están expuestos a algo de glucosa en el cuerpo humano vivo. Un porcentaje inferior al 3.0 % es biológicamente incompatible con la supervivencia, salvo en casos de errores de laboratorio o variantes genéticas extremas de la hemoglobina.

Análisis

Para el límite superior no habría problema, pero el límite inferior sería imposible que se presentase clínicamente según los valores de la consulta. Sin embargo, no hay fuentes proporcionadas para contrastar la información, por lo que aún no se acepta.

Prompt

proporcióname fuentes para verificar el límite inferior

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

El fundamento biológico de por qué la hemoglobina glicosilada (HbA1c) tiene un límite inferior biológico e imposible de cruzar (en torno al 2.5 % - 3.0 %) en un ser humano vivo se sustenta en consensos clínicos, guías metabólicas y estudios hematológicos.
Las principales bases científicas y los mecanismos médicos que explican este límite son los siguientes:
1. El proceso de glicación no enzimática (Efecto químico inevitable)
La HbA1c no se produce por una orden del organismo, sino por una reacción química pasiva y lenta llamada reacción de Maillard. La glucosa presente en el plasma se adhiere de forma permanente a la hemoglobina de los glóbulos rojos durante sus aproximadamente 120 días de vida útil. Como el cerebro y otros órganos vitales necesitan un mínimo de glucosa constante en sangre para no sufrir muerte cerebral (incluso en estados de ayuno extremo o desnutrición), siempre hay azúcar en el torrente sanguíneo. Al haber glucosa circulante y glóbulos rojos conviviendo en el mismo espacio, es químicamente imposible que la tasa de glicación caiga a cero.
2. Guías Clínicas de la ADA (American Diabetes Association)
En los estándares de atención médica de la American Diabetes Association (ADA), se describe que el percentil más bajo observado en poblaciones humanas sanas sin patologías hematológicas se sitúa alrededor del 4.0 % al 4.5 %. Los laboratorios clínicos calibrados bajo el Programa Nacional de Estandarización de Glicohemoglobinas (NGSP) configuran sus rangos de referencia con este suelo protector. Cualquier valor que comience a bajar del 4.0 % enciende alertas automáticas de hipoglucemia severa o de anomalías en la muestra.
3. Falsos Negativos e Ïmposibles"por Interferencias Hematológicas
Cuando un laboratorio registra un valor extraordinariamente bajo (cercano o inferior al 3.0 %), la literatura médica indica que casi nunca se debe a un nivel real de azúcar, sino a alteraciones en la vida del glóbulo rojo o a variantes genéticas. Las principales referencias sobre estas interferencias analizan los siguientes fenómenos:
• Acortamiento de la vida útil del eritrocito: Publicaciones indexadas en PubMed Central (PMC) detallan que condiciones como la anemia hemolítica aguda o las

transfusiones de sangre recientes provocan que los glóbulos rojos se destruyan mucho antes de los 120 días o se reemplacen por células nuevas sin glicar. Al no darles tiempo físico de entrar en contacto con el azúcar, el valor de HbA1c cae artificialmente, simulando un límite biológico alterado.

• Deficiencia de G6PD y Hemoglobinopatías: Un estudio publicado en la revista científica Diabetes Care evidenció cómo ciertas condiciones genéticas silenciosas (como la deficiencia de la enzima glucosa-6-fosfato deshidrogenasa o variantes de la hemoglobina como la HbS o HbC) arrojan valores de HbA1c falsamente bajos e inexactos debido a métodos de medición interferidos o a una menor supervivencia celular, a pesar de que el paciente presente glucemias normales o incluso elevadas.

• Errores metodológicos en Laboratorio: Revistas especializadas como JAMA señalan que la presencia de hemoglobinas anormales interfiere físicamente con tecnologías estándar como la Cromatografía Líquida de Alta Resolución (HPLC). El equipo confunde las variantes proteicas y reporta lecturas erróneas o ïmposibles"que requieren métodos alternativos de control (como la medición de fructosamina).

En resumen, un valor inferior al 3 % es catalogado en medicina como un artefacto técnico (error de medición) o una emergencia biológica incompatible con la homeostasis humana general.

Análisis

Ahora que se han podido consultar las fuentes, se acepta la decisión de considerar un valor inferior al 3 % como error para la hemoglobina glicosilada.

Prompt

```
# E. Incoherencia de lípidos: Colesterol total menor que la suma de sus fracciones (HDL + LDL)
inval_lipidos = (
    df["colesterol_total"] < (df["colesterol_hdl"] + df["colesterol_ldl"])
).sum()
incoherencias_clinicas["Colesterol Total < (HDL + LDL)"] = inval_lipidos
```

Esta comprobacion esta mal, no deberíamos comprobar si se cumple la fórmula?

Respuesta IA — Gemini Flash 3.8 en Google AI Studio

Tienes razón en tu intuición de que la fórmula completa debe considerarse, pero
la comprobación que tienes escrita no está conceptualmente mal, sino que es una
comprobación de mínimos absolutos.

Sin embargo, no es suficiente y tiene matices importantes de laboratorio que
debes conocer para no descartar datos válidos.

Aquí te explico por qué tu comprobación actual sirve como filtro de corte, por
qué validar la fórmula exacta puede ser un error, y cuál es la forma correcta de

```
programarlo.

1. ¿Por qué la comprobación que tienes es válida (como cota mínima)?

El colesterol total se compone de tres fracciones principales:
\text{Colesterol Total} = \text{HDL} + \text{LDL} + \text{VLDL}

Dado que el colesterol VLDL (asociado a los triglicéridos) nunca puede
ser
negativo (\text{VLDL} > 0), matemáticamente se cumple siempre que:
\text{Colesterol Total} > \text{HDL} + \text{LDL}

Por tanto, si en tus datos: colesterol_total < (colesterol_hdl +
colesterol_ldl)

Significa que el todo es menor que la suma de solo dos de sus partes.
Esto es
una imposibilidad matemática y biológica directa. Si esto ocurre, hay un
error
seguro de digitación. Por eso tu código actual es un excelente detector
de
fallos graves.

2. ¿Por qué NO deberías exigir que la fórmula se cumpla de forma exacta?

Podrías tener la tentación de comprobar la Fórmula de Friedewald:
\text{Colesterol Total} \approx \text{HDL} + \text{LDL} +
\frac{\text{Triglicéridos}}{5}

Exigir que esta igualdad se cumpla al 100% en tu dataset te dará falsos
positivos de error (creerás que los datos están mal cuando en realidad
son
correctos) por dos motivos:

1.  Método de medición directa de LDL: Hoy en día, muchos laboratorios ya
no
    estiman el LDL con la fórmula matemática, sino que lo miden
    directamente en
    sangre mediante reactivos químicos (Direct LDL assay). Las mediciones
    directas tienen márgenes de error analítico (\pm 3-5\%) que hacen que
    la
    suma nunca cuadre al milímetro.
2.  Fallo de la fórmula con triglicéridos altos: La aproximación
    \text{VLDL} \approx \text{Triglicéridos} / 5 deja de ser válida si
    los
    triglicéridos son > 400\text{ mg/dL}, y falla bastante si son
    > 200\text{ mg/dL} o < 50\text{ mg/dL}.

3. La forma correcta y robusta de comprobarlo en Python
```

```
Lo ideal es permitir una pequeña tolerancia fisiológica/analítica
(alrededor de
un 5% a 10%) para no penalizar el redondeo del laboratorio, pero marcar
aquellos
casos donde la discrepancia sea un disparate.

Aquí tienes cómo estructurar esta validación de forma profesional:

# 1. Error Grave e Imposible: El Total es significativamente menor que
HDL + LDL
# Dejamos un margen de tolerancia del 5% por variabilidad analítica del
laboratorio
tolerancia = 0.95
error_suma_basica = (
    df["colesterol_total"] < (df["colesterol_hdl"] +
    df["colesterol_ldl"]) * tolerancia
).sum()

# 2. Comprobación de la Fórmula de Friedewald (solo válida si
Triglicéridos < 400)
# Estimamos el colesterol esperado según la fisiología clásica
colesterol_esperado = (
    df["colesterol_hdl"] + df["colesterol_ldl"] + (df["trigliceridos"] /
    5)
)

# Calculamos la discrepancia porcentual respecto al total registrado
diferencia_relativa = (
    abs(df["colesterol_total"] - colesterol_esperado) /
    df["colesterol_total"]
)

# Filtramos: Solo evaluamos donde Triglicéridos < 400 y la diferencia sea
> 15% (muy grande)
error_friedewald = (
    (df["trigliceridos"] < 400) & (diferencia_relativa > 0.15)
).sum()

# Guardamos los resultados
incoherencias_clinicas["Error Crítico: Total < (HDL + LDL)"] =
error_suma_basica
incoherencias_clinicas["Discrepancia severa en perfil lipídico (>15%)"] =
(
    error_friedewald
)

Resumen para tu pipeline:

  - Conserva tu comprobación original (con o sin el pequeño margen del
  5%): si
```

```
  Total < HDL + LDL, el dato está irremediablemente corrupto.
- Si quieres ir más allá y verificar la fórmula de Friedewald completa,
hazlo
  siempre excluyendo a pacientes con triglicéridos \ge 400\text{ mg/dL}
  y
  permitiendo un margen de error del 10% al 15%.
```

Análisis

Se acepta la respuesta de la IA de que la comprobación que hacía mi compañero para detectar valores no válidos para el colesterol total es correcto, cuando se pretendía cambiar esta línea por la fórmula que relacionan los triglicéridos con los dos tipos de colesterol para obtener el colesterol total. También sugiere que a la hora de comprobar los valores esperados con la fórmula, se use un margen de error. Se acepta la sugerencia.

Prompt

Me ha salido un 24.8 % de casos en los que no se cumple la fórmula de total <(DDL + LDL) y un 57.76 % de los valores, con un margen del 15 %, me salen que tienen discrepancia. Además, apenas hay correlación del colesterol total con los triglicéridos y con los dos tipos de colesterol y solo hay un 0.10 de correlación con el target. Crees que esta columna podría eliminarse o no?

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Con esos datos en la mano, la respuesta corta es: Sí, eliminar
colesterol_total
es la decisión más sensata y técnicamente defendible.

De hecho, los números que acabas de obtener revelan una realidad muy
común en
ciencia de datos aplicada a la salud: tu columna de colesterol_total está
corrupta o el dataset es sintético y fue mal generado.

Aquí te explico los motivos analíticos, clínicos y de modelado que
justifican su
eliminación:

1. Diagnóstico del problema: Datos rotos o sintéticos

  - Un 24.8% de imposibilidad biológica: Que un cuarto de tu base de
  datos viole
    una ley física básica (Total < HDL + LDL) no es un "margen de error
    de
    laboratorio" (que suele rondar el 2-3%). Es un error sistemático
    masivo.
  - Falta de correlación interna: En biología real, la correlación entre
  el
```

```
    Colesterol Total y el LDL suele ser altísima (r \approx 0.80 - 0.90).
    Si en
    tu dataset no correlacionan entre sí, significa que las variables se
    generaron de forma independiente (ruido aleatorio) o que las columnas
    se
    desalinearon al importar los datos.
  - Un modelo no puede extraer conocimiento médico de una variable cuyos
  valores
    no respetan la estructura subyacente del dominio.

2. Justificación clínica: El Colesterol Total ya es obsoleto por sí solo

Incluso en la medicina real moderna (guías europeas ESC/EAS o americanas
AHA/ACC), el colesterol total apenas se usa de forma aislada para estimar
riesgo
porque es una métrica "ciega":

  - Un paciente puede tener un colesterol total alto (240\text{ mg/dL})
  porque
    tiene un HDL ("bueno") altísimo (90\text{ mg/dL}), lo cual es
    protector.
  - Otro puede tener el mismo total (240\text{ mg/dL}) a expensas de un
  LDL
    ("malo") de 180\text{ mg/dL}, lo cual es peligroso.

Los verdaderos inductores de placa aterogénica y riesgo cardiovascular
son el
LDL, los Triglicéridos y la falta de HDL. Esas tres variables ya las
tienes en
el dataset de forma individual. Mantener un "Total" inconsistente solo
aporta
ruido y confusión (colinealidad rota).

3. Justificación de Machine Learning

  - Correlación de 0.10 con el target: Tiene una capacidad predictiva
    prácticamente nula (linealmente hablando).
  - Sobreasociación espuria: Si dejas la columna, algoritmos basados en
  árboles
    (XGBoost, Random Forest) podrían encontrar "reglas tontas" basadas en
    el
    ruido de esa columna corrupta (overfitting), restando peso a
    variables que
    sí son clínicamente causales como la glucosa, la tensión arterial o
    el LDL.
```

Análisis

Mi intención al consultarte sobre las inconsistencias y la baja correlación del colesterol total era validar si esos datos anómalos justificaban descartar la variable

sin perder información clave para el modelo y así tener una primera opinión. Aunque tomo en cuenta sus argumentos técnicos y considero muy viable su eliminación, dejo la decisión final en pausa hasta comentarlo con mi compañero.

Prompt

en machine learning qué mide el skewness

Respuesta IA `Gemini Flash Lite 3.5 en Gemini`

En machine learning y estadística, el skewness (o asimetría) mide el grado de distorsión o falta de simetría en la distribución de los datos con respecto a su media.
En términos sencillos, te indica si los datos están ”sesgados”hacia un lado o si se distribuyen de forma uniforme y simétrica (como la clásica campana de Gauss).
Los 3 tipos de Skewness
Tipo de Asimetría Valor del Skewness Significado Relación Media/Mediana
Simétrica Igual a 0 (o muy cercano) La distribución es perfectamente equilibrada en ambos lados. Media = Mediana = Moda
Positiva (A la derecha) Mayor que 0 (> 0) La cola larga de la distribución se extiende hacia los valores más altos (derecha). La mayoría de los datos se concentran en valores bajos. Media >Mediana >Moda
Negativa (A la izquierda) Menor que 0 (< 0) La cola larga se extiende hacia los valores más bajos (izquierda). La mayoría de los datos se concentran en valores altos. Media <Mediana <Moda
¿Por qué es crucial en Machine Learning?
Muchos algoritmos de machine learning asumen que las variables predictoras (features) tienen una distribución normal (simétrica). Medir el skewness es el primer paso para corregirlo por las siguientes razones:
1. Afecta a los modelos lineales: Modelos como la Regresión Lineal, Regresión Logística o el Análisis de Discriminante Lineal (LDA) pierden mucha precisión si los datos están muy sesgados.
2. Impacto de los Outliers: Un skewness alto suele indicar la presencia de valores atípicos (outliers) en las colas largas, lo que puede distorsionar el entrenamiento.
3. Inestabilidad en algoritmos basados en distancia: Algoritmos como KNN o K-Means pueden verse afectados si una variable tiene una escala comprimida por el sesgo.
¿Cómo se soluciona en el pipeline de datos?
Cuando detectas un skewness alto (generalmente mayor a |1| o |0,5| según el criterio), se aplican transformaciones matemáticas para normalizar los datos antes de entrenar el modelo:
• Para skewness positivo: Transformación logarítmica (log(x)), raíz cuadrada (sqrt(x)) o la transformación Box-Cox.
• Para skewness negativo: Transformación exponencial o elevar al cuadrado (x^2).
Si estás analizando un conjunto de datos en este momento, cuéntame: ¿qué tipo de datos estás evaluando (ej. ingresos, edades, precios) o qué algoritmo de machine learning planeas utilizar? Así podré recomendarte la mejor estrategia de

transformación.

Análisis

Se acepta la explicación de qué mide el skewness, aunque no queda claro por qué mi compañero considera 0.5 como el límite para clasificar el límite de asimetría.

Prompt

```
if n_outliers > 0 or abs(skew) > 0.8:
    outlier_skew_list.append(
        {
            "Variable": col,
            "Nº Outliers": n_outliers,
            "% Outliers": pct_outliers,
            "Skewness": skew,
            "Tipo Asimetría": (
                "Positiva (Cola dcha)"
                if skew > 0.5
                else (
                    "Negativa (Cola izq)"
                    if skew < -0.5
                    else "Simétrica (~Normal)"
                )
            ),
        }
    )
```

Cámbiame este código y pon estos límites mejor:
In the interpretation it is stated - following Bulmer (1979) - that: Ïf skewness is less than -1 or greater than $+1$, the distribution is highly skewed. If skewness is between -1 and $-½$ or between $+½$ and $+1$, the distribution is moderately skewed. If skewness is between $-½$ and $+½$, the distribution is approximately symmetric.”

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Aquí tienes el código adaptado con la regla clásica de Bulmer (1979).

He ajustado tanto la condición de entrada abs(skew) > 0.5 (para capturar cualquier variable que deje de ser aproximadamente simétrica) como la categorización en los 5 casos posibles (altamente sesgada, moderadamente sesgada
o simétrica, tanto a la izquierda como a la derecha):

```
# Según Bulmer (1979):
# - |skew| > 1: Altamente sesgada
# - 0.5 < |skew| <= 1: Moderadamente sesgada
# - |skew| <= 0.5: Aproximadamente simétrica

if n_outliers > 0 or abs(skew) > 0.5:
    if skew > 1:
        tipo_asimetria = "Altamente sesgada (Positiva / Cola dcha)"
    elif 0.5 < skew <= 1:
        tipo_asimetria = "Moderadamente sesgada (Positiva / Cola dcha)"
    elif -1 <= skew < -0.5:
```

```
        tipo_asimetria = "Moderadamente sesgada (Negativa / Cola izq)"
    elif skew < -1:
        tipo_asimetria = "Altamente sesgada (Negativa / Cola izq)"
    else:
        tipo_asimetria = "Aproximadamente simétrica"

    outlier_skew_list.append(
        {
            "Variable": col,
            "Nº Outliers": n_outliers,
            "% Outliers": pct_outliers,
            "Skewness": round(skew, 3),
            "Tipo Asimetría": tipo_asimetria,
        }
    )

Cambios aplicados:

1.  Condición de filtrado: Se cambió abs(skew) > 0.8 por abs(skew) > 0.5.
Si
    dejas 0.8, estarías ignorando variables con sesgo moderado (entre 0.5
    y 0.8)
    que no tengan outliers detectados.
2.  Estructura clara de decisión: Se sustituyó el ternario anidado por un
bloque
    if/elif/else mucho más legible y fiel a la literatura estadística.
3.  Redondeo: Se añadió un round(skew, 3) para presentar el valor en el
reporte
    o tabla de forma más limpia.
```

Análisis

Se cambia la interpretación del valor del skewness que había antes por una interpretación oficial. Dada esta nueva clasificación incluida en el prompt, la IA implementó la nueva clasificación correctamente. Se acepta.

Prompt

```
frecuencia_ejercicio_semanal (Numérico): Días a la semana con actividad
física moderada o intensa.

nivel_actividad (Nominal): Grado habitual de actividad física diaria.
Valores posibles: 'Sedentario', 'Ligeramente activo', 'Moderadamente
activo', 'Muy activo'.

Tengo estas dos variables en mi dataset. ¿Crees que ambas variables se
refieren a lo mismo o se podrían estar refiriendo a actividades
diferentes, como la actividad física en el día a día, por ejemplo,
trabajando, frente a ejercicio y deporte?
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

¡Tienes toda la razón en tu intuición! Aunque ambas variables están relacionadas con el movimiento y el esfuerzo físico, **se refieren a dimensiones diferentes** y complementarias de la actividad física.

No son redundantes; de hecho, en estudios epidemiológicos, de salud o de ciencia de datos, es muy común encontrar ambas por separado porque capturan realidades distintas del estilo de vida de una persona.

Diferencia clave entre ambas variables

* **`frecuencia_ejercicio_semanal` (Ejercicio estructurado):**
Mide la **intencionalidad y el tiempo de ocio dedicado al deporte o entrenamiento**. Cuenta los días en que la persona realiza una actividad física planificada (como ir al gimnasio, salir a correr, nadar, jugar al fútbol o asistir a clases dirigidas).
* **`nivel_actividad` (Actividad global / Estilo de vida):**
Mide el **gasto energético total derivado de la rutina diaria** (a menudo asociado al concepto de *PAL - Physical Activity Level* o al NEAT). Esto incluye no solo el ejercicio, sino también el tipo de trabajo (si es de oficina sentado o de pie cargando peso), las tareas domésticas, los desplazamientos a pie o en bicicleta y el movimiento espontáneo a lo largo del día.

Ejemplos prácticos para entenderlo

Para ver por qué son distintas, imagina estos dos perfiles típicos en un dataset:

* **Perfil 1 (Oficinista deportista):** Trabaja sentado frente a un ordenador 8 horas al día, pero es muy disciplinado y va al gimnasio o sale a correr **4 días a la semana**.
* *`frecuencia_ejercicio_semanal`* = `4`
* *`nivel_actividad`* = Podría ser *"Ligeramente activo"* o *"Moderadamente activo"* (el resto del día es sedentario).

* **Perfil 2 (Trabajador de la construcción o repartidor):** Trabaja todo el día de pie, caminando y cargando peso de forma activa, pero nunca va al gimnasio ni hace deporte en su tiempo libre (**0 días de ejercicio planificado**).
* *`frecuencia_ejercicio_semanal`* = `0`
* *`nivel_actividad`* = Podría ser *"Muy activo"* debido al esfuerzo físico constante de su jornada laboral.

```
---

### ¿Por qué es útil tener ambas en un dataset?

Tener las dos variables permite a los modelos (o a los analistas)
distinguir entre el **ejercicio recreativo/saludable** y la **carga
física ocupacional o de estilo de vida**, los cuales tienen impactos
diferentes en la salud, el metabolismo y el gasto calórico total.
```

Análisis

En relación con saber si las variables frecuencia_ejercicio_semanal y nivel_actividad se refieren a lo mismo o a cosas diferentes, la IA distinguió ambas dimensiones (ejercicio estructurado y planificado frente a gasto energético global o actividad ocupacional/diaria) y aportó ejemplos claros para justificarlo. Aunque es una respuesta en principio a tomar en cuenta, se debe discutir este asunto con el equipo para adoptar una solución de interpretación definitiva. Esta consulta únicamente sirve para tener una primera opinión al respecto.

Prompt

y qué ocurre con los sedentarios que hacen 7 días? Según la descripción de las variables, se deberían considerar valores incoherentes o son actividades físicas diferentes?

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
¡Qué excelente observación! Has dado en el clavo con un dilema clásico de
**calidad de datos (*Data Quality*)** y validación lógica en ciencia de
datos y salud.

En la práctica, encontrarte con un perfil que indica `nivel_actividad =
'Sedentario'` pero una `frecuencia_ejercicio_semanal = 7` (entrenar todos
los días) abre dos escenarios muy claros:

---

### 1. Una incoherencia o error de datos (Lo más probable)

En la gran mayoría de cuestionarios de salud estandarizados (como el IPAQ
- *International Physical Activity Questionnaire*), el **nivel de
actividad global** se calcula sumando el gasto energético total, donde el
ejercicio estructurado aporta una cantidad masiva de puntos (METs).

Si el dataset asigna la etiqueta *"Sedentario"* basándose en una
puntuación global o en una escala donde el ejercicio cuenta, esta
combinación es **metodológicamente imposible**. Suele deberse a:
```

```
* **Errores de auto-reporte:** Las personas suelen sobreestimar su
ejercicio ("voy al gimnasio 7 días" aunque sea 10 minutos a mirar el
móvil) o malinterpretan las preguntas del formulario.
* **Contradicciones en la fuente:** Inconsistencias lógicas introducidas
al fusionar bases de datos o por errores humanos en la digitalización de
encuestas.

### 2. Una separación estricta de dominios (Caso muy atípico pero real)

Si el cuestionario o dataset fue diseñado para aislar completamente el
**trabajo/rutina diaria** del **tiempo libre**, teóricamente podría
ocurrir.

* *Imagina a un teletrabajador extremo:* Pasa 12 horas al día sentado
frente al ordenador programando o teletrabajando (lo que clasifica su día
a día puramente laboral/cotidiano como *"Sedentario"*), pero por
disciplina militar sale a correr o hace estiramientos intensos **todos
los días del año** (7 días de ejercicio).
* Fisiológicamente es posible, pero en términos de estilo de vida general,
llamarlo "sedentario" resulta confuso porque alguien que entrena 7 días a
la semana difícilmente encaja en el concepto clínico de sedentarismo.

---

### ¿Cómo deberías manejar esto si estás analizando el dataset?

Si estás limpiando los datos para un análisis o un modelo de Machine
Learning, esta combinación debe tratarse como una **alerta de anomalía**:

1. **Haz una tabulación cruzada (*crosstab*):** Cruza ambas variables en
una tabla para ver cuántos casos caen en esta combinación (por ejemplo,
¿cuántos "sedentarios" hay con 5, 6 o 7 días de ejercicio?). Si son
muchos, hay un problema en la definición de las variables o en cómo se
recolectó la información.
2. **Criterio de limpieza (Imputación o Filtrado):** Dependiendo de tu
objetivo, podrías reclasificar a estos perfiles como "Moderadamente
activos" o "Muy activos" (dado que 7 días de ejercicio anulan el
sedentarismo real), o bien marcarlos como valores atípicos (*outliers*)
para eliminarlos si corrompen la lógica del modelo.
```

Análisis

Se le pregunta a la IA sobre un caso que genera cierta ambigüedad (ser sedentario con 7 días de actividad física moderada o intensa). Aunque la IA se inclina a considerarlo una incoherencia, no descarta que se deba a medir dominios separados. Ante la falta de una conclusión cerrada, se comentará con el equipo para decidir una solución.

Prompt

comprueba en un df['anos_fumador'] en python cuál es la cantidad de decimales más grande. Es un float

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Para comprobar cuál es la cantidad máxima de decimales en una columna de tipo float (df['anos_fumador']) en Pandas, puedes utilizar dos enfoques principales según tus necesidades:

Opción 1: Método rápido (manipulación de strings)
Este método es ideal si tus datos no contienen notación científica. Convierte los valores a texto, los divide por el punto decimal, elimina los ceros finales sobrantes y calcula la longitud máxima.

Python

```
import pandas as pd

# Calcula el número máximo de decimales ignorando valores nulos (NaN)
max_decimales = (
    df['anos_fumador']
    .dropna()
    .astype(str)
    .str.split('.')
    .str[1]
    .str.rstrip('0')
    .str.len()
    .max()
)

print(f"Cantidad máxima de decimales: {max_decimales}")
```

Opción 2: Método robusto (con la librería decimal)
Si tu DataFrame contiene números en notación científica (por ejemplo, 1.2e-5) o necesitas una precisión matemática absoluta al analizar los exponentes, la librería estándar decimal es la opción más segura:

Python

```
from decimal import Decimal
import pandas as pd

max_d = 0
for val in df['anos_fumador'].dropna():
    try:
        d = Decimal(str(val))
        exp = d.as_tuple().exponent
        decimals = -exp if exp < 0 else 0
        if decimals > max_d:
            max_d = decimals
    except Exception:
        pass
```

```
print(f"Cantidad máxima de decimales (con Decimal): {max_d}")
Explicación rápida:
dropna(): Evita errores si hay valores nulos (NaN) en la columna.

rstrip('0') / Exponentes: Evitan que se cuenten como decimales reales los
ceros innecesarios al final (por ejemplo, tratar 10.0 como 0 decimales en
lugar de 1)
```

Análisis

El programa es correcto y se acepta. Se elige la primera opción, que es más simple.

Prompt

```
D. Discordancia de comorbilidad vs fármaco:

¿Toma antihipertensivos sin diagnóstico de hipertensión?

farmaco_sin_diag_hta = ( (df["uso_antihipertensivos"] == 1) &
(df["hipertension_previa"] == 0) ).sum() print( f"7. Uso de
antihipertensivos
sin diagnóstico previo de HTA: {farmaco_sin_diag_hta} casos" ) 7. Uso de
antihipertensivos sin diagnóstico previo de HTA: 478 casos

Qué mira esto? Son valores que están mal en mi dataset?
```

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
No significa necesariamente que estén mal.

Ese bloque de código está buscando pacientes que declaran tomar
medicación para
la tensión (uso_antihipertensivos == 1), pero en su historial figura que
no
tienen diagnóstico de hipertensión (hipertension_previa == 0).

Que tengas 478 casos así se debe habitualmente a una de estas dos razones
(una
médica y otra de calidad de datos):

1. La razón médica (Es un dato real y válido)

Los fármacos antihipertensivos (como los IECA, ARA-II o betabloqueantes)
no solo
se recetan para la presión alta. Tienen muchas otras indicaciones
médicas:

  - Protección renal: A los pacientes con diabetes se les recetan
```

```
    antihipertensivos (como enalapril o losartán) aunque tengan la
    tensión
    perfecta, porque protegen los riñones.
  - Insuficiencia cardíaca o arritmias: Se usan betabloqueantes (como
    bisoprolol) para controlar el ritmo cardíaco, no por ser hipertensos.
  - Profilaxis de migrañas o temblores: Fármacos como el propranolol se
  usan
    para evitar dolores de cabeza crónicos.

Si filtras esos 478 pacientes, probablemente verás que muchos tienen
diabetes_mellitus == 1, problemas renales (clearance_creatinina bajo) o
edad
avanzada. En estos casos, el dato es 100% real.

2. La razón de sesgo de encuesta / recogida de datos (Subdiagnóstico)

Si la base de datos se hizo mediante un cuestionario al paciente:

  - El paciente confunde su estado: Mucha gente piensa: "Como me tomo la
    pastilla y mi tensión ahora da 120/80, ya no soy hipertenso", y
    marcan
    hipertension_previa = 0.
  - Falta de registro formal: El médico le mandó la pastilla de forma
  preventiva
    pero en el sistema nunca se codificó formalmente la etiqueta de
    "Hipertensión esencial".

¿Qué debes hacer con esos 478 casos?

1.  NO LOS BORRES NI LOS CAMBIES A CERO. Si borras 478 filas por esto,
estarías
    destruyendo información muy valiosa sobre pacientes medicados (que
    suelen
    tener más riesgo).
2.  Revisa su presión actual: Si tienen uso_antihipertensivos == 1,
    hipertension_previa == 0, pero ves que su presion_sistolica media es
    de 145
    mmHg o tienen diabetes, confirma la hipótesis de que están enfermos o
    mal
    controlados.
3.  Decisión para el modelo: El algoritmo de Machine Learning (como
XGBoost o
    Random Forest) es capaz de gestionar esto sin problema. Simplemente
    aprenderá que el hecho de tomar el fármaco (uso_antihipertensivos ==
    1) es
    una señal de riesgo por sí misma, independientemente de la casilla de
    diagnóstico.

En resumen: No es un error grave de tipo "persona con 300 años" o
"presión
```

```
diastólica mayor que sistólica". Es una inconsistencia clínica común en
la
práctica médica real y debes conservarla.
```

Análisis

Al ver que se consideraba en el EDA 478 casos como incorrectos, se consultó a la IA para validarlo. La respuesta es coherente y determina que no es un error de datos. Al consultarlo con mi compañero, se decidió no aplicar ninguna modificación a estos registros.

Prompt

```
esto es una buena forma de saber si alguien tiene diabetes y no ha sido
diagnosticada?
# E. Diabetes no diagnosticada (HbA1c >= 6.5% o Glucosa >= 126 mg/dL sin
DM previa)
diabetico_oculto = (
    (df["diabetes_mellitus"] == 0)
    & ((df["hemoglobina_glicosilada"] >= 6.5) | (df["glucosa_ayunas"] >=
    126))
).sum()
print(
    f"7. Criterio de diabetes analítica no diagnosticada previamente:
    {diabetico_oculto} casos"
)
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
Sí, es una excelente aproximación analítica basada en los criterios
diagnósticos oficiales de la Asociación Americana de Diabetes (ADA) y la
Organización Mundial de la Salud (OMS).

¿Por qué es médicamente correcta?

Los umbrales que utilizas son los estándar de referencia para el
diagnóstico de diabetes:

Hemoglobina glicosilada (HbA1c) $\ge 6.5\%$: Refleja el promedio de
glucosa en sangre de los últimos 3 meses. Un valor igual o superior a
este porcentaje indica diabetes.

Glucosa en ayunas $\ge 126\text{ mg/dL}$ (en ausencia de ingesta calórica
previa de al menos 8 horas): Es el segundo criterio analítico principal.

df["diabetes_mellitus"] == 0: Filtra correctamente a los pacientes que no
tienen un diagnóstico previo registrado, cumpliendo con la definición
exacta de "diabetes no diagnosticada" u oculta.
```

Sin embargo, ten en cuenta estas limitaciones clínicas (para que tu análisis sea más robusto):

La regla de la confirmación: En la práctica clínica real, un solo valor alterado de glucosa en ayunas o HbA1c no basta para dar el diagnóstico definitivo. Los protocolos médicos exigen una repetibilidad (repetir la prueba en un segundo análisis o usar otra métrica confirmatoria) a menos que existan síntomas clásicos de hiperglucemia (poliuria, polidipsia, pérdida de peso inexplicable).

Tu código identifica correctamente a los casos "con sospecha analítica fuerte", pero en epidemiología se suele hablar de casos que cumplen criterios de cribado.

El ayuno estricto: Para la glucosa en ayunas, asegúrate de que los datos del dataset provengan realmente de pacientes en ayunas, ya que una comida previa puede elevar puntualmente la glucosa sin que signifique diabetes.

Interferencias en la HbA1c: Ciertas condiciones (como hemoglobinopatías, anemias severas, insuficiencia renal crónica o embarazos) pueden alterar los valores de la HbA1c dando falsos positivos o negativos.

Conclusión

Tu código es matemáticamente preciso y conceptualmente riguroso para un análisis de datos o estudio epidemiológico sobre un dataset. Si estás preparando un informe, puedes presentar esta métrica con confianza, aclarando que se trata de casos que cumplen con los criterios analíticos de laboratorio para diabetes sin un diagnóstico previo registrado en el sistema.

Análisis

Al ver que mi compañero marcaba 597 casos como pacientes con diabetes sin detectar, procedo a verificar si la comprobación es correcta. Aunque dice ser datos oficiales de la OMS, necesito corraborarlo con fuentes, puesto que supondría una modificación de alreredor del 25 % de la columna de diabetes.

Prompt

pero los valores utilizados estan avalados por fuentes?

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Sí, están 100% avalados por las principales organizaciones médicas del mundo.

```
Los valores que utilizaste en tu código ($\text{HbA1c} \ge 6.5\%$ y
$\text{Glucosa en ayunas} \ge 126\text{ mg/dL}$) son los criterios
estándar internacionales de diagnóstico respaldados por las máximas
autoridades en endocrinología y salud pública.

Las fuentes de referencia oficiales:

American Diabetes Association (ADA): En sus guías anuales de práctica
clínica (Standards of Care in Diabetes), la ADA establece explícitamente
estos dos cortes exactos para el diagnóstico de diabetes.

Organización Mundial de la Salud (OMS): Define los mismos umbrales en sus
manuales globales de diagnóstico y clasificación de la diabetes mellitus.

Grupo de Trabajo Internacional de Expertos en HbA1c: Recomienda
formalmente el uso de la hemoglobina glicosilada ($\ge 6.5\%$) como punto
de corte para diagnosticar la enfermedad sin necesidad de realizar
pruebas de glucosa orales adicionales.

Guías de Práctica Clínica del Ministerio de Sanidad (y homólogos
internacionales): Adaptan íntegramente estos mismos valores para los
protocolos de salud pública y atención primaria.

Resumen de los puntos de corte oficiales:

Estado,Glucosa en Ayunas,Hemoglobina Glicosilada (HbA1c)

Normal,<100 mg/dL,<5.7%

Prediabetes,100–125 mg/dL,5.7%–6.4%

Diabetes (Tu código), 126 mg/dL, 6.5%

Puedes incluir las guías de la ADA (Standards of Care) o los documentos
de la OMS en las citas o notas al pie de tu proyecto con total
tranquilidad técnica y científica.
```

Análisis

Aunque me vuelve a insistir que los datos están 100 % avalados y me proporciona más fuentes de referencia oficiales que usan este criterio, sigo sin aceptar la respuesta de la IA, ya que supone un cambio enorme a una columna del dataset.

Prompt

pasame enlace de una fuente para verlo

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
Aquí tienes el enlace directo al capítulo oficial de las Guías de
Práctica Clínica de la American Diabetes Association (ADA):

ADA Standards of Care: 2. Diagnosis and Classification of Diabetes

¿Dónde verlo directamente en el documento?

Dentro del enlace, si vas a la Tabla 2.1 (Criteria for the Diagnosis of
Diabetes), verás exactamente los siguientes umbrales:

A1C $\ge 6.5\%$ ($\ge 48\text{ mmol/mol}$)

Fasting Plasma Glucose (FPG) $\ge 126\text{ mg/dL}$ ($\ge 7.0\text{
mmol/L}$) - definiendo ayuno como ausencia de ingesta calórica durante al
menos 8 horas.

También puedes revisar el resumen para pacientes directamente en el
portal de la American Diabetes Association (Diabetes.org).
```

Análisis

Para terminar de confirmar que el código para detectar diabetes se puede aceptar, pido enlace a fuentes directas. Al revisarlas, puedo confirmar que es correcto.

Prompt

pero esta bien con diabetes mellitus? o es un tipo diferente

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
Sí, está perfectamente bien. La Diabetes Mellitus Tipo 2 representa la
inmensa mayoría de estos casos en la práctica clínica y epidemiológica.

Los umbrales que usaste ($\text{HbA1c} \ge 6.5\%$ y $\text{Glucosa en
ayunas} \ge 126\text{ mg/dL}$) son universales para diagnosticar el
estado metabólico de Diabetes Mellitus, pero no diferencian por sí solos
qué tipo de diabetes tiene el paciente.

¿Por qué tu código aplica correctamente para Diabetes Mellitus?

La inmensa mayoría son Tipo 2: En adultos y en análisis de datos
poblacionales, más del 90-95% de los casos de diabetes no diagnosticada
son Diabetes Mellitus Tipo 2. Es una enfermedad de desarrollo lento y
silencioso (asintomática durante años), por lo que es la que se detecta
en este tipo de cribados analíticos.
```

```
La Diabetes Tipo 1 rara vez es "oculta": La Diabetes Tipo 1 suele debutar
de forma aguda con síntomas muy severos y rápidos (cetoacidosis,
deshidratación grave), por lo que los pacientes acuden a urgencias y se
diagnostican de inmediato, raramente permaneciendo como un "caso oculto"
en un dataset de control de rutina.

Mismos criterios analíticos iniciales: Las guías clínicas (ADA/OMS)
aplican exactamente los mismos valores de glucosa y HbA1c para confirmar
el estado de hiperglucemia diabética en la Tipo 1, Tipo 2 o diabetes
secundaria.

Cómo se diferencian en la práctica médica

Si quisieras precisar exactamente el tipo de diabetes en la clínica, se
requieren pruebas adicionales que no suelen ser de rutina en un análisis
de sangre básico:

Tipo,Proporción típica,¿Por qué ocurre?,¿Cómo se diferencia en
laboratorio?
Diabetes Tipo 2,~90-95%,Resistencia a la insulina y deficiencia
relativa.,"C-péptido normal/alto, anticuerpos negativos, relacionada con
IMC/edad."
Diabetes Tipo 1,~5%,Destrucción autoinmune de las células beta del
páncreas.,"Anticuerpos positivos (anti-GAD, IA-2, ICA) y C-péptido muy
bajo o nulo."
LADA / LADA 1.5,~1-3%,Autoinmune de inicio lento en
adultos.,"Inicialmente parece Tipo 2, pero presenta anticuerpos
autoinmunes positivos."

Si tu dataset etiqueta la variable como diabetes_mellitus (que es el
término médico general), tu lógica es totalmente válida y rigurosa. Si
quieres afinar la redacción de tus hallazgos, puedes describirlos como
"Casos que cumplen criterios analíticos para Diabetes Mellitus
(predominantemente Tipo 2 no diagnosticada)".
```

Análisis

Al notar que el enunciado se refería específicamente a la diabetes mellitus tipo 2 y no a la diabetes en general, pregunté si el código seguía siendo correcto. Aunque olvidé especificar el tipo 2 en mi mensaje, la IA lo interpretó correctamente en su respuesta, por lo que el código se dio por aceptado.

Prompt

```
y esto es correcto para comprobar que los datos del dataset son malos?
pasame una fuente para comprobarlo
1. DISCORDANCIA ANTROPOMÉTRICA (IMC vs Perímetro de Cintura)
Obesidad grado II/III (IMC > 35) con cintura de persona delgada (< 70 cm)
o viceversa
incoherencia_antropo = (
```

```
    ((df["indice_masa_corporal"] > 35) & (df["perimetro_cintura"] < 70))
    | ((df["indice_masa_corporal"] < 18.5) & (df["perimetro_cintura"] >
    110))
).sum()
print(
    f"\n 2. Inconsistencia severa entre IMC y Perímetro de cintura:
    {incoherencia_antropo} casos"
)
1. DISCORDANCIA GLUCOSA vs HEMOGLOBINA GLICOSILADA
HbA1c normal (<5.5%) con Glucosa diabética severa (>180 mg/dL) o
viceversa
incoherencia_glucemica = (
    ((df["hemoglobina_glicosilada"] < 5.5) & (df["glucosa_ayunas"] >
    180))
    | ((df["hemoglobina_glicosilada"] > 9.0) & (df["glucosa_ayunas"] <
    80))
).sum()
print(
    f"\n 3. Incoherencia metabólica cruzada (Glucosa vs HbA1c):
    {incoherencia_glucemica} casos"
)
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Sí, es una excelente estrategia técnica. En análisis de datos e informática médica, esto se denomina detección de inconsistencias lógicas o biológicas cruzadas (cross-validation of biological plausibility). Tu código no solo demuestra que hay errores en los datos, sino que identifica imposibilidades fisiológicas o errores de transcripción/medición extremos.

¿Por qué son correctas tus reglas lógicas?

1. Incoherencia Antropométrica (IMC vs. Cintura)

IMC > 35 con Cintura < 70 cm: Salvo en casos extremadamente raros de fisicoculturistas con una masa muscular extrema (e incluso en ellos la cintura rara vez baja de 70 cm con 35 de IMC), es biológicamente incompatible tener una masa corporal compatible con Obesidad Grado II/III y un contorno abdominal de un adulto marcadamente delgado.

IMC < 18.5 con Cintura > 110 cm: El perímetro abdominal $\ge 110\text{ cm}$ indica obesidad abdominal severa según la OMS. Es físicamente imposible tener una cintura de ese tamaño y mantener un peso global tan bajo como para dar un IMC de bajo peso (< 18.5).

2. Incoherencia Glucémica (Glucosa en Ayunas vs. HbA1c)

```
HbA1c < 5.5% con Glucosa > 180 mg/dL: La HbA1c refleja el promedio de la
glucosa de los últimos 2-3 meses. Un valor de HbA1c de 5.5% corresponde a
una glucosa promedio estimada (eAG) de unos $111\text{ mg/dL}$. Una
glucosa puntual en ayunas de $> 180\text{ mg/dL}$ (hiperglucemia severa)
en alguien con promedio normal sostenido solo ocurriría en eventos de
estrés glucémico agudo extremo o administración de fármacos en urgencias,
pero en un dataset de control/cribado rutinario indica casi con certeza
un error de digitación o falta de ayuno.

HbA1c > 9.0% con Glucosa < 80 mg/dL: Una HbA1c de $9.0\%$ implica una
glucosa media diaria de $\approx 212\text{ mg/dL}$. Encontrar una glucosa
basal de $< 80\text{ mg/dL}$ es extremadamente improbable a menos que el
paciente haya recibido una dosis masiva de insulina justo antes de la
toma de muestra (o sea una muestra mal procesada donde la glucosa se haya
degradado por hemólisis en el tubo de ensayo).

Fuentes médicas y científicas oficiales para justificarlo

Para justificar estas reglas de limpieza de datos en la memoria o informe
de tu proyecto, puedes citar estas dos referencias oficiales:

Para la Incoherencia Antropométrica:

Fuente: World Health Organization (WHO) Guidelines & Technical Report
Series (Obesity: preventing and managing the global epidemic).

Enlace directo: WHO Guidelines: Waist Circumference and Waist-Hip Ratio

Justificación: La OMS establece la correlación directa entre el
incremento de la circunferencia de la cintura y el IMC para clasificar el
riesgo cardiometabólico. La desviación extrema entre ambos refleja error
de medición.

Para la Incoherencia Glucémica (Relación HbA1c y Glucosa Promedio):

Fuente: ADA Standards of Care in Diabetes (Table: Correlation of A1C with
Estimated Average Glucose - eAG).

Enlace directo: ADA Standards of Care: Diabetes Care Journal
```

Análisis

Al no estar seguro de si ambas comprobaciones eran correctas (aunque ya conocía el rango de IMC entre 35 y 70), le pedí a la IA que las verificara con fuentes fiables. Tras revisar la información proporcionada, se dieron por aceptadas.

Prompt

Quiero comprobar en mi df qué variables tienen varianza cercana a 0. Para ello, quiero que uses estas dos métricas de forma conjunta:

el ratio entre la frecuencia del valor más frecuente y la frecuencia del segundo valor más frecuente (ratio de frecuencia): 1 para variables balanceadas, grande para variables mal balanceadas.
el porcentaje de valores únicos sobre el total objetos, que se aproximará cero a medida que la granularidad de la variable aumenta.

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
Para identificar qué variables en tu DataFrame tienen una varianza
cercana a 0 (también conocido en Machine Learning como características de
*cuasi-cero varianza* o *near-zero variance*, similar al enfoque del
paquete `caret` en R), puedes implementar una función en Python que
calcule ambas métricas de forma conjunta.

### Definición de las métricas

1. **Ratio de frecuencia:** Mide el desequilibrio entre el valor más
frecuente y el segundo más frecuente
($\frac{\text{Frecuencia}_1}{\text{Frecuencia}_2}$). Si la variable está
perfectamente equilibrada, tiende a 1; si es casi constante, este número
se dispara (o se vuelve infinito si solo existe un valor único).
2. **Porcentaje de valores únicos:** Mide la granularidad calculando qué
tanto porcentaje representan los valores únicos frente al total de filas
($\frac{\text{Valores únicos}}{\text{Total de filas}} \times 100$). En
variables constantes o con muy poca variabilidad, este porcentaje se
aproxima a 0.

---

### Código en Python

Puedes utilizar la siguiente función para analizar tu DataFrame (`df`):

```python
import pandas as pd
import numpy as np

def detectar_baja_varianza(df, freq_ratio_umbral=19,
unique_pct_umbral=10):
    """
    Identifica variables con varianza cercana a 0 usando el ratio de
    frecuencia
    y el porcentaje de valores únicos.

    Parámetros:
    - df: DataFrame de pandas a analizar.
    - freq_ratio_umbral: Umbral mínimo para considerar el ratio
    desequilibrado (por defecto 19:1).
    - unique_pct_umbral: Umbral máximo del porcentaje de valores únicos
    sobre el total (por defecto 10%).
```
```

```
    """
    resultados = []
    n_filas = len(df)

    for col in df.columns:
        # Contar frecuencias de cada valor (incluyendo nulos si se desea
        evaluar su peso)
        conteo_valores = df[col].value_counts(dropna=False)

        if len(conteo_valores) == 0:
            continue

        n_unicos = len(conteo_valores)
        pct_unicos = (n_unicos / n_filas) * 100

        # Calcular el ratio de frecuencia
        if n_unicos >= 2:
            freq_1 = conteo_valores.iloc[0]
            freq_2 = conteo_valores.iloc[1]
            # Evitar división por cero si la frecuencia del segundo es 0
            freq_ratio = freq_1 / freq_2 if freq_2 > 0 else np.inf
        else:
            # Si solo hay 1 valor único (columna 100% constante)
            freq_ratio = np.inf

        resultados.append({
            'variable': col,
            'n_unicos': n_unicos,
            'porcentaje_unicos': pct_unicos,
            'ratio_frecuencia': freq_ratio,
            'valor_mas_frecuente': conteo_valores.index[0],
            'freq_valor_mas_frecuente': conteo_valores.iloc[0]
        })

    df_metricas = pd.DataFrame(resultados)

    # Filtrar aquellas variables que cumplen ambas condiciones de baja
    varianza
    df_baja_var = df_metricas[
        (df_metricas['ratio_frecuencia'] > freq_ratio_umbral) &
        (df_metricas['porcentaje_unicos'] < unique_pct_umbral)
    ].reset_index(drop=True)

    return df_metricas, df_baja_var

# Ejemplo de uso:
# metricas_completas, columnas_baja_varianza = detectar_baja_varianza(df)
# print(columnas_baja_varianza)

```
```
```

```
### ¿Cómo interpretar los resultados?

* **Variables descartables por baja varianza:** Aquellas donde el
`ratio_frecuencia` supere un umbral estricto (por ejemplo, mayor a `19`,
lo que significa que el valor más común aparece al menos 19 veces más que
el segundo) **Y** cuyo `porcentaje_unicos` sea extremadamente bajo (por
ejemplo, menor al `10%`).
* **Valores infinitos (`inf`):** Si una columna tiene un único valor en
todo el DataFrame (variable completamente constante), el segundo valor no
existe, por lo que el `ratio_frecuencia` se registrará como `inf` y el
porcentaje de únicos será cercano a 0 (`1 / n_filas`), detectándose de
forma inmediata como un caso crítico de varianza cero.
```

Análisis

En la consulta se pide que se realice un código para calcular la varianza cercana a 0 como se indica en los apuntes. El código es correcto con ls dos métricas que se indica. Sin embargo, no estoy de acuerdo con utilizar los valores nulos, ya que se están contando como un valor más y aparecer como el valor con más frecuencia. Tampoco me gusta que no se impriman los valores ni que esté considerando el target. Por lo tanto, se acepta el código pero para usarlo de base y hacerle modificaciones. No estoy seguro sobre el valor de los dos límites establecidos.

Prompt

lo de los dos límites no me convencen. No has seguido ningún criterio ni ningún estándar

Respuesta IA Gemini Flash Lite 3.5

```
¡Qué va! Lejos de ser una ocurrencia al azar, esos dos números
específicos (`19` y `10`) responden a un **estándar muy conocido y
consolidado en Machine Learning y ciencia de datos**: las heurísticas del
paquete **`caret` de R**, diseñado por Max Kuhn (autor de referencia y
coautor de *Applied Predictive Modeling*).

Este enfoque se creó precisamente para solucionar el problema de las
variables de **cuasi-cero varianza** (*near-zero variance* o NZV).

Los criterios detrás de estos números son los siguientes:

* **El ratio de frecuencia = 19 (la regla del 95/5):**
Equivale a un desequilibrio donde el valor más frecuente aparece **19
veces más** que el segundo valor más frecuente (es decir, una
distribución aproximada del 95% frente al 5%). Max Kuhn estableció
empíricamente este umbral para detectar variables donde una categoría
domina abrumadoramente y aporta poquísima información útil al modelo.
* **El porcentaje de valores únicos < 10%:**
```

```
Actúa como una red de seguridad para medir la granularidad. Ayuda a
distinguir variables que, aunque tengan múltiples valores, están tan
concentrados en muy pocas categorías o rangos que su variedad real es
mínima frente al tamaño total del dataset (`n_filas`).
```

Análisis

Lejos de pensar que estos valores son al azar, la IA me demuestra que los límites establecidos son parte de un estándar muy consolidado (el cual al investigar nearZeroVar de caret se pudo comprobar que vienen dichos valores por defecto), por lo que se acepta la sugerencia.

Alejandro:

Prompt

en el notebook de preprocesamiento vamos a estructurar el trabajo comparando una referencia y preprocesados incrementales donde cada uno añade una sola técnica para poder aislar qué mejora cada cosa. hemos definido p0 como referencia basica con dummies y mediana sin contar como tecnica, p1 que es la limpieza clinica con standardscaler, y dejamos p2 y p3 por decidir. redactame la introduccion del notebook y la tabla de resumen explicando que p0 no cuenta como tecnica para cumplir lo del apartado 3.1 del enunciado

Respuesta IA Gemini Flash 3.8 en Google AI Studio

````
Aquí tienes la redacción del planteamiento inicial para el encabezado del
notebook:

# Preprocesamiento
Apartado del enunciado: 3.1, mínimo tres técnicas de preprocesamiento,
indicando qué problemática resuelve cada una.

Este cuaderno define y justifica los preprocesados que se compararán
después con todos los modelos. Todo el código está en las celdas.

## Planteamiento
Se definen una referencia y preprocesados incrementales. Cada uno añade
una sola técnica al anterior. Así, la diferencia de métricas entre dos
preprocesados consecutivos se puede atribuir a la técnica añadida, y no a
una mezcla de cambios.

| ID | Técnica que añade | Problemática del EDA que resuelve |
| :--- | :--- | :--- |
| P0 | Referencia: dummies y mediana (no cuenta como técnica) | Ninguna.
Es la base de comparación |
````

```
| P1 | Técnica 1: limpieza clínica + StandardScaler | Valores imposibles,
menores, colesterol_total incoherente, ejercicio no entero, nulos,
escalas distintas |
| P2 | Técnica 2: por decidir | Ver "Problemáticas pendientes" al final |
| P3 | Técnica 3: por decidir | Ver "Problemáticas pendientes" al final |

Esta disposición garantiza que P0 actúe como control metodológico
estricto, permitiendo auditar el impacto marginal neto de cada decisión
de ingeniería de datos.
```

Análisis

Se acepta el esquema planteado. Acordamos en el equipo comparar los preprocesados de forma estrictamente incremental (P0 ->P1 ->P2 ->P3) para no mezclar técnicas a la vez y poder justificar en la memoria qué problemática resuelve cada una sin ambigüedades.

Prompt

en la reunion acordamos hacer una particion train test 80/20 estratificada con semilla 123. el test lo queremos reservar para la evaluacion final y todo lo demas se elegira con validacion cruzada sobre el train. ademas el test tiene que tener solo adultos porque es lo que pide el enunciado. dame el codigo en python que haga la particion, filtre los menores del test y compruebe que no queda ningun menor en test y que las proporciones del target en train y test son iguales

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Aquí tienes el bloque de código para realizar la partición preservando las proporciones del target y asegurando la integridad del test:

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

pd.set_option("display.max_colwidth", None, "display.width", 200)

SEED = 123
DATA_PATH = "../data/raw/enfermedad_cardiovascular.csv"
TARGET = "riesgo_cardiovascular"
EDAD_MINIMA = 18

df = pd.read_csv(DATA_PATH)
X, y = df.drop(columns=TARGET), df[TARGET]

# Partición 80/20 estratificada por el target
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED
)
```

```
# El test solo debe contener adultos según el criterio del estudio
filtro_adultos_te = X_te["edad"] >= EDAD_MINIMA
X_te, y_te = X_te[filtro_adultos_te], y_te[filtro_adultos_te]

print("Total", X.shape, "| train", X_tr.shape, "| test", X_te.shape)
print("Menores: en train", int((X_tr["edad"] < EDAD_MINIMA).sum()),
      "| en test", int((X["edad"] < EDAD_MINIMA).sum() - (X_tr["edad"] <
      EDAD_MINIMA).sum()))

# Comprobación de proporciones idénticas en ambas particiones
display(pd.DataFrame({
    "train": y_tr.value_counts(normalize=True),
    "test": y_te.value_counts(normalize=True)
}).sort_index().round(3))
```

Análisis

Se acepta el código para la partición hold-out. Al ejecutarlo se comprueba que de los 4 menores de la base de datos, los 4 cayeron por azar estratificado en el conjunto de entrenamiento y 0 en test, manteniendo las proporciones de las tres clases idénticas (50

Prompt

tenemos claras las reglas de lo que es imposible segun el eda: - hdl <= 0 y trigliceridos <= 0 - alcohol <0 - imc <13 - ejercicio >7 - hba1c <3 - presion sistolica <= diastolica o sistolica <60 o diastolica <40 - anos fumador >edad y para eliminar filas solo los menores (<18) y si tiene mas anos fumando que edad porque ese paciente no tiene sentido. hazme dos funciones: $\text{mascaras}_{i}nvalidas(X)quedevuelvaundiccionarioconlasmascarasbooleanasdecadacolumnayfilas_{ae}l$

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Aquí tienes la implementación modular de ambas funciones lógicas basadas en tus umbrales de dominio clínico:

```
EDAD_MINIMA = 18
IMC_MINIMO = 13
HBA1C_MINIMA = 3
EJERCICIO_MAXIMO = 7
SISTOLICA_MINIMA = 60
DIASTOLICA_MINIMA = 40

def mascaras_invalidas(X):
    """Devuelve {columna: máscara booleana} con las celdas imposibles
    según el EDA."""
    invertida = X["presion_sistolica"] <= X["presion_diastolica"]
    return {
```

```
        "colesterol_hdl": X["colesterol_hdl"] <= 0,
        "trigliceridos": X["trigliceridos"] <= 0,
        "consumo_alcohol_semanal": X["consumo_alcohol_semanal"] < 0,
        "indice_masa_corporal": X["indice_masa_corporal"] < IMC_MINIMO,
        "frecuencia_ejercicio_semanal": X["frecuencia_ejercicio_semanal"]
        > EJERCICIO_MAXIMO,
        "hemoglobina_glicosilada": X["hemoglobina_glicosilada"] <
        HBA1C_MINIMA,
        "presion_sistolica": invertida | (X["presion_sistolica"] <
        SISTOLICA_MINIMA),
        "presion_diastolica": invertida | (X["presion_diastolica"] <
        DIASTOLICA_MINIMA),
        "anos_fumador": X["anos_fumador"] > X["edad"],
    }

def filas_a_eliminar(X):
    """Menores y pacientes con más años fumando que edad."""
    return (X["edad"] < EDAD_MINIMA) | (X["anos_fumador"] > X["edad"])
```

Análisis

Se aceptan las funciones. Al ser reglas fijas del dominio médico que no calculan medias ni parámetros estadísticos de los datos, no provocan fuga de información aunque se evalúen sobre el dataframe antes de pasar al pipeline.

Prompt

dame una celda que junte las mascaras de imposibles que acabamos de definir con el conteo de valores nulos del dataframe para tener una tabla ordenada con todos los problemas detectados en cada columna, y que me imprima cuantas filas en total tienen algun valor imposible y cuantas filas se van a eliminar para entrenar

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Con este bloque generas el informe cuantitativo consolidado de celdas
imposibles y valores nulos:

mascaras = mascaras_invalidas(X)
resumen = pd.DataFrame({
    "imposibles": {c: int(m.sum()) for c, m in mascaras.items()},
    "nulos": X.isna().sum(),
}).fillna(0).astype(int)

# Filtramos solo las columnas que tienen al menos un problema registrado
resumen = resumen[resumen.sum(axis=1) > 0].sort_values(["imposibles",
"nulos"], ascending=False)

print("Filas con algún valor imposible:", int(pd.concat(mascaras,
axis=1).any(axis=1).sum()), "de", len(X))
```

```
print("Filas que no se usan para entrenar:",
int(filas_a_eliminar(X).sum()))
display(resumen)
```

Análisis

Se acepta el código para resumir las problemáticas. Los resultados muestran que hay 279 filas afectadas por valores imposibles en todo el dataset (siendo alcohol con 177 casos y las presiones con 44 y 47 los mayores causantes), mientras que las filas a descartar para entrenar son solo 4.

Prompt

en el informe tenemos que justificar por que no eliminamos los valores atipicos que no son imposibles tipo la pcr muy alta o la glucosa alta. redactame la justificacion clinica de por que solo corregimos lo imposible y conservamos el resto de atipicos

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Aquí tienes la justificación metodológica y clínica para el apartado de
criterios generales:

### 4.1 Atípicos: solo se corrige lo imposible
Decisión: un valor solo se modifica si es imposible según las reglas de
dominio. El resto de atípicos se conserva.

Justificación:
- En medicina, los valores extremos (PCR, glucosa, triglicéridos o
presión arterial muy altas) no son fallos de lectura, sino que
caracterizan precisamente a los pacientes con daño vascular severo o alto
riesgo cardiovascular (Clase 2), que es el grupo clínico crítico que el
sistema debe detectar.
- Eliminar o recortar a ciegas estos registros privaría a los modelos de
la señal predictiva más relevante.
- La problemática de su impacto sobre estimadores sensibles a la escala
(regresión logística, SVM) se abordará mediante escalado robusto o
transformaciones en los preprocesados posteriores, sin necesidad de
destruir muestras válidas de pacientes reales.
```

Análisis

Nos quedamos con la justificación de momento, aunque sea posible que en futuras iteraciones la cambiemos

Prompt

dame una celda para justificar que nos quedamos con nivel actividad aunque sea subjetiva. saca una tabla con el conteo de cada nivel, su ejercicio medio y su riesgo medio para justificar por que la mantenemos en el notebook

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Esta tabla sintetiza la paradoja entre la percepción subjetiva y el valor
predictivo de la variable:

ORDINAL = "nivel_actividad"
ORDEN_ACTIVIDAD = {"Sedentario": 0, "Ligeramente activo": 1,
"Moderadamente activo": 2, "Muy activo": 3}

display(pd.DataFrame({
    "n": X[ORDINAL].value_counts(),
    "ejercicio_medio":
    X.groupby(ORDINAL)["frecuencia_ejercicio_semanal"].mean(),
    "riesgo_medio": y.groupby(X[ORDINAL]).mean(),
}).loc[list(ORDEN_ACTIVIDAD)].round(2))
```

Análisis

Se acepta el código y se incorpora la tabla. Aunque el ejercicio medio apenas varía entre sedentarios (2.66 días) y muy activos (2.79 días), el riesgo cardiovascular medio cae de forma estrictamente monótona del 0.98 al 0.36, demostrando que aporta una señal predictiva enorme que no se debe eliminar.

Prompt

para la tecnica 1 (p1) de limpieza clinica hemos decidido recortar al limite valido cosas como alcohol negativo, imc bajo, ejercicio >7 o hba1c <3, en vez de poner nan o eliminar filas. explicame en texto para el notebook por que recortar conserva mejor la señal que imputar con la mediana usando el dato del riesgo medio del alcohol

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Aquí tienes la justificación comparativa de las tres estrategias:

### 6.1 ¿Por qué recortar los imposibles y no pasarlos a NaN ni eliminar
la fila?
Hay tres estrategias posibles:

| Estrategia | Problema |
| :--- | :--- |
| Eliminar la fila | Se perderían 279 de 2.254 pacientes (12% del total
de la cohorte). |
| Pasar a NaN e imputar con la mediana | Borra la información y
distorsiona el perfil real del paciente. |
| Recortar al límite válido | Conserva el orden de magnitud y la señal
clínica original. |
```

El diagnóstico evidenció que los valores imposibles no son ruido puramente estocástico: los pacientes con consumo_alcohol_semanal negativo presentan un riesgo medio de 0.21, frente al 0.70 de la media global de la cohorte. Un valor de -5 procede de ruido de generación sobre pacientes no bebedores. Si se recorta a 0, la variable sigue indicando "abstemio / consumo nulo", preservando intacta su condición protectora. Si se pasara a NaN, la mediana le asignaría un consumo positivo medio falseando su estilo de vida.

Para la presión arterial y los años de tabaquismo no existe un límite fijo universal porque dependen de otra variable acoplada (la diastólica y la edad), por lo que en esos casos sí se convierten a NaN para imputación controlada.

Análisis

Se adopta la explicación como base para la memoria del notebook.

Prompt

dame la clase TratarImposibles compatible con scikit learn que use las funciones de antes y recorte con LIMITES$_{RECORTE}$ las que tienen limite propio, ya presion y anos fumador les meta NaN

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Aquí tienes la implementación del transformador personalizado para Scikit-Learn:

```
from sklearn.base import BaseEstimator, TransformerMixin

# Límite válido al recortar: (mínimo, máximo); None indica sin límite por
ese extremo
LIMITES_RECORTE = {
    "colesterol_hdl": (1, None),
    "trigliceridos": (1, None),
    "consumo_alcohol_semanal": (0, None),
    "indice_masa_corporal": (IMC_MINIMO, None),
    "frecuencia_ejercicio_semanal": (None, EJERCICIO_MAXIMO),
    "hemoglobina_glicosilada": (HBA1C_MINIMA, None),
}

class TratarImposibles(BaseEstimator, TransformerMixin):
    """Recorta los imposibles a su límite válido. Si la variable no tiene
    límite propio
    (presión y años fumando), la celda pasa a NaN. No aprende de los
    datos, fit devuelve self."""

    def fit(self, X, y=None):
        return self
```

```
    def transform(self, X):
        X = X.copy()
        for columna, mascara in mascaras_invalidas(X).items():
            if columna in LIMITES_RECORTE:
                X[columna] = X[columna].clip(*LIMITES_RECORTE[columna])
            else:
                X.loc[mascara, columna] = np.nan
        return X
```

Análisis

Se acepta e incluye la clase en el cuaderno.

Prompt

quiero comprobar con datos que la presion de pulso solo da problemas cuando es menor o igual a cero y no cuando es baja pero positiva. hazme una celda que calcule la diferencia entre sistolica y diastolica, la divida en los tramos <=0, de 0 a 10, de 10 a 25 y mayor de 25, y me saque el recuento de pacientes y el riesgo medio de cada tramo junto a los que tienen sistolica <60 y diastolica <40

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Este análisis empírico valida la decisión sobre el umbral hemodinámico:

pulso = X["presion_sistolica"] - X["presion_diastolica"]
tramos = pd.cut(pulso, [-np.inf, 0, 10, 25, np.inf], labels=["<= 0", "(0,
10]", "(10, 25]", "> 25"])

display(pd.DataFrame({
    "n": tramos.value_counts(),
    "riesgo_medio": y.groupby(tramos, observed=False).mean()
}).round(2))

print("Sistólica < 60:", int((X["presion_sistolica"] <
SISTOLICA_MINIMA).sum()),
      "| diastólica < 40:", int((X["presion_diastolica"] <
      DIASTOLICA_MINIMA).sum()))
```

Análisis

Se integra la celda.

Prompt

para colesterol_total hemos visto que no cuadra con la formula de friedewald (hdl + ldl + tg/5) y que en un 26 por ciento de los pacientes es menor que hdl + ldl. dame el codigo que calcule la correlacion entre el valor registrado y la formula, el porcentaje de casos do

hdl+ldl, la correlacion de ambos con el target y los casos fuera de fridewald ese. redacta me tambien el text

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Aquí tienes el bloque de análisis y la justificación metodológica completa:

```python
formula = X["colesterol_hdl"] + X["colesterol_ldl"] + X["trigliceridos"]
/ 5
validos = X["colesterol_ldl"].notna()

print("Correlación registrado-fórmula:",
round(X["colesterol_total"].corr(formula), 3))
print("Total registrado < HDL + LDL:",
      f"{(X['colesterol_total'] < X['colesterol_hdl'] +
      X['colesterol_ldl'])[validos].mean():.1%}",
      "de los pacientes con LDL conocido")
print("Correlación con el target: registrado",
round(X["colesterol_total"].corr(y), 3),
      "| fórmula", round(formula.corr(y), 3))
print("Triglicéridos > 400 (fuera del rango de Friedewald):",
int((X["trigliceridos"] > 400).sum()),
      "| nulos de LDL:", int(X["colesterol_ldl"].isna().sum()))
```

6.4 Colesterol total: se elimina

Decisión: colesterol_total se elimina definitivamente de la matriz de características. No se recalcula con la fórmula.

Por qué el valor registrado no sirve:
- La correlación entre el valor registrado y la fórmula bioquímica es de apenas 0.026 (nula).
- En el 26.4% de los sujetos con LDL medido, el colesterol total es estrictamente inferior a la suma de sus fracciones HDL + LDL, lo cual representa una imposibilidad física.
- Su correlación con el riesgo cardiovascular es residual (0.082).

Por qué no recalcularla mediante Friedewald:
1. No aporta información nueva: sería una combinación lineal exacta de HDL, LDL y triglicéridos, variables que los modelos ya reciben directamente.
2. Es una estimación con restricciones: la ecuación no es válida con triglicéridos > 400 mg/dL (54 casos en la cohorte).
3. Propagación de imputaciones: al tener colesterol_ldl 135 valores nulos, recalcular el total forzaría a imputar previamente el LDL, transmitiendo el error de imputación a una variable sintética artificial.
4. No mejora el poder discriminatorio: la fórmula mantiene una correlación casi idéntica con el target (0.074).

Análisis

Se acepta tanto el bloque de código. El texto no sirve como base para la explicación.

Prompt

hazme la clase TransformarColumnas que aplique en transform() el redondeo hacia arriba con np.ceil en frecuencia$_e$$jercicio_s$$emanal, conviertanivel_a$$ctividadanumerosconeldiccionarioORDEN_ACTIVID$

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Aquí tienes la clase TransformarColumnas adaptada al flujo de
transformadores del pipeline:

class TransformarColumnas(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X["frecuencia_ejercicio_semanal"] =
        np.ceil(X["frecuencia_ejercicio_semanal"])
        X[ORDINAL] = X[ORDINAL].map(ORDEN_ACTIVIDAD).astype(float)
        return X.drop(columns="colesterol_total")
```

Análisis

Se acepta el código de la clase. Agrupa limpiamente las transformaciones de columnas de P1 sin necesidad de almacenar estado.

Prompt

dame la celda final que aplique TratarImposibles y TransformarColumnas sobre X y compare en una tabla el minimo, maximo y numero de nulos antes y despues en todas las columnas afectadas, ademas de comprobar que el ejercicio pasa a tener solo valores enteros del 0 al 7 y como cambia su media

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Con esta celda se audita cuantitativamente el efecto exacto de la técnica
de limpieza:

limpio =
TransformarColumnas().fit_transform(TratarImposibles().transform(X))
cols = list(mascaras)

display(pd.DataFrame({
    "min_original": X[cols].min(), "min_limpio": limpio[cols].min(),
    "max_original": X[cols].max(), "max_limpio": limpio[cols].max(),
```

```
    "nulos_original": X[cols].isna().sum(), "nulos_limpio":
    limpio[cols].isna().sum(),
}).round(2))

e0, e1 = X["frecuencia_ejercicio_semanal"],
limpio["frecuencia_ejercicio_semanal"]
print("Ejercicio. Antes: no enteros", f"{(e0 != e0.round()).mean():.1%}",
"| media", round(e0.mean(), 2),
      "|| Después: valores", sorted(e1.unique().astype(int)), "| media",
      round(e1.mean(), 2))
```

Análisis

Se acepta y se comprueba la tabla de salida. Se constata que los mínimos negativos desaparecen (alcohol pasa de -5.0 a 0.0, HDL y triglicéridos a 1.0, IMC a 13.0), el ejercicio pasa de tener un 81.5

Alejandro:

Prompt

```
Queremos documentar en el EDA la justificación bibliográfica de los
mínimos plausibles para la presión sistólica (60 mmHg) y diastólica (40
mmHg). Busca y verifica fuentes médicas acreditadas para citarlas. Además,
ayuda a modularizar el código de preprocesamiento que diseñamos en el
borrador técnico del workspace para estructurarlo en funciones
reproducibles y extensibles a nuevos preprocesados sin salir del
directorio de trabajo.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
EDA: añadió el apartado "Mínimos plausibles de la presión arterial"
(sistólica 60 mmHg, diastólica 40 mmHg) con tres enlaces que comprobó
antes de citarlos:
- NHLBI: la hipotensión empieza por debajo de 90/60 mmHg.
- MedlinePlus: la presión normal está entre 90/60 y 120/80 mmHg; el shock
es una emergencia.
- Surviving Sepsis Campaign 2021: objetivo de presión arterial media >=
65 mmHg. Con 60/40 sale unos 47 mmHg.
Añadió también código que cuenta los 4 pacientes por debajo de los
mínimos.

Preprocesamiento: estructuró el cuaderno siguiendo las especificaciones
del borrador técnico:
- Cada paso es una función df -> df que imprime lo que hace de forma
transparente.
- Las reglas de valores imposibles e incoherencias están en dos tablas
(REGLAS_RANGO, REGLAS_CRUZADAS).
- PREPROCESADOS guarda cada preprocesado como una lista de pasos: P0_base,
P1_limpieza, P2_diabetes, P3_pcr.
```

```
- Codificación, mediana y escalado dentro de un Pipeline ajustado solo
con el train.
- Evaluación con modelos de referencia (Random Forest, SVM, KNN),
computando accuracy, F1 macro y recall de alto riesgo en train y test.

Ajustes técnicos adicionales: particiones estratificadas compartidas
entre variantes, preservación de pacientes con PCR nula al filtrar casos
agudos (> 10 mg/L) y fijación global de SEED.
```

Análisis

El diseño de la limpieza (definición de cotas, reglas cruzadas y comparación incremental P0–P3) fue definido previamente por el equipo. La herramienta se utilizó para agilizar la modularización del código y localizar fuentes oficiales que respalden los umbrales de shock circulatorio. Se verificó manualmente la validez de los enlaces clínicos incorporados. Al refactorizar, la herramienta omitió el redondeo hacia arriba del ejercicio que teníamos previsto; se ordenó su reincorporación en la interacción posterior.

Prompt

```
Aplica una regularización mínima de control a los clasificadores base
para mitigar el sobreajuste severo sin entrar aún en optimización formal.
Reincorpora el redondeo hacia arriba en la frecuencia de ejercicio
semanal y realiza una auditoría comparativa frente a los hallazgos del
EDA para reportar qué inconsistencias quedan sin tratar en esta primera
iteración.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
Modelos (ajuste de control para estabilizar la evaluación preliminar):
- Random Forest: max_depth=8, min_samples_leaf=5. Diferencia F1
train-test: 0,39 -> 0,31.
- SVM: C=10 -> C=1. Diferencia F1 train-test: 0,41 -> 0,21; F1 en test:
0,59 -> 0,65.

Redondeo: frecuencia_ejercicio_semanal hacia arriba, posterior al
enmascaramiento de valores > 7.

Comparación con el EDA (auditoría informativa de 10 puntos):
1. Desbalanceo de clases sin tratar (class_weight="balanced" sube el
recall de alto riesgo de 0,53 a 0,75).
2. Asimetría de PCR y ácido úrico sin transformar.
3. Sensibilidad a valores atípicos en estimadores lineales/geométricos
(SVM y KNN).
4. Discrepancia entre nivel_actividad subjetivo y frecuencia deportiva.
5. Presencia de decimales continuos en otras magnitudes discretas.
6. Mínimos biológicos pendientes en sueño, cintura y pulso.
7. Presencia de años de exposición tabáquica en no fumadores activos.
```

```
8. Mecanismo de generación de datos faltantes no categorizado.
9. Conflicto en el orden de ejecución: la regla sistólica < 60 se evalúa
antes de la transposición diastólica >= sistólica.
10. La regla de años fumando > edad no afecta a adultos (el único caso
era menor de edad).
```

Análisis

Se restringió la intervención de la IA a una regularización de salvaguarda básica, ya que el ajuste exhaustivo de hiperparámetros corresponde a fases posteriores (requisito 3.3). Asimismo, se exigió que el chequeo de discrepancias con el EDA fuera estrictamente analítico, reservando al equipo la decisión de qué problemáticas abordar en cada variante.

Prompt

```
Respecto al informe anterior, ¿se ha corregido la prioridad en la regla
de presión arterial? Por otra parte, para comparar los preprocesados no
podemos basarnos únicamente en valores medios; incluye la desviación
estándar (media +- std) en todas las métricas para evaluar si las
diferencias observadas son estadísticamente significativas o mero ruido
de partición.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
Punto 9 (orden de reglas): se mantenía sin corregir a nivel de código,
figurando únicamente como advertencia en el informe.

Métricas y variabilidad: se actualizaron todas las salidas para presentar
media +- desviación típica a lo largo de las iteraciones de validación,
computando también la brecha train-test. Al visibilizar la dispersión, se
comprueba que las variaciones de rendimiento entre las primeras variantes
de preprocesamiento se encuentran dentro del margen de error de la
partición.
```

Análisis

Intervención crítica del equipo: la IA calculaba únicamente métricas puntuales promedio, lo que inducía al error metodológico de atribuir mejoras a cambios de preprocesamiento cuando en realidad quedaban solapados por la varianza de la partición. Incorporar la desviación típica aportó el rigor empírico necesario para interpretar los resultados de forma realista.

Prompt

```
Explica con precisión cuál es el conflicto técnico en el punto 9 con el
orden de las reglas de presión y por qué afecta al filtrado de datos.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
Un paciente de la cohorte presenta sistólica de 52,2 mmHg y diastólica de
90,6 mmHg (caso simultáneo de hipotensión severa e inversión de
presiones).
Si se aplica primero la regla de rango mínimo (sistólica < 60), el valor
52,2 se sustituye inmediatamente por NaN.
Al evaluarse a continuación la regla cruzada (diastólica >= sistólica),
la comparación lógica 90,6 >= NaN resulta en False. Como consecuencia, el
registro no se cataloga como presión invertida y la diastólica errónea de
90,6 mmHg sobrevive intacta.
Invirtiendo el orden, se detecta primero la incongruencia global y ambas
magnitudes se convierten a NaN para su imputación correcta.
```

Análisis

Se solicitó el diagnóstico detallado para verificar el impacto matemático antes de autorizar cualquier modificación del pipeline. Se constató que se trataba de un efecto de borde generado por la propagación de nulos en comparaciones booleanas sobre Pandas.

Prompt

```
Reordena las operaciones en el pipeline para anteponer la detección de
presión invertida a los límites individuales y documenta brevemente el
motivo en la sección correspondiente del notebook.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
Se independizó el paso en la función nan_presion_invertida, ubicándolo
con precedencia sobre la comprobación de límites absolutos. Se redactó
una nota técnica explicativa en el apartado 3.2. Con este cambio, los 44
casos invertidos son capturados íntegramente y la regla de corte inferior
a 60 mmHg pasa a computar 0 incidencias redundantes.
```

Análisis

Se validó la solución inspeccionando directamente el índice del paciente afectado, verificando que ambos campos hemodinámicos quedan asignados a NaN de forma consistente para su tratamiento conjunto.

Prompt

```
Los estimadores basados en árboles en versiones modernas de scikit-learn
gestionan de forma nativa los valores faltantes. ¿Se está teniendo esto
en cuenta en el diseño, o se está forzando la imputación por mediana a
todos los modelos por defecto?
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
El pipeline inicial aplicaba SimpleImputer(strategy="median")
indistintamente a toda la batería de clasificadores. En scikit-learn (>=
1.4), RandomForestClassifier soporta valores NaN en el entrenamiento
mediante enrutamiento por histograma/potencial de partición. Una prueba
preliminar evidenció que la discrepancia en F1 entre entrenar con soporte
nativo de NaN o con imputación por mediana queda contenida dentro del
rango de la desviación típica.
```

Análisis

Corrección metodológica planteada por el equipo: cuestionamos la aplicación homogénea de la imputación univariante sobre arquitecturas que admiten datos faltantes de manera intrínseca. Esta observación sirvió para diseñar un sub-experimento específico que evalúa el impacto de la imputación sobre ensambles frente a modelos de vectores de soporte y vecinos más cercanos.

Prompt

```
¿Es metodológicamente riguroso comparar un Random Forest que procesa NaN
de forma nativa frente a modelos como SVM que exigen imputación previa?
Además, añade una celda explicativa en el EDA detallando el
comportamiento y la casuística de las presiones invertidas.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
La comparativa es metodológicamente válida siempre que compartan
idénticas particiones de train/test y se evalúen con las mismas métricas
sin filtración. No obstante, cambia el foco de la hipótesis:
- Comparar modelos entre sí exige condiciones de preprocesamiento
homogéneas (todos con imputación).
- Comparar Random Forest (con mediana) frente a Random Forest (nativo con
NaN) aísla el impacto de la técnica de imputación.

En el EDA se añadieron dos bloques que documentan la presencia de los 44
casos invertidos, los 4 registros bajo el umbral vital y el caso singular
que compartía ambas anomalías, justificando la secuencia requerida en la
limpieza.
```

Análisis

Se aceptó la puntualización teórica. Se acordó mantener la imputación homogénea como base para el contraste inter-modelo, reservando la variante sin imputación como un análisis ablación específico sobre el ensamble para responder al apartado 3.1 del enunciado.

Prompt

```
Define formalmente los objetivos del notebook de preprocesamiento:
evaluar el impacto relativo de las técnicas de limpieza bajo un esquema
de regularización base y analizar experimentalmente si la imputación
explícita aporta ventaja en ensambles frente a su manejo nativo.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
- Se formalizaron ambos objetivos al inicio del cuaderno.
- Se parametrizó construir_pipeline(imputar=True/False) para desplegar en
paralelo la variante "Random Forest (NaN)".
- Se añadió una aserción de control verificando scikit-learn >= 1.4.
- Se implementó el cálculo de diferencias pareadas (Delta = F1_nativo -
F1_imputado) sobre idénticos pliegues.
Resultado cuantitativo: el Delta medio se sitúa cercano a 0 con una
desviación que absorbe la diferencia, concluyendo que la imputación
simple no degrada ni beneficia de forma crítica al ensamble.
```

Análisis

Se validó la estructura de evaluación comparativa por pares. El análisis confirma empíricamente que la imputación por mediana constituye un procedimiento neutro y seguro que permite alimentar de forma equitativa tanto a los modelos lineales/paramétricos como a los algoritmos basados en árboles.

Prompt

```
Revisemos las advertencias sobre variables fisiológicas: las horas de
descanso nocturno y la circunferencia de cintura registradas son
clínicamente viables dentro de los rangos de la muestra. En cuanto a la
frecuencia cardíaca, detalla la plausibilidad médica de los valores
inferiores a 40 lpm antes de tomar cualquier decisión de filtrado. Además,
implementa el redondeo a enteros en las magnitudes discretas pendientes.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
Redondeo discreto: se estructuró la función redondear_enteros mediante el
diccionario REDONDEO, aplicando casteo al entero más próximo sobre
anos_fumador, frecuencia_cardiaca_reposo y consumo_alcohol_semanal. La
variación en media y correlación con el target resultó insignificante (<
0,01).
```

```
Frecuencia cardíaca en reposo < 40 lpm: representa bradicardia severa,
pero no constituye necesariamente una imposibilidad médica (fisiología
del deportista de fondo, bloqueo auriculoventricular o tratamiento
concomitante con betabloqueantes). De los 10 registros identificados,
solo 2 (25,8 y 26,6 lpm) presentan carácter dudoso extremo, mientras que
los restantes (38--40 lpm) son clínicamente plausibles. Se adoptó el
criterio de conservarlos al igual que las cotas de sueño y perímetro de
cintura.
```

Análisis

Criterio propio del equipo frente a la sugerencia inicial de la IA: la herramienta tendía a clasificar valores estadísticamente raros como errores que debían suprimirse. Defendimos que las horas de sueño reportadas y las medidas corporales reflejan la variabilidad poblacional real de la cohorte. En la frecuencia cardíaca, al representar menos del 0,5 % de los datos y existir justificación patológica plausible, se decidió mantener los registros para no comprometer la detección de perfiles críticos.

Prompt

```
Documenta esa conclusión en el EDA. En cuanto a la edad, dado que el
diccionario de variables especifica 'años cumplidos', el redondeo
matemático no es el más adecuado; debe aplicarse un redondeo hacia abajo
(función suelo / truncamiento entero) sobre la población adulta.
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
Variable edad: se corrigió el procedimiento aplicando np.floor tras el
filtrado de menores de 18 años, ajustándose estrictamente a la definición
de "años cumplidos". La edad mínima de la muestra se consolida en 18 años
exactos, no se generan anomalías en la relación edad-tabaquismo y la
correlación con la variable objetivo permanece inalterada.

EDA: se añadieron tres celdas analíticas documentando el espectro de
bradicardia severa (< 40 lpm), justificando la no exclusión de los
pacientes por plausibilidad fisiológica ambulatoria y escaso volumen
relativo (< 0,5\%).
```

Análisis

Corrección conceptual directa a la herramienta: la IA aplicaba un redondeo estándar (`round`) que en una persona de 17,9 años habría asignado artificialmente 18 años. Se impuso el uso de `np.floor` para respetar la semántica médica de años cumplidos tras asegurar el criterio de inclusión de adultos.

Análisis crítico

Correcciones del equipo a la IA

- **Evaluación con métricas de dispersión:** La IA reportaba inicialmente medias aritméticas aisladas en la comparativa de preprocesamientos. Se exigió reportar media ± desviación típica, lo que evidenció que las variaciones menores de F1 se debían a la partición y no al preprocesamiento.
- **Gestión de valores faltantes en árboles:** La herramienta forzaba imputación univariante universal sobre todos los modelos. El equipo introdujo la distinción técnica de los clasificadores modernos basados en árboles (Random Forest), generando un experimento comparativo específico.
- **Preservación de la señal en transformaciones:** La IA eliminó transitoriamente el redondeo de días de actividad; se exigió su restitución y se amplió el tratamiento a las variables de naturaleza entera (`anos_fumador`, `frecuencia_cardiaca_reposo` y `consumo_alcohol_semanal`).
- **Semántica de la variable edad:** La IA propuso un redondeo simétrico; el equipo corrigió la operación a truncamiento inferior (`np.floor`) para honrar la definición clínica de «años cumplidos».
- **Defensa de atípicos clínicos:** La herramienta catalogó valores extremos de sueño y cintura como candidatos a eliminación. El equipo rechazó la poda al verificar que son biológicamente admisibles.
- **Precedencia lógica en la presión arterial:** Se detectó que el orden de enmascaramiento propuesto por la IA destruía la detección de la presión invertida al introducir un valor `NaN` previo; se obligó a anteponer la regla cruzada.

Prompt

Cómo puedo imprimir los nulos del df por columna?

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
¡Claro! Para analizar los valores ausentes de forma clara y estructurada
en
Pandas, lo más recomendable es construir una tabla resumen que muestre
tanto el
```

```
recuento absoluto como el porcentaje relativo sobre el total de
registros.

Aquí tienes un fragmento de código eficiente y directo para lograrlo:

# 1. Total global de valores nulos en todo el DataFrame
print("Valores nulos: ", df.isnull().sum().sum())

# 2. Construcción de una tabla resumen con cantidad y porcentaje por
columna
df_resumen_nulos = pd.DataFrame({
    'Nulos': df.isnull().sum(),
    'Porcentaje (%)': (df.isnull().mean() * 100).round(2)
})

# 3. Filtrar solo las columnas que presentan valores nulos y ordenar
descendentemente
df_resumen_nulos = df_resumen_nulos[df_resumen_nulos['Nulos'] >
0].sort_values(by='Nulos', ascending=False)

# 4. Visualizar la tabla resultante
df_resumen_nulos

¿Por qué es una buena solución?

  - Eficiencia: El uso de df.isnull().mean() * 100 calcula directamente
  la
    proporción relativa sin necesidad de llamar a len(df) de forma
    manual.
  - Claridad en el EDA: Al filtrar con [df_resumen_nulos['Nulos'] > 0] y
  ordenar
    de mayor a menor, aíslas inmediatamente las variables problemáticas
    sin
    sobrecargar la pantalla con las columnas que están completas al 100%.
    Esto
    te permite priorizar qué atributos requerirán imputación (como
    SimpleImputer
    o KNNImputer) en la fase de preprocesamiento.
```

Análisis

La respuesta a la consulta responde con lo pedido, pero además incluye el porcentaje de nulos por columna y los ordena por número de nulos, lo que es bastante útil. Se acepta.

Prompt

```
df = df[df['edad'] >= 18]

Cómo muestro cuántos he eliminado?
```

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Para verificar y dejar constancia de cuántos registros se han descartado
durante
el filtrado de la cohorte, una forma muy sencilla y limpia es guardar el
número
de filas antes de la operación y compararlo con el recuento posterior.

Aquí tienes el código para implementarlo de forma directa:

# Guardar el número inicial de registros en el dataset
filas = df.shape[0]

# Filtrar para conservar únicamente a la población adulta (edad >= 18)
df = df[df['edad'] >= 18]

# Mostrar la diferencia de registros eliminados y el total restante
print(filas - df.shape[0], "filas eliminadas por ser menores de edad. Hay
", df.shape[0], "filas restantes.")

# Actualizar la variable con el nuevo conteo de filas
filas = df.shape[0]

¿Por qué es recomendable hacerlo así?

  - Trazabilidad en la memoria: En proyectos del ámbito biomédico es
  fundamental
    documentar cada paso de exclusión de pacientes (siguiendo diagramas
    de flujo
    tipo CONSORT o STARD). Al calcular la diferencia directamente con
    df.shape[0], te aseguras de que el informe refleje con exactitud
    cuántos
    casos pediátricos fueron eliminados (en este caso, 4 registros).
  - Consistencia del entorno: Al reasignar filas = df.shape[0] al final,
  dejas
    esa variable actualizada para poder reutilizarla en futuros filtros
    de
    limpieza (por ejemplo, si más adelante descartas filas con errores
    graves)
    sin tener que definir variables auxiliares adicionales.
```

Análisis

Responde a lo pedido. Se acepta.

Prompt

Cómo pongo a NaN los valores negativos o 0 de colesterol$_h dl$?

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Para sustituir los valores biológicamente imposibles (\le 0) por np.nan
sin
alterar el resto de columnas del paciente, la forma más directa, legible
y
recomendada en Pandas es utilizar .loc con una condición booleana.

Aquí tienes el código con la comprobación de cuántos registros se han
modificado:

import numpy as np

# 1. Contar cuántos valores inválidos hay antes del cambio (para
trazabilidad)
n_invalidos = (df['colesterol_hdl'] <= 0).sum()

# 2. Asignar np.nan a los valores menores o iguales a cero
df.loc[df['colesterol_hdl'] <= 0, 'colesterol_hdl'] = np.nan

# 3. Confirmar la operación
print(f"Se han reemplazado {n_invalidos} valores de 'colesterol_hdl' (<=
0) por NaN.")

Alternativa funcional elegante: .mask()

Si prefieres no usar indexación por .loc, Pandas cuenta con el método
.mask(),
que reemplaza por NaN (por defecto) aquellos valores que cumplan la
condición
indicada:

# Reemplaza cualquier valor <= 0 por NaN de forma directa
df['colesterol_hdl'] = df['colesterol_hdl'].mask(df['colesterol_hdl'] <=
0)

Justificación metodológica (para la memoria):

  - No descartar al paciente: Al convertir los valores absurdos a np.nan
  en
    lugar de borrar la fila completa (df.dropna()), se preserva el resto
    del
    historial clínico del paciente (edad, tensión, hábitos,
    comorbilidades).
  - Compatibilidad con imputación: Estos NaN recién creados serán
  rellenados de
    manera coherente en la siguiente fase de preprocesamiento mediante
    técnicas
    como KNNImputer o SimpleImputer(strategy='median'), evitando
    introducir
    sesgo en los clasificadores.
```

Análisis

De las dos opciones se elige la primera al ser más intuitiva, pero cambiando el print().

Prompt

Haz que sea NaN en ambas columnas si IMC es mayor que 35 y el perímetro de la cintura es menor que 70

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Para invalidar ambas variables de forma simultánea cuando se detecta esa
discordancia antropométrica (obesidad severa con una cintura incompatible
de
extrema delgadez), puedes usar .loc pasando una lista con los nombres de
las dos
columnas.

Aquí tienes el código limpio y con control del número de casos afectados:

import numpy as np

# 1. Definir la condición de inconsistencia antropométrica severa
condicion_antropo = (df['indice_masa_corporal'] > 35) &
(df['perimetro_cintura'] < 70)

# 2. Contar cuántos pacientes presentan esta anomalía antes de modificar
n_inconsistencias = condicion_antropo.sum()

# 3. Asignar NaN a ambas columnas simultáneamente en las filas afectadas
df.loc[condicion_antropo, ['indice_masa_corporal', 'perimetro_cintura']]
= np.nan

# 4. Mensaje informativo de confirmación
print(f"Se han puesto a NaN ambas columnas en {n_inconsistencias}
registros por inconsistencia IMC-Cintura.")

Justificación para la memoria técnica:

  - Incoherencia fisiológica cruzada: Un IMC > 35 kg/m² indica obesidad
  grado II
    o III, lo que anatómicamente resulta incompatible con una
    circunferencia de
    cintura inferior a 70 cm en un adulto.
  - Neutralización del dato corrupto: Dado que no es posible determinar
  cuál de
    los dos valores fue el erróneo en la toma de datos clínicos (si el
    peso/altura o la cinta métrica), se enmascaran ambos a np.nan para
    que un
```

algoritmo de imputación multivariante (como KNNImputer) los reconstruya a
partir del resto de variables del paciente (edad, tensión, analítica sanguínea, etc.).

Análisis

Se acepta, pero variando el código ligeramente para que sea más intuitivo y ocupe menos líneas.

Prompt

Qué debería de aplicarle a antecedentes_familiares_tipo y tipo_alimentacion? un
one-hot-encoding? y a nivel_actividad un ordinal? Recuerda que nivel_actividad
era Sedentario, Ligeramente activo, Moderadamente activo y Muy activo

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Sí, tu intuición es totalmente correcta. Esa es exactamente la decisión metodológica adecuada tanto desde el punto de vista matemático como clínico.

Aquí tienes la justificación de cada una y cómo implementarlo para evitar errores comunes:

1. tipo_alimentacion \rightarrow One-Hot Encoding

- Por qué: Es una variable nominal pura. No existe un orden matemático intrínseco (la dieta Mediterránea no es "mayor" o "menor" que la Vegetariana
  ni que la Baja en carbohidratos).
- Si le asignaras números (1, 2, 3, 4), modelos como la Regresión Logística
  interpretarían erróneamente que una dieta es el triple que otra.

2. antecedentes_familiares_tipo \rightarrow One-Hot Encoding

- Por qué: Sus valores son 'Ninguno', 'Materno', 'Paterno' y 'Ambos'.
- Aunque 'Ambos' representa una mayor carga genética que 'Ninguno', entre
  'Materno' y 'Paterno' no existe un orden biológico jerárquico (la línea
  paterna no es "superior" a la materna). Tratarla como ordinal obligaría a
  darle un número mayor a uno que al otro sin justificación médica.
- Con One-Hot Encoding cada línea de antecedente se evalúa como una hipótesis

```
    independiente sin forzar una jerarquía artificial.

3. nivel_actividad \rightarrow Ordinal Encoding

  - Por qué: Es una variable ordinal estricta. Existe una relación
  monótona
    clara de menor a mayor gasto energético:
    \text{Sedentario (0)} < \text{Ligeramente activo (1)} <
    \text{Moderadamente activo (2)} < \text{Muy activo (3)}
  - Ventajas:
      - Permite al modelo aprender que pasar de Sedentario a Muy activo
      tiene un
        efecto acumulativo protector frente al riesgo cardiovascular.
      - Ahorra dimensionalidad: genera una sola columna numérica en lugar
      de
        crear 4 variables dummy.

 Cuidado con un error muy común en Scikit-Learn

Si usas OrdinalEncoder() por defecto, ordenará las categorías
alfabéticamente,
asignando:

  - Ligeramente activo \rightarrow 0
  - Moderadamente activo \rightarrow 1
  - Muy activo \rightarrow 2
  - Sedentario \rightarrow 3 (¡lo cual rompería el orden clínico!)

Para solucionarlo, debes pasarle la lista con el orden explícito en el
parámetro
categories.

Implementación óptima con ColumnTransformer:

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

# 1. Definir las listas de columnas
cols_nominales = ['tipo_alimentacion', 'antecedentes_familiares_tipo']
cols_ordinales = ['nivel_actividad']

# 2. Definir el orden clínico exacto para la variable ordinal
orden_actividad = [['Sedentario', 'Ligeramente activo', 'Moderadamente
activo', 'Muy activo']]

# 3. Construir el transformador categórico
encoder_categorico = ColumnTransformer(transformers=[
    ('nom', OneHotEncoder(drop='first', handle_unknown='ignore'),
    cols_nominales),
    ('ord', OrdinalEncoder(categories=orden_actividad), cols_ordinales)
```

```
], remainder='passthrough')

(Nota: Usar drop='first' en el OneHotEncoder es fundamental para evitar
la
trampa de la multicolinealidad en modelos lineales como la Regresión
Logística).
```

Análisis

Se confirma que el tratamiento de nominales es el correcto, pero no se buscaba hacer la transformación aún, sino preparar las funciones para que sean reutilizables.

Prompt

no quiero realizar la transformación aún

Respuesta IA Gemini Flash 3.8 en Google AI Studio

```
Es una excelente práctica dejar los transformadores solo configurados e
instanciados, sin aplicar el .fit_transform() todavía, para reservarlos y
acoplarlos más adelante al pipeline de entrenamiento evitando cualquier
tipo de
data leakage.

Siguiendo esa estructura modular (declarando primero las columnas y
jerarquías
fuera para que el código quede limpio y desacoplado), la forma
recomendada de
definirlo es la siguiente:

from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

#
-------------------------------------------------------------------------------
# 1. DEFINICIÓN DE GRUPOS DE VARIABLES Y JERARQUÍAS CLÍNICAS
#
-------------------------------------------------------------------------------
# Variables nominales puras (sin orden intrínseco)
cols_nominales = ['tipo_alimentacion', 'antecedentes_familiares_tipo']

# Variable ordinal (con progresión física monótona)
cols_ordinales = ['nivel_actividad']

# Escala de actividad física ordenada estrictamente de menor a mayor
esfuerzo
orden_nivel_actividad = [
    ['Sedentario', 'Ligeramente activo', 'Moderadamente activo', 'Muy
    activo']
]
```

```
#
------------------------------------------------------------------------------
# 2. INSTANCIACIÓN DE ENCODERS (SIN APLICAR TRANSFORMACIÓN AÚN)
#
------------------------------------------------------------------------------
# Encoder para nominales: drop='first' para prevenir colinealidad en
modelos lineales
encoder_nominal = OneHotEncoder(
    drop='first',
    handle_unknown='ignore'
)

# Encoder para ordinales: mapeo clínico controlado a escala [0, 1, 2, 3]
encoder_ordinal = OrdinalEncoder(
    categories=orden_nivel_actividad
)

¿Por qué dejarlo así preparado?

  - Separación de responsabilidades: Mantienes las listas de atributos
    (cols_nominales, cols_ordinales) y las jerarquías en variables
    independientes, facilitando cambios futuros sin tocar la
    configuración de
    los estimadores.
  - Listo para el Pipeline: Al estar instanciados como objetos de
  Scikit-Learn
    (encoder_nominal y encoder_ordinal), quedan preparados para ser
    introducidos
    directamente en el ColumnTransformer cuando se construya el esquema
    de
    validación cruzada.
```

Análisis

Se acepta, pero simplificándolo en 2 líneas.

Prompt

```
def metricas(y_real, y_pred, conjunto):
    return {
        f"{conjunto}_accuracy": accuracy_score(y_real, y_pred),
        f"{conjunto}_f1_macro": f1_score(y_real, y_pred,
average="macro"),
        f"{conjunto}_recall_alto": recall_score(y_real, y_pred,
labels=[2], average="macro"),
    }

def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
```

```
    X, y = datos.drop(columns=TARGET), datos[TARGET]
    filas = []
    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
{nombre_preprocesado}...")
    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
y), start=1):
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]
        pipeline.fit(X_train_fold, y_train_fold)
        for nombre_modelo, (modelo, escalar, imputar) in MODELOS.items():
            pipeline = construir_pipeline(clone(modelo), escalar,
imputar).fit(X_train, y_train)
            fila = {"preprocesado": nombre, "modelo": nombre_modelo,
"iteracion": iteracion,
                    "n_train": len(y_train), "n_test": len(y_test),
                    **metricas(y_train, pipeline.predict(X_train),
"train"),
                    **metricas(y_test, pipeline.predict(X_test), "test")}
            filas.append(fila)
            if verbose:
                print(f"{nombre_modelo.upper()}: train F1
{fila['train_f1_macro']:.4f} | "
                      f"test F1 {fila['test_f1_macro']:.4f} | test recall
alto {fila['test_recall_alto']:.4f}")
    return pd.DataFrame(filas)


Como puedes ver, ahora se mete un pipeline individual en vez de ejecutar
todos los modelos. Quiero que me me lo adaptes para ello
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Aquí tienes la función adaptada para recibir un único **pipeline individual** (y clonarlo en cada iteración de la validación cruzada para evitar fugas o sobreescrituras entre folds), corrigiendo también algunas variables globales del bucle original (`X_train`, `nombre`, etc.) para que apunten correctamente a cada fold (`_fold`):

```python
from sklearn.base import clone
import pandas as pd

def metricas(y_real, y_pred, conjunto):
    return {
        f"{conjunto}_accuracy": accuracy_score(y_real, y_pred),
        f"{conjunto}_f1_macro": f1_score(y_real, y_pred,
        average="macro"),
        f"{conjunto}_recall_alto": recall_score(y_real, y_pred,
        labels=[2], average="macro"),
```

```
    }

def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    filas = []

    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
        {nombre_preprocesado}...")

    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
    y), start=1):
        # Separar los datos de train y test para este fold
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]

        # Clonar el pipeline para que empiece limpio en cada fold y
        ajustarlo
        pipeline_fold = clone(pipeline)
        pipeline_fold.fit(X_train_fold, y_train_fold)

        # Generar predicciones
        y_pred_train = pipeline_fold.predict(X_train_fold)
        y_pred_test = pipeline_fold.predict(X_test_fold)

        # Registrar métricas de la iteración
        fila = {
            "preprocesado": nombre_preprocesado,
            "modelo": nombre_modelo,
            "iteracion": iteracion,
            "n_train": len(y_train_fold),
            "n_test": len(y_test_fold),
            **metricas(y_train_fold, y_pred_train, "train"),
            **metricas(y_test_fold, y_pred_test, "test")
        }
        filas.append(fila)

        if verbose:
            print(f"  - Iteración {iteracion} | train F1:
            {fila['train_f1_macro']:.4f} | "
                  f"test F1: {fila['test_f1_macro']:.4f} | test recall
                  alto: {fila['test_recall_alto']:.4f}")

    return pd.DataFrame(filas)

```

Cambios principales realizados:

```
1. **Eliminación del bucle interno `MODELOS.items()**`: Ahora la función
se centra en evaluar el pipeline específico que se le pasa como
parámetro.
2. **Uso de `clone(pipeline)**`: Es una buena práctica en scikit-learn al
hacer validación cruzada manual para asegurar que cada fold entrena desde
cero con un objeto fresco.
3. **Corrección de nombres y variables**: Se cambió `nombre` por
`nombre_preprocesado` y se aseguraron los tamaños y conjuntos con sufijo
`_fold` (`X_train_fold`, `y_train_fold`, etc.) para evitar usar variables
globales erróneas.
```

Análisis

Se busca elminar la iteración de modelos y mejorar la función. La respuesta de la IA es insuficiente, falta como mínimo incluir resultados visuales de las métricas. Sin embargo, ayuda a darme cuenta de que había eliminado el *clone* de mi compañero.

Prompt

pero imprime los valores de las metricas

Respuesta IA Gemini Flash Lite 3.5 en Gemini

````
Aquí tienes la función adaptada para recibir un único **pipeline
individual** (y clonarlo en cada iteración de la validación cruzada para
evitar fugas o sobreescrituras entre folds), corrigiendo también algunas
variables globales del bucle original (`X_train`, `nombre`, etc.) para
que apunten correctamente a cada fold (`_fold`):

```python
from sklearn.base import clone
import pandas as pd

def metricas(y_real, y_pred, conjunto):
    return {
        f"{conjunto}_accuracy": accuracy_score(y_real, y_pred),
        f"{conjunto}_f1_macro": f1_score(y_real, y_pred,
        average="macro"),
        f"{conjunto}_recall_alto": recall_score(y_real, y_pred,
        labels=[2], average="macro"),
    }

def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    filas = []

    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
        {nombre_preprocesado}...")
````
````

```
    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
    y), start=1):
        # Separar los datos de train y test para este fold
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]

        # Clonar el pipeline para que empiece limpio en cada fold y
        ajustarlo
        pipeline_fold = clone(pipeline)
        pipeline_fold.fit(X_train_fold, y_train_fold)

        # Generar predicciones
        y_pred_train = pipeline_fold.predict(X_train_fold)
        y_pred_test = pipeline_fold.predict(X_test_fold)

        # Registrar métricas de la iteración
        fila = {
            "preprocesado": nombre_preprocesado,
            "modelo": nombre_modelo,
            "iteracion": iteracion,
            "n_train": len(y_train_fold),
            "n_test": len(y_test_fold),
            **metricas(y_train_fold, y_pred_train, "train"),
            **metricas(y_test_fold, y_pred_test, "test")
        }
        filas.append(fila)

        if verbose:
            print(f"  - Iteración {iteracion} | train F1:
            {fila['train_f1_macro']:.4f} | "
                  f"test F1: {fila['test_f1_macro']:.4f} | test recall
                  alto: {fila['test_recall_alto']:.4f}")

    return pd.DataFrame(filas)

```

Cambios principales realizados:

1. **Eliminación del bucle interno `MODELOS.items()**`: Ahora la función se centra en evaluar el pipeline específico que se le pasa como parámetro.
2. **Uso de `clone(pipeline)**`: Es una buena práctica en scikit-learn al hacer validación cruzada manual para asegurar que cada fold entrena desde cero con un objeto fresco.
3. **Corrección de nombres y variables**: Se cambió `nombre` por `nombre_preprocesado` y se aseguraron los tamaños y conjuntos con sufijo `_fold` (`X_train_fold`, `y_train_fold`, etc.) para evitar usar variables globales erróneas.

Análisis

El código sirve como base, pero hay que hacerle retoques como eliminar valores innecesarios para el dataframe que devuelve y aún queda un resumen global con las desviaciones.

Prompt

```
def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    X, y = datos.drop(columns=TARGET), datos[TARGET]
    filas = []
    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
{nombre_preprocesado}...")
    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
y), start=1):
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]
        pipeline_fold = clone(pipeline)
        pipeline.fit(X_train_fold, y_train_fold)
        y_pred_train = pipeline.predict(X_train_fold)
        y_pred_test = pipeline.predict(X_test_fold)
        filas.append({
            "preprocesado": nombre_preprocesado,
            "modelo": nombre_modelo,
            "iteracion": iteracion,
            **metricas(y_train_fold, y_pred_train, "train"),
            **metricas(y_test_fold, y_pred_test, "test")
        })
        if verbose:
            print(f"\n--- Iteración {iteracion} ---")
            print(f"  [TRAIN] F1 Macro: {fila['train_f1_macro']:.4f} |
Recall Alto: {fila['train_recall_alto']:.4f} | Accuracy:
{fila['train_accuracy']:.4f}")
            print(f"  [TEST]  F1 Macro: {fila['test_f1_macro']:.4f}  |
Recall Alto: {fila['test_recall_alto']:.4f} | Accuracy:
{fila['test_accuracy']:.4f}")
    return pd.DataFrame(filas)


Incluye un print con un resumen global con el +- de la desviación
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Aquí tienes la función con el resumen global al finalizar el bucle. Calcula la **media** y la **desviación típica (std)** de todas las métricas obtenidas en los folds y las imprime con formato `media ± std`:

```python
from sklearn.base import clone
```

```
import pandas as pd

def metricas(y_real, y_pred, conjunto):
    return {
        f"{conjunto}_accuracy": accuracy_score(y_real, y_pred),
        f"{conjunto}_f1_macro": f1_score(y_real, y_pred,
        average="macro"),
        f"{conjunto}_recall_alto": recall_score(y_real, y_pred,
        labels=[2], average="macro"),
    }

def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    # Nota: si X e y ya se pasan como argumentos, reasignarlos desde una
    variable global 'datos'
    # podría sobreescribirlos, pero mantenemos tu lógica si así lo
    requieres:
    X, y = datos.drop(columns=TARGET), datos[TARGET]
    filas = []

    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
        {nombre_preprocesado}...")

    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
    y), start=1):
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]

        # Ojo: estabas clonando en 'pipeline_fold' pero luego ajustando y
        prediciendo con 'pipeline'.
        # Lo correcto es usar 'pipeline_fold' en todo este bloque:
        pipeline_fold = clone(pipeline)
        pipeline_fold.fit(X_train_fold, y_train_fold)

        y_pred_train = pipeline_fold.predict(X_train_fold)
        y_pred_test = pipeline_fold.predict(X_test_fold)

        fila = {
            "preprocesado": nombre_preprocesado,
            "modelo": nombre_modelo,
            "iteracion": iteracion,
            **metricas(y_train_fold, y_pred_train, "train"),
            **metricas(y_test_fold, y_pred_test, "test")
        }
        filas.append(fila)

        if verbose:
            print(f"\n--- Iteración {iteracion} ---")
```

```
            print(f"  [TRAIN] F1 Macro: {fila['train_f1_macro']:.4f} |
            Recall Alto: {fila['train_recall_alto']:.4f} | Accuracy:
            {fila['train_accuracy']:.4f}")
            print(f"  [TEST]  F1 Macro: {fila['test_f1_macro']:.4f}  |
            Recall Alto: {fila['test_recall_alto']:.4f}  | Accuracy:
            {fila['test_accuracy']:.4f}")

    df_resultados = pd.DataFrame(filas)

    # Resumen global con media y desviación típica si verbose es True
    if verbose:
        print("\n" + "="*50)
        print(f"RESUMEN GLOBAL ({nombre_modelo} +
        {nombre_preprocesado})")
        print("="*50)

        columnas_metricas = [c for c in df_resultados.columns if c not in
        ["preprocesado", "modelo", "iteracion"]]

        for col in columnas_metricas:
            media = df_resultados[col].mean()
            std = df_resultados[col].std()
            print(f"  {col.upper():<22}: {media:.4f} ± {std:.4f}")
        print("="*50)

    return df_resultados

```

Corrección menor añadida:

En tu fragmento original hacías `pipeline_fold = clone(pipeline)` pero luego usabas `pipeline.fit(...)` y `pipeline.predict(...)`. Lo he corregido para usar **`pipeline_fold`** en todo el bucle, garantizando que cada fold se entrene de forma independiente y limpia.

Análisis

No devuelve los valores correctos, debería devolver el resumen global con las desviaciones típicas, no por iteraciones. Sin embargo, ayuda a detectar un problema con el pipeline, ya que no se usaba el clonado.

Prompt

No deberías de devolver en df_resultados los resultados globales con sus desviaciones en vez de separado por iteraciones?

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Tienes toda la razón. Si el propósito principal de evaluar con validación cruzada es obtener una estimación robusta y única del rendimiento del modelo, lo ideal es que `df_resultados` devuelva una **fila resumen por modelo/preprocesamiento con las medias y desviaciones típicas**, en lugar de tener las filas desglosadas por cada iteración.

Aquí tienes la función adaptada para que devuelva un DataFrame con una sola fila consolidada (o tantas como combinaciones ejecutes) mostrando las métricas en formato **Media ± Desviación Típica**, manteniendo el desglose opcional solo para cuando quieras imprimirlo:

```python
from sklearn.base import clone
import pandas as pd
import numpy as np

def metricas(y_real, y_pred, conjunto):
    return {
        f"{conjunto}_accuracy": accuracy_score(y_real, y_pred),
        f"{conjunto}_f1_macro": f1_score(y_real, y_pred,
        average="macro"),
        f"{conjunto}_recall_alto": recall_score(y_real, y_pred,
        labels=[2], average="macro"),
    }

def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    X, y = datos.drop(columns=TARGET), datos[TARGET]
    filas = []

    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
        {nombre_preprocesado}...")

    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
    y), start=1):
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]

        pipeline_fold = clone(pipeline)
        pipeline_fold.fit(X_train_fold, y_train_fold)

        y_pred_train = pipeline_fold.predict(X_train_fold)
        y_pred_test = pipeline_fold.predict(X_test_fold)

        fila = {
            "preprocesado": nombre_preprocesado,
            "modelo": nombre_modelo,
            "iteracion": iteracion,
```

```
            **metricas(y_train_fold, y_pred_train, "train"),
            **metricas(y_test_fold, y_pred_test, "test")
        }
        filas.append(fila)

        if verbose:
            print(f"\n--- Iteración {iteracion} ---")
            print(f"  [TRAIN] F1 Macro: {fila['train_f1_macro']:.4f} |
            Recall Alto: {fila['train_recall_alto']:.4f} | Accuracy:
            {fila['train_accuracy']:.4f}")
            print(f"  [TEST]  F1 Macro: {fila['test_f1_macro']:.4f}  |
            Recall Alto: {fila['test_recall_alto']:.4f}  | Accuracy:
            {fila['test_accuracy']:.4f}")

    df_iteraciones = pd.DataFrame(filas)

    # Construir el DataFrame resumen con la media y desviación típica
    columnas_metricas = [c for c in df_iteraciones.columns if c not in
    ["preprocesado", "modelo", "iteracion"]]

    resumen_dict = {
        "preprocesado": nombre_preprocesado,
        "modelo": nombre_modelo
    }

    for col in columnas_metricas:
        media = df_iteraciones[col].mean()
        std = df_iteraciones[col].std()
        # Opcional: guardarlo como string formateado "media ± std" para
        visualización directa
        resumen_dict[col] = f"{media:.4f} ± {std:.4f}"

        # Si prefieres conservar los valores numéricos separados para
        futuras operaciones matemáticas,
        # puedes descomentar las siguientes líneas en lugar del string:
        # resumen_dict[f"{col}_mean"] = media
        # resumen_dict[f"{col}_std"] = std

    df_resumen = pd.DataFrame([resumen_dict])

    if verbose:
        print("\n" + "="*50)
        print(f"RESUMEN GLOBAL ({nombre_modelo} +
        {nombre_preprocesado})")
        print("="*50)
        for col in columnas_metricas:
            print(f"  {col.upper():<22}: {resumen_dict[col]}")
        print("="*50)

    return df_resumen
```

```
```

Análisis

Se acepta la respuesta para adaptarla al código, cambiando algunos aspectos como el orden de impresión de las métricas. En cuanto a su segunda opción, se prefiere la primera porque es más cómoda para mostrar visualmente. También se determina con esta respuesta que no hace falta guardar ya en filas la iteración.

Prompt

no me gusta que se imprima primero train y despues test, debería imprimirse f1 train y despues test, etc.

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Aquí tienes la función ajustada para que el orden de impresión agrupe primero la métrica (por ejemplo, **F1 Macro** con su valor de *train* y luego *test*), en lugar de agrupar por conjunto:

```python
from sklearn.base import clone
import pandas as pd
import numpy as np

def metricas(y_real, y_pred, conjunto):
    return {
        f"{conjunto}_accuracy": accuracy_score(y_real, y_pred),
        f"{conjunto}_f1_macro": f1_score(y_real, y_pred,
        average="macro"),
        f"{conjunto}_recall_alto": recall_score(y_real, y_pred,
        labels=[2], average="macro"),
    }

def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    X, y = datos.drop(columns=TARGET), datos[TARGET]

    metricas_folds = {
        "train_accuracy": [], "train_f1_macro": [], "train_recall_alto":
        [],
        "test_accuracy": [], "test_f1_macro": [], "test_recall_alto": []
    }

    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
        {nombre_preprocesado}...")
```

```
    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
    y), start=1):
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]

        pipeline_fold = clone(pipeline)
        pipeline_fold.fit(X_train_fold, y_train_fold)

        y_pred_train = pipeline_fold.predict(X_train_fold)
        y_pred_test = pipeline_fold.predict(X_test_fold)

        m_train = metricas(y_train_fold, y_pred_train, "train")
        m_test = metricas(y_test_fold, y_pred_test, "test")

        for k, v in {**m_train, **m_test}.items():
            metricas_folds[k].append(v)

        if verbose:
            print(f"\n--- Iteración {iteracion} ---")
            print(f"  F1 Macro   -> Train: {m_train['train_f1_macro']:.4f}
            | Test: {m_test['test_f1_macro']:.4f}")
            print(f"  Recall Alto -> Train:
            {m_train['train_recall_alto']:.4f} | Test:
            {m_test['test_recall_alto']:.4f}")
            print(f"  Accuracy   -> Train: {m_train['train_accuracy']:.4f}
            | Test: {m_test['test_accuracy']:.4f}")

    resumen_dict = {
        "preprocesado": nombre_preprocesado,
        "modelo": nombre_modelo
    }

    for metrica, valores in metricas_folds.items():
        media = np.mean(valores)
        std = np.std(valores)
        resumen_dict[metrica] = f"{media:.4f} ± {std:.4f}"

    df_resumen = pd.DataFrame([resumen_dict])

    if verbose:
        print("\n" + "="*50)
        print(f"RESUMEN GLOBAL ({nombre_modelo} +
        {nombre_preprocesado})")
        print("="*50)
        print(f"  F1 MACRO   -> Train: {resumen_dict['train_f1_macro']} |
        Test: {resumen_dict['test_f1_macro']}")
        print(f"  RECALL ALTO-> Train: {resumen_dict['train_recall_alto']}
        | Test: {resumen_dict['test_recall_alto']}")
        print(f"  ACCURACY   -> Train: {resumen_dict['train_accuracy']} |
        Test: {resumen_dict['test_accuracy']}")
```

```
        print("="*50)

    return df_resumen

```

Análisis

Los cambios hacen el código mucho más simple e intuitivo. Se acepta para hacer los cambios con el que ya está en el notebook de preprocesamiento. Sin embargo, el orden al imprimir en pantalla sigue siendo diferente al que se pedía en la consulta.

Prompt

creo que en df_resumen no los metes para que se impriman en dicho orden

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Tienes toda la razón. Como en `resumen_dict` las claves se añadían de forma plana, al imprimir iterando directamente sobre el diccionario o sus claves (`for metrica in metricas_folds.keys()`), el orden dependía de cómo se hubieran declarado las claves originalmente (que empezaban por todo `train_` y luego todo `test_`).

Para solucionarlo y asegurarnos de que tanto en el resumen global de consola como en el orden de las columnas del DataFrame se respete exactamente el criterio de **agrupar por métrica** (primero F1 Macro, luego Recall Alto, etc., con su Train y Test juntos), podemos estructurarlo explícitamente así:

```python
from sklearn.base import clone
import pandas as pd
import numpy as np

def metricas(y_real, y_pred, conjunto):
    return {
        f"{conjunto}_accuracy": accuracy_score(y_real, y_pred),
        f"{conjunto}_f1_macro": f1_score(y_real, y_pred,
        average="macro"),
        f"{conjunto}_recall_alto": recall_score(y_real, y_pred,
        labels=[2], average="macro"),
    }

def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    X, y = datos.drop(columns=TARGET), datos[TARGET]

    metricas_folds = {
```

```
        "train_accuracy": [], "train_f1_macro": [], "train_recall_alto":
        [],
        "test_accuracy": [], "test_f1_macro": [], "test_recall_alto": []
    }

    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
        {nombre_preprocesado}...")

    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
    y), start=1):
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]

        pipeline_fold = clone(pipeline)
        pipeline_fold.fit(X_train_fold, y_train_fold)

        y_pred_train = pipeline_fold.predict(X_train_fold)
        y_pred_test = pipeline_fold.predict(X_test_fold)

        m_train = metricas(y_train_fold, y_pred_train, "train")
        m_test = metricas(y_test_fold, y_pred_test, "test")

        for k, v in {**m_train, **m_test}.items():
            metricas_folds[k].append(v)

        if verbose:
            print(f"\n--- Iteración {iteracion} ---")
            print(f"  F1 Macro   -> Train: {m_train['train_f1_macro']:.4f}
            | Test: {m_test['test_f1_macro']:.4f}")
            print(f"  Recall Alto -> Train:
            {m_train['train_recall_alto']:.4f} | Test:
            {m_test['test_recall_alto']:.4f}")
            print(f"  Accuracy   -> Train: {m_train['train_accuracy']:.4f}
            | Test: {m_test['test_accuracy']:.4f}")

    # Calcular medias y desviaciones
    calculadas = {}
    for metrica, valores in metricas_folds.items():
        media = np.mean(valores)
        std = np.std(valores, ddof=1)
        calculadas[metrica] = f"{media:.4f} ± {std:.4f}"

    # Construir el diccionario respetando el orden visual deseado (y
    ordenando las columnas del DF)
    resumen_dict = {
        "preprocesado": nombre_preprocesado,
        "modelo": nombre_modelo,
        "train_f1_macro": calculadas["train_f1_macro"],
        "test_f1_macro": calculadas["test_f1_macro"],
```

```
        "train_recall_alto": calculadas["train_recall_alto"],
        "test_recall_alto": calculadas["test_recall_alto"],
        "train_accuracy": calculadas["train_accuracy"],
        "test_accuracy": calculadas["test_accuracy"],
    }

    df_resumen = pd.DataFrame([resumen_dict])

    if verbose:
        print("\n" + "="*55)
        print(f"RESUMEN GLOBAL ({nombre_modelo} +
        {nombre_preprocesado})")
        print("="*55)
        print(f"  F1 MACRO   -> Train: {resumen_dict['train_f1_macro']} |
        Test: {resumen_dict['test_f1_macro']}")
        print(f"  RECALL ALTO-> Train: {resumen_dict['train_recall_alto']}
        | Test: {resumen_dict['test_recall_alto']}")
        print(f"  ACCURACY   -> Train: {resumen_dict['train_accuracy']} |
        Test: {resumen_dict['test_accuracy']}")
        print("="*55)

    return df_resumen

```
```

Análisis

Eliminada la última problemática, se acepta el código.

Prompt

```
se pueden concatenar pipelines o algo así? tipo:

def construir_pipeline(modelo, escalar, imputar=True):
    codificacion = ColumnTransformer(
        [
            ("nominales", OneHotEncoder(drop="first",
handle_unknown="ignore"), NOMINALES),
            ("ordinal", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]),
[ORDINAL]),
        ],
        remainder="passthrough",
    )
    pasos = [("codificacion", codificacion)]
    if imputar:
        pasos.append(("imputacion", SimpleImputer(strategy="median")))
    if escalar:
        pasos.append(("escalado", StandardScaler()))
    pasos.append(("modelo", modelo))
    return Pipeline(pasos)
```
```

En esta funcion lo de one-hot y ordinal en esas variables 100\% lo voy a usar siempre, entonces no me gustaría estar añadiéndolo a todos mis pipelines (voy a prescindir de esta función y posiblemente crearé otros pipelines con opciones diferentes a las que hay aquí).

Respuesta IA Gemini Flash Lite 3.5 en Gemini

¡Sí, totalmente! De hecho, **es la mejor forma de trabajar en scikit-learn** para no repetir código y mantener la limpieza mental en tus experimentos.

Si la codificación (`OneHotEncoder` y `OrdinalEncoder`) y el tratamiento de booleanos o variables categóricas van a estar **siempre fijos**, puedes definir un **preprocesador base global** (o un minipipeline de codificación) una sola vez al principio del cuaderno, y luego **encadenarlo** o meterlo dentro de cualquier otro pipeline o función de experimentos que crees después.

Tienes dos formas estupendas de hacerlo:

Opción 1: Definir un `ColumnTransformer` fijo y meterlo dentro de tus pipelines personalizados

En lugar de reescribir la codificación cada vez, la guardas en una variable global (por ejemplo, `preprocesador_base`) y cuando quieras construir un experimento rápido, simplemente la encadenas al principio de los pasos:

```python
# 1. Esto lo defines UNA SOLA VEZ al principio del notebook y te olvidas
preprocesador_base = ColumnTransformer(
    transformers=[
        ("nominales", OneHotEncoder(drop="first",
        handle_unknown="ignore"), NOMINALES),
        ("ordinal", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]),
        [ORDINAL]),
        # Si tienes booleanos u otras cosas fijas, los metes aquí también
    ],
    remainder="passthrough", # Deja pasar el resto de numéricas intactas
)

# 2. Cuando quieras hacer un experimento loco, simplemente concatenas el
bloque base:
pipeline_experimento_1 = Pipeline([
    ("codificacion_fija", preprocesador_base),   # <-- Lo que usas
    siempre
    ("imputacion", SimpleImputer(strategy="median")),
    ("escalado", RobustScaler()),               # <-- Cambias
    StandardScaler por RobustScaler al vuelo
```

```
    ("modelo", KNeighborsClassifier())          # <-- Cambias el modelo
])

pipeline_experimento_2 = Pipeline([
    ("codificacion_fija", preprocesador_base),   # <-- Lo reutilizas sin
    escribirlo más
    ("modelo", RandomForestClassifier())        # <-- O uno sin escalar
    ni imputar
])

```

Opción 2: Anidar Pipelines (Un pipeline dentro de otro)

Scikit-learn permite meter un `Pipeline` completo como un paso dentro de otro `Pipeline` (como si fuera una matrioshka). Esto es comodísimo si quieres agrupar toda la parte de transformación y luego pegarla a un modelo:

```python
# Tu bloque fijo de transformación de variables
pipeline_transformacion = Pipeline([
    ("codificacion", ColumnTransformer([
        ("nominales", OneHotEncoder(drop="first",
        handle_unknown="ignore"), NOMINALES),
        ("ordinal", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]),
        [ORDINAL])
    ], remainder="passthrough")),
    ("imputacion", SimpleImputer(strategy="median"))
])

# Ahora puedes "concatenarlo" con cualquier estimador o transformador
extra de forma súper limpia:
pipeline_final = Pipeline([
    ("transformaciones_estandar", pipeline_transformacion), # El bloque
    fijo
    ("escalador_extra", RobustScaler()),                    # Lo nuevo
    que quieras probar hoy
    ("modelo", SVC())                                       # Tu modelo
])

```

¿Por qué mola esto para experimentar?

```
Te quitas la función rigurosa de en medio. Tu `preprocesador_base` o
`pipeline_transformacion` actúa como unos **cimientos inamovibles**, y tú
en cada celda de experimento solo tienes que escribir los 3 o 4 pasos
específicos que cambian (el escalador, si metes PCA, o el modelo final),
concatenándolos de forma natural con un simple `[("base",
preprocesador_base), ("lo_nuevo", ...)]`.
```

Análisis

La consulta resuelve mi duda sobre cómo hacer los pipelines más libres y sobre cómo encadenarlos entre sí.

Prompt

```
# Común a todos los pipelines de preprocesado
pipeline_codificacion = Pipeline([
    ("codificacion", ColumnTransformer([
        ("nominales", OneHotEncoder(drop="first",
handle_unknown="ignore"), NOMINALES),
        ("ordinal", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]),
[ORDINAL])
    ], remainder="passthrough"))
])

# Preprocesado que imputa valores nulos con la mediana y escala los datos
numéricos con escalado estándar
pipeline_imputacionMediana_escaladoEstandar = Pipeline([
    ("codificacion", pipeline_codificacion),
    ("imputacion", SimpleImputer(strategy="median")),
    ("escalado", StandardScaler())
])

Quiero hacer una prueba con logistic regression (con evaluación y todo).
Cómo lo haría? Voy a usar DATOS["P3_pcr"]
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
¡Perfecto! Ya tienes la base modular montada de forma impecable. Como tu
`pipeline_codificacion` y tus bloques intermedios son un `Pipeline` de
scikit-learn, anidarlos es tan fácil como meterlos en la lista de pasos
del pipeline final.

Para hacer la prueba completa con **Logistic Regression** usando tu
función `evaluar` y tus particiones (`splitter`), solo tienes que
ejecutar esto en una celda de tu cuaderno:

```python
# 1. Montar el pipeline completo para Logistic Regression
# (Usamos tu pipeline base y le añadimos imputación, escalado y el
modelo)
```
```

```
pipeline_lr_estandar = Pipeline([
    ("preprocesado_base", pipeline_imputacionMediana_escaladoEstandar),
    ("modelo", LogisticRegression(max_iter=1000, random_state=SEED))
])

# 2. Elegir el dataset que quieres probar (por ejemplo, P3_pcr)
# Asegúrate de usar el DataFrame procesado correspondiente de tu
diccionario DATOS
df_experimento = DATOS["P3_pcr"]

# 3. Lanzar la evaluación con tu función genérica
resultados_lr = evaluar(
    pipeline=pipeline_lr_estandar,
    df_dataset=df_experimento,
    nombre_modelo="Logistic Regression",
    nombre_preprocesado="P3_pcr",
    verbose=True  # Ponlo a True para ver el detalle por iteración y el
    resumen global
)

# 4. Ver el DataFrame de resultados
resultados_lr

```

¿Por qué te soluciona la vida esta estructura?

Si mañana quieres probar **Logistic Regression con `RobustScaler**` en lugar de `StandardScaler`, no tienes que volver a escribir la codificación ni tocar nada de arriba; solo creas un pipeline nuevo al vuelo combinando las piezas:

```python
from sklearn.preprocessing import RobustScaler

pipeline_lr_robust = Pipeline([
    ("codificacion", pipeline_codificacion),
    ("imputacion", SimpleImputer(strategy="median")),
    ("escalado", RobustScaler()), # <-- Cambiado al vuelo
    ("modelo", LogisticRegression(max_iter=1000, random_state=SEED))
])

# Y lo evalúas igual:
resultados_robust = evaluar(pipeline_lr_robust, DATOS["P3_pcr"], "LogReg
+ Robust", "P3_pcr", verbose=True)

```

Análisis

La consulta resuelve la duda, aunque olvida la estructura de *evaluar* (hay que pasarle X, y, no datos). Parece indicar que para realizar esto va a ser más sencillo realizar una función para añadir modelos a pipelines fácilmente.

Prompt

es que se podría crear una función que fuese que si se mete el grid como parametro se hace con grid y que si no se hace metiendo el modelo y ya

Respuesta IA Gemini Flash Lite 3.5 en Gemini

````
¡Sí, exacto! De hecho, se puede hacer una **sola función inteligente**
que detecte automáticamente si le pasas un modelo normal o un
`GridSearchCV`. Como en scikit-learn tanto los modelos como el
`GridSearchCV` tienen los mismos métodos (`fit`, `predict`), puedes
unificarlo todo sin romper nada.

Aquí tienes cómo puedes definir esa función única:

```python
from sklearn.model_selection import GridSearchCV

def evaluar_modelo_o_grid(pipeline_preprocesado, estimador_o_grid,
df_dataset, nombre_modelo, nombre_preprocesado, param_grid=None,
verbose=True):
    """
    Evalúa un modelo o un GridSearchCV utilizando un pipeline de
    preprocesado base.
    Si se proporciona param_grid, envuelve automáticamente el estimador
    en un GridSearchCV.
    """
    # 1. Construir el pipeline completo base
    pipeline_completo = Pipeline([
        ("preprocesado", pipeline_preprocesado),
        ("modelo", estimador_o_grid)
    ])

    # 2. Si se pasa un param_grid, lo envolvemos en un GridSearchCV de
    forma automática
    if param_grid is not None:
        estimador_final = GridSearchCV(
            estimator=pipeline_completo,
            param_grid=param_grid,
            cv=3,
            scoring='f1_macro',
            n_jobs=-1
        )
        nombre_modelo_final = f"{nombre_modelo} (Grid)"
    else:
````
````

```
        estimador_final = pipeline_completo
        nombre_modelo_final = nombre_modelo

    # 3. Llamar a tu función 'evaluar' de siempre
    return evaluar(
        pipeline=estimador_final,
        df_dataset=df_dataset,
        nombre_modelo=nombre_modelo_final,
        nombre_preprocesado=nombre_preprocesado,
        verbose=verbose
    )

```

¿Cómo se usaría en tu cuaderno?

De esta forma tan limpia, **la misma función sirve para las dos situaciones**:

Caso A: Sin Grid (un modelo directo y rápido)

```python
resultado_simple = evaluar_modelo_o_grid(
    pipeline_preprocesado=pipeline_imputacionMediana_escaladoEstandar,
    estimador_o_grid=LogisticRegression(max_iter=1000,
    random_state=SEED),
    df_dataset=DATOS["P3_pcr"],
    nombre_modelo="Logistic Regression",
    nombre_preprocesado="P3_pcr",
    verbose=True
)

```

Caso B: Con Grid (buscando hiperparámetros)

```python
resultado_grid = evaluar_modelo_o_grid(
    pipeline_preprocesado=pipeline_imputacionMediana_escaladoEstandar,
    estimador_o_grid=LogisticRegression(max_iter=1000,
    random_state=SEED),
    df_dataset=DATOS["P3_pcr"],
    nombre_modelo="Logistic Regression",
    nombre_preprocesado="P3_pcr",
    param_grid={'modelo__C': [0.01, 0.1, 1.0, 10.0], 'modelo__solver':
    ['lbfgs']}, # ¡Con solo rellenar esto, ya hace el grid!
    verbose=True
)

```

Así te ahorras tener funciones duplicadas o andar montando pipelines a mano cada vez que quieras afinar parámetros.

Análisis

La consulta sirve para confirmar que la idea de crear una función pra simplificar código y crear un pipeline directo para la función de evaluación es correcta, pero incluir la evaluación en la función parece simplificación excesiva y no ayuda a la legibilidad del código.

Prompt

no metas la función de evaluar, tipo la llamaremos desde fuera de la funcion

Respuesta IA Gemini Flash Lite 3.5 en Gemini

¡Ah, perfecto! Entendido al 100%. Te refieres a que la función única se encargue únicamente de **construir y empaquetar** (ya sea el pipeline plano o metiéndolo dentro del `GridSearchCV`), pero **sin llamar a `evaluar()` dentro**, de forma que la llamada a `evaluar()` la hagas tú explícitamente desde fuera como hasta ahora.

Aquí tienes esa función de montaje inteligente:

```python
from sklearn.model_selection import GridSearchCV

def construir_estimador(pipeline_preprocesado, estimador,
param_grid=None):
    """
    Une el preprocesado y el modelo en un pipeline.
    Si se proporciona param_grid, lo envuelve automáticamente en un
    GridSearchCV.
    """
    pipeline_completo = Pipeline([
        ("preprocesado", pipeline_preprocesado),
        ("modelo", estimador)
    ])

    if param_grid is not None:
        return GridSearchCV(
            estimator=pipeline_completo,
            param_grid=param_grid,
            cv=3,
            scoring='f1_macro',
            n_jobs=-1
        )

    return pipeline_completo
```

````
```

---

### ¿Cómo lo usas con tu función `evaluar()` desde fuera?

Tanto si vas a probar un modelo simple como un grid, la llamas antes de
pasarle el resultado a tu `evaluar()`:

**1. Si es un modelo simple (sin grid):**

```python
# 1. Construyes el estimador con la función
estimador_lr = construir_estimador(
    pipeline_preprocesado=pipeline_imputacionMediana_escaladoEstandar,
    estimador=LogisticRegression(max_iter=1000, random_state=SEED)
)

# 2. Se lo pasas a tu evaluar() de siempre desde fuera
res_lr = evaluar(
    pipeline=estimador_lr,
    df_dataset=DATOS["P3_pcr"],
    nombre_modelo="Logistic Regression",
    nombre_preprocesado="P3_pcr",
    verbose=True
)

```

**2. Si lleva hiperparámetros (con grid):**

```python
# 1. Construyes el estimador pasando el param_grid
grid_lr = construir_estimador(
    pipeline_preprocesado=pipeline_imputacionMediana_escaladoEstandar,
    estimador=LogisticRegression(max_iter=1000, random_state=SEED),
    param_grid={'modelo__C': [0.01, 0.1, 1.0, 10.0], 'modelo__solver':
    ['lbfgs']}
)

# 2. Se lo pasas exactamente igual a tu evaluar() desde fuera
res_grid = evaluar(
    pipeline=grid_lr,
    df_dataset=DATOS["P3_pcr"],
    nombre_modelo="Logistic Regression (Grid)",
    nombre_preprocesado="P3_pcr",
    verbose=True
)

```
````
````

De esta manera, `evaluar()` se queda completamente intacta y agnóstica, y tú tienes una fábrica súper limpia (`construir_estimador`) que decide por ti si empaquetarlo en un Grid o dejarlo como un pipeline normal.

Análisis

El código de la función es correcto, aunque ha puesto que el scoring sea f1 macro si hay grid, lo cual debería ser correcto para un problema de clasificación multiclase. Sin embargo, como no sabe el enunciado ni el contexto de la práctica, se le debería preguntar si los parámetros del grid son correctos.

Prompt

Por cierto, el problema es de medicina de detección de riesgo cardiovascular con 3 variables. Sabiendo esto, el grid supongo que sigue sienod correcto, no?

Respuesta IA Gemini Flash Lite 3.5 en Gemini

¡Sí, totalmente! De hecho, **usar `GridSearchCV` sigue siendo completamente correcto y muy recomendable**, independientemente de que el problema sea médico o de cualquier otro ámbito.

Aquí te detallo por qué no hay ningún problema clínico ni metodológico en usarlo, y qué debes tener en cuenta:

1. ¿Por qué el Grid es correcto en medicina?

En problemas médicos (y más en la predicción de riesgo cardiovascular, donde te juegas que un falso negativo signifique no detectar a un paciente grave), afinar los hiperparámetros es clave para equilibrar métricas críticas como el **Recall de alto riesgo** o el **F1 macro** que estás usando.

El `GridSearchCV` te permite buscar de forma sistemática si un modelo necesita más o menos regularización (por ejemplo, el parámetro `C` en Logistic Regression o `max_depth` en Random Forest) para no sobreajustar con los datos clínicos.

2. El único peligro a vigilar (y cómo lo tienes resuelto)

El único "pecado" capital al hacer un Grid en machine learning médico es **caer en fugas de información (*data leakage*)**: optimizar los hiperparámetros usando datos que luego van a parar al conjunto de test.

Pero **en tu caso lo estás haciendo de forma impecable**:

* Tu función `evaluar` hace la partición por folds (`splitter.split`).

```
* El `GridSearchCV` se entrena y busca los mejores hiperparámetros
**dentro de cada fold de entrenamiento**, utilizando su propia validación
cruzada interna (`cv=3` dentro del Grid).
* El test de cada fold se mantiene sagrado y nunca ve el proceso de
optimización del Grid.

### 3. Un pequeño detalle a tener en cuenta con `scoring`

* Actualmente lo tienes en `scoring='f1_macro'`, lo cual está muy bien
para balancear todas las clases por igual.
* Si en algún momento ves que te interesa priorizar a toda costa la clase
de alto riesgo por encima de las demás, podrías cambiarlo en el Grid a
`scoring='recall_macro'` o incluso pasarle una métrica personalizada
enfocada exclusivamente en esa clase crítica. Pero para empezar y
comparar preprocesados, `f1_macro` es un estándar estupendo.

¡Vas con la arquitectura perfecta para sacar adelante los experimentos de
tu TFG o práctica médica con total rigor científico!
```

Análisis

Se confirma que el uso de f1 macro para scoring es correcto y que no hay ningún parámetro incorrecto en el grid search dado el tipo de problema que se aborda en la práctica.

Prompt

```
Tengo esto:

# Común a todos los pipelines de preprocesado
pipeline_codificacion = Pipeline([
    ("codificacion", ColumnTransformer([
        ("nominales", OneHotEncoder(drop="first",
handle_unknown="ignore"), NOMINALES),
        ("ordinal", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]),
ORDINAL)
    ], remainder="passthrough"))
])

# Preprocesado que imputa valores nulos con la mediana/moda y escala los
datos numéricos con escalado estándar
pipeline_imputacion_escaladoEstandar = Pipeline([
    ("codificacion", pipeline_codificacion),
    ("imputacion", SimpleImputer(strategy="median")),
    ("escalado", StandardScaler())
])
```

Entre mis variables hay booleanos, nominales a los que aplicar one-hot, nominales a los que aplicar ordinal y floats. No tengo nulos que no sean float. Crees que está bien esto?

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Echando un vistazo a tu código y a cómo has definido los bloques, hay un detalle técnico muy importante que **debes corregir**, ya que tal como está ahora mismo provocará un error en scikit-learn.

El problema: ¿Qué pasa con los booleanos?

En tu código actual estás pasando las variables nominales y las ordinales al `ColumnTransformer`, pero **no has incluido las variables booleanas (`BOOLEANOS`) en ningún sitio**.

Al poner `remainder="passthrough"` en el `ColumnTransformer`, las columnas booleanas pasarán de largo sin romperse, pero el problema viene justo después: **el `SimpleImputer(strategy="median")` y el `StandardScaler()` se van a aplicar a todo lo que pase**.

* La mediana no se puede calcular en columnas de tipo booleano (`True`/`False`).
* El `StandardScaler` intentará restar la media y dividir por la desviación típica en columnas booleanas, convirtiéndolas en floats continuos (lo cual deforma el sentido original de un booleano).

La solución: Tratar los booleanos explícitamente

Los booleanos en scikit-learn se suelen tratar mejor convirtiéndolos a enteros (`0` y `1`) o pasándolos por un imputador simple de frecuencia si tienen nulos, o simplemente asegurándonos de que el `ColumnTransformer` los gestione bien.

Lo más limpio y robusto para integrarlo en tu `pipeline_codificacion` es **añadir los booleanos al `ColumnTransformer` usando un `SimpleImputer(strategy="most_frequent")` o simplemente dejándolos pasar (`passthrough`) si estás 100% seguro de que no tienen nulos**.

Dado que mencionaste que *«no tengo nulos que no sean float»* (lo que significa que tus booleanos y categóricas no tienen nulos), puedes añadirlos al `ColumnTransformer` indicando que pasen tal cual (`"passthrough"`):

```python
# Común a todos los pipelines de preprocesado
pipeline_codificacion = Pipeline([
    ("codificacion", ColumnTransformer([
```

```
        ("nominales", OneHotEncoder(drop="first",
        handle_unknown="ignore"), NOMINALES),
        ("ordinal", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]),
        ORDINAL),
        ("booleanos", "passthrough", BOOLEANOS)  # <-- ¡Añadido aquí para
        que no sufran el escalador!
    ], remainder="drop"))  # <-- Ojo: cambiado a "drop" por seguridad
    para que ninguna columna extraña pase por descuido
])

```

> **Nota sobre el `remainder`:** Si cambias `remainder="passthrough"` a `remainder="drop"`, le estás diciendo al `ColumnTransformer` que *solo* procese y deje pasar las columnas que tú has declarado explícitamente (`NOMINALES`, `ORDINAL` y `BOOLEANOS`). Esto es genial porque **las numéricas (`NUMERICAS`) se quedan fuera de este primer bloque**, permitiendo que pasen intactas al siguiente paso del pipeline (la mediana y el escalador), que es exactamente lo que buscas.

Cómo quedaría tu bloque completo y seguro:

```python
# 1. Codificación de categóricas y paso de booleanos
pipeline_codificacion = Pipeline([
    ("codificacion", ColumnTransformer([
        ("nominales", OneHotEncoder(drop="first",
        handle_unknown="ignore"), NOMINALES),
        ("ordinal", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]),
        ORDINAL),
        ("booleanos", "passthrough", BOOLEANOS)
    ], remainder="drop")) # Solo dejamos pasar lo que controlamos
    explícitamente
])

# 2. Pipeline final que coge lo anterior, imputa y escala SOLO las
numéricas restantes
pipeline_imputacion_escaladoEstandar = Pipeline([
    ("codificacion", pipeline_codificacion),
    ("imputacion", SimpleImputer(strategy="median")), # Actúa sobre las
    numéricas que salieron del transformer anterior
    ("escalado", StandardScaler())                  # Escala también
    solo las numéricas
])

```

Con este ajuste te aseguras de que el `StandardScaler` y la mediana no toquen ni las nominales codificadas, ni las ordinales, ni los booleanos, aplicando el escalado exclusivamente a tus variables cuantitativas/floats. ¡Así evitas sorpresas extrañas durante el entrenamiento de los modelos!

Análisis

Surge la duda sobre si se debería haber aplicado el scaler a todo o deberían haberse descartado alguna columna, como las nominales o las booleanas. La respuesta parece indicar que efectivamente es mejor descartar estas columnas del escalado, pero habría que asegurarse de que efectivamente funciona así incluso en modelos que necesitan escalado, como SVC.

Prompt

pero estás seguro de que no debo aplicar scaler? piensa que hay modelos como SVC que son sensibles a que no estén escalados

Respuesta IA Gemini Flash Lite 3.5 en Gemini

¡Tienes toda la razón y haces muy bien en dudar! Me he explicado mal en el punto anterior.

Sí que hay que escalar las variables numéricas (`StandardScaler`), y de hecho por eso lo tenías metido en tu `pipeline_imputacion_escaladoEstandar` (`("escalado", StandardScaler())`). Modelos como **SVC**, **KNN** o **Logistic Regression** *exigen* que las variables numéricas estén escaladas para funcionar bien; si no, las variables con números más grandes (como el colesterol o los triglicéridos) aplastarán por completo a las que tienen rangos pequeños (como el IMC o la edad).

El matiz importante al que me refería no era quitar el escalador, sino **a qué columnas se le aplica**.

El problema de aplicar el escalador "a lo bruto"

Si en un pipeline plano pones el `StandardScaler()` al final de todo (después del `ColumnTransformer`), el escalador intentará aplicarse a **absolutamente todo lo que le llegue**, incluyendo:

1. Las columnas numéricas (¡perfecto, aquí sí debe ir!).
2. Las columnas **booleanas** (que se convertirán en floats raros con medias y desviaciones típicas).
3. Las columnas nominales que salieron del `OneHotEncoder` (que son columnas de ceros y unos; escalarlas arruina la lógica de la codificación binaria).

````
### ¿Cómo se soluciona correctamente para que las numéricas se escalen y
las demás no?

La forma profesional y limpia en scikit-learn es aplicar el
`ColumnTransformer` **dividiendo el trabajo**, de modo que el escalador y
la mediana afecten *únicamente* a las variables numéricas (`NUMERICAS`),
mientras que el resto va por su camino sin sufrir transformaciones
matemáticas extrañas.

Así es como estructurarías tus bloques para que **las numéricas se
escalen sí o sí**, pero sin contaminar las demás:

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder,
StandardScaler
from sklearn.impute import SimpleImputer

# 1. Bloque exclusivo para variables NUMÉRICAS (aquí sí entra la mediana
y el escalado)
pipeline_numericas = Pipeline([
    ("imputacion", SimpleImputer(strategy="median")),
    ("escalado", StandardScaler())  # <-- ¡Aquí está tu StandardScaler
    para SVC, KNN y Regresión Logística!
])

# 2. Pipeline global de preprocesamiento unificado
pipeline_preprocesado_completo = Pipeline([
    ("columnas", ColumnTransformer([
        ("num", pipeline_numericas, NUMERICAS),
        # Se imputan y se escalan
        ("nominales", OneHotEncoder(drop="first",
        handle_unknown="ignore"), NOMINALES), # One-hot limpio
        ("ordinal", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]),
        ORDINAL),         # Ordinal limpio
        ("booleanos", "passthrough", BOOLEANOS)
        # Booleanos intactos
    ], remainder="drop"))
])

```

### ¿Por qué esta estructura es perfecta para ti?

* **Protege a SVC y KNN:** Las variables numéricas van perfectamente
escaladas con `StandardScaler`.
* **Protege a las categóricas y booleanas:** No les aplica la media ni la
desviación típica a cosas que no son números continuos.
````

```
* **Modularidad total:** Si mañana en tus experimentos quieres probar un
`RobustScaler` en lugar de un `StandardScaler`, solo cambias esa línea en
el `pipeline_numericas` sin romper nada del resto del código.

¿Ves como así se respeta la necesidad del escalador para los modelos
sensibles pero con control quirúrgico sobre cada tipo de columna?
```

Análisis

La consulta busca confirmar si realmente se deberían descartar dichas columnas, lo cual parece lógico, lanzando una pregunta negativa hacia esta aplicación. Según la respuesta de la IA, destruir variables categóricas y booleanas convirtiéndolas en continuos rompe la lógica de la clasificación binaria, por lo que se acepta.

Prompt

Prefiero si los procesamientos nominales me los declaras aparte

Respuesta IA Gemini Flash Lite 3.5 en Gemini

````
Aquí tienes la estructura limpia y modular con los transformadores
categóricos declarados por separado, incluyendo la imputación por moda
(`most_frequent`) tanto para las variables nominales/ordinales como para
los booleanos, tal como has pedido:

```python
#
================================================================================
# TRANSFORMADORES CATEGÓRICOS Y BOOLEANOS
#
================================================================================
# Definimos los bloques de preprocesamiento para variables no numéricas
de forma
# independiente. Se añade un SimpleImputer con la moda ('most_frequent')
por seguridad
# ante posibles valores nulos, seguido de sus respectivas codificaciones.
transformadores_categoricos = [
    ("nominales", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("ohe", OneHotEncoder(drop="first", handle_unknown="ignore"))
    ]), NOMINALES),

    ("ordinal", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("ord", OrdinalEncoder(categories=[ORDEN_ACTIVIDAD]))
    ]), ORDINAL),

    ("booleanos", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent"))
    ]), BOOLEANOS)
````
````

```
]

#
================================================================================
# PREPROCESADO GLOBAL (NUMÉRICOS + CATEGÓRICOS)
#
================================================================================
# Este ColumnTransformer unifica todo: aplica imputación por mediana y
escalado
# estándar (StandardScaler) exclusivamente a las variables numéricas, y
le concatena
# todos los transformadores categóricos y booleanos definidos arriba.
pipeline_imputacionMediana_escaladoEstandar = ColumnTransformer(
    transformers=[
        ("numericos", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler())
        ]), NUMERICAS)
    ] + transformadores_categoricos,
    remainder="drop"  # Descartar cualquier columna que no esté
    explícitamente declarada
)

```

```
### ¿Por qué te viene genial dejarlo así?

1. **Independencia total:** Puedes reutilizar la lista
`transformadores_categoricos` en cualquier otro preprocesado futuro sin
tener que reescribir la codificación o la imputación por moda.
2. **Protección contra fugas (*Data Leakage*):** Al meter la imputación
dentro de los sub-pipelines del `ColumnTransformer`, la moda y la mediana
se calcularán **únicamente con el conjunto de entrenamiento** de cada
fold, manteniendo tu validación cruzada 100% rigurosa.
```

Análisis

La consulta busca que los preprocesamientos nominales se declaren aparte, lo cual es correcto con lo pedido. Sin embargo, incluye imputación de moda, lo cual es incorrecto porque no hay valores nulos pra estas variables, por lo que no es necesario.

Prompt

Háblame de la regresión logística. Dime qué aspectos hay te tener en cuenta, si el modelo acepta valores nulos, si es obligatorio escalar las variables, etc. Mi problema es sobre clasificación multiclase de riesgo cardiovascular

Respuesta IA Gemini Flash Lite 3.5 en Gemini

La **regresión logística** es uno de los algoritmos de clasificación más utilizados, robustos y fáciles de interpretar en ciencia de datos y medicina. Como tu problema es de **clasificación multiclase de riesgo cardiovascular** (por ejemplo, clasificar a los pacientes en riesgo *bajo, moderado, alto y muy alto*), se utiliza una extensión conocida como **Regresión Logística Multinomial** (o regresión Softmax).

A continuación, te detallo los aspectos clave que debes tener en cuenta para tu caso específico:

---

1. ¿Acepta valores nulos (missing values)?

* **No directamente.** Las implementaciones estándar en librerías populares como `scikit-learn` (`LogisticRegression`) **no aceptan valores nulos (`NaN`)** y arrojarán un error si intentas entrenar el modelo con ellos.
* **Qué debes hacer:** Tendrás que realizar un proceso previo de limpieza de datos (**imputación**). Puedes usar técnicas simples (como rellenar con la mediana o la media) o avanzadas (como KNN Imputer o MICE), teniendo mucho cuidado de calcular los estadísticos solo en el conjunto de entrenamiento para evitar el *data leakage* (fuga de datos). En medicina, un valor nulo a veces contiene información implícita (ej. si no se midió cierto biomarcador, podría ser por bajo riesgo o protocolo clínico), por lo que crear indicadores binarios de ausencia ("missingness indicators") suele ser útil.

2. ¿Es obligatorio escalar las variables?

* **Matemáticamente estricto:** No es obligatorio para que el algoritmo converja (a diferencia de las redes neuronales o las máquinas de vectores de soporte - SVM).
* **En la práctica (¡Muy recomendable!):** Si vas a utilizar **regularización** (como L1/Lasso o L2/Ridge, que vienen activadas por defecto en scikit-learn con el parámetro `penalty`), **el escalado es obligatorio**.
* **Por qué en tu problema importa:** En riesgo cardiovascular mezclarás variables con escalas muy distintas (por ejemplo: la *edad* de 30 a 80 años, el *colesterol total* de 150 a 300 mg/dL y la *presión arterial sistólica* de 90 a 180 mmHg). Sin escalar, las variables con números más grandes dominarán la penalización de la regularización, haciendo que el modelo interprete erróneamente su importancia. Utiliza herramientas como `StandardScaler` (media 0, desviación estándar 1).

---

3. Aspectos clave a tener en cuenta para el Riesgo Cardiovascular

```
* **Multiclase (Softmax):** Al tener múltiples categorías de riesgo (ej.
4 niveles), el modelo calculará la probabilidad de pertenecer a cada
categoría utilizando la función **Softmax**, asignando al paciente la
clase con mayor probabilidad.
* **Interpretabilidad clínica (Odds Ratios):** Una de las mayores
ventajas de la regresión logística en medicina es que sus coeficientes se
pueden exponencializar para obtener **Odds Ratios (OR)**. Esto te permite
explicar a nivel médico cuánto aumenta o disminuye la probabilidad de
riesgo cardiovascular por cada unidad que sube una variable (ej. por cada
año de edad o cada 10 mg/dL de colesterol).
* **Desbalance de clases (Class Imbalance):** En cardiología preventiva,
la mayoría de la población suele estar en riesgo bajo o moderado,
mientras que los casos de riesgo muy alto son minoría. Si tus clases
están muy desbalanceadas, el modelo tenderá a ignorar las clases
minoritarias.
* *Solución:* Utiliza el parámetro `class_weight='balanced'` en
scikit-learn o técnicas de remuestreo (SMOTE).


* **Linealidad en el Log-Odds:** La regresión logística asume una
relación lineal entre las variables independientes y el logaritmo de las
probabilidades de las clases. Si alguna variable continua (como la
presión arterial o la glucosa) tiene una relación no lineal con el riesgo
cardiovascular, el modelo perderá precisión a menos que apliques
transformaciones (como polinomios o *splines*).

---
```

Análisis

La consulta busca una explicación sobre la regresión logística (uno de los modelos elegidos como base), indagando en aspectos como el manejo de valores nulos y la necesidad de escalar las variables para comprobar a qué preprocesamientos es sensible. La respuesta aborda de manera precisa estos puntos al indicar que el modelo no admite nulos directamente (exigiendo imputación) y que el escalado es altamente recomendable. Además, al indicar en la consulta el tipo de problema que se aborda recomienda el uso de *class_weight='balanced'* si las clases están desbalanceadas, lo cual se tendrá en cuenta como opción durante el entrenamiento.

Prompt

ahora sobre LinearSVC

Respuesta IA Gemini Flash Lite 3.5 en Gemini

El **`LinearSVC` (Linear Support Vector Classifier)** es otra excelente opción lineal, pero con una filosofía distinta a la regresión logística. Mientras que la regresión logística busca maximizar la probabilidad (estimando probabilidades de clase), `LinearSVC` busca **encontrar el hiperplano óptimo que maximice el margen** de separación entre las diferentes clases de riesgo cardiovascular.

A continuación, te detallo cómo se comporta respecto a los mismos aspectos y qué debes tener en cuenta en tu problema multiclase:

---

1. ¿Acepta valores nulos (missing values)?

* **Tampoco los acepta.** Al igual que la gran mayoría de modelos en `scikit-learn`, `LinearSVC` fallará si le pasas matrices con valores `NaN`.
* **Qué debes hacer:** Debes aplicar exactamente el mismo preprocesamiento de imputación que en la regresión logística antes de entrenar el modelo.

---

2. ¿Es obligatorio escalar las variables?

* **Sí, de forma estricta y absoluta.** A diferencia de la regresión logística (donde el escalado es muy recomendable por la regularización), en los SVMs **el escalado es fundamental** porque el algoritmo se basa en el cálculo de distancias geométricas y en la maximización de márgenes.
* **Por qué en tu problema importa:** Si no escalas las variables (por ejemplo, manteniendo la *edad* en una escala de 30 a 80 y los *triglicéridos* de 50 a 500), el hiperplano estará dominado por completo por las variables con magnitudes numéricas más grandes. El modelo interpretará erróneamente que esas variables son más importantes solo por sus unidades de medida. Utiliza siempre `StandardScaler` o `MinMaxScaler`.

---

3. Aspectos clave para tu problema de Riesgo Cardiovascular

* **Multiclase mediante One-vs-Rest (OvR):** Por defecto, `LinearSVC` maneja problemas multiclase utilizando la estrategia **"Uno contra el resto" (One-vs-Rest)**. Si tienes 4 niveles de riesgo cardiovascular, el algoritmo entrenará 4 clasificadores binarios independientes (cada uno aprende a separar un nivel de riesgo frente a todos los demás combinados).

```
* **Control de la regularización (`C`):** Al igual que en la regresión
logística, el parámetro `C` controla el equilibrio entre maximizar el
margen y minimizar los errores de clasificación en el entrenamiento. Un
`C` alto penaliza más los errores (puede sobreajustar); un `C` bajo
permite un margen más suave (puede subajustar). Deberás ajustarlo
mediante validación cruzada.
* **Manejo de desbalance (`class_weight`):** Al igual que la regresión
logística, `LinearSVC` incluye el parámetro `class_weight='balanced'`, lo
cual es vital si en tu base de datos los pacientes con riesgo
cardiovascular muy alto son una minoría escasa.

---
```

Análisis

Siguiendo con LinearSVC, otro modelo elegido como modelo base, se preguntan las mismas cuestiones que con regresión logística, que, a diferencia de la regresión logística, en LinearSVC el escalado de variables es estricta y absolutamente obligatorio. También señala la necesidad de imputar los valores nulos. Además, indica qué efecto se obtendría al aumentar o disminuir el valor del parámetro C, lo cual fue útil saber para más adelante saber la causa del sobreajuste causado por los valores iniciales altos del C.

Prompt

ahora svm

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
Cuando hablamos de un **SVM genérico (`SVC` en scikit-learn)**, por lo
general nos referimos a modelos que utilizan **kernels no lineales**
(siendo el más famoso el kernel RBF o Radial Basis Function). A
diferencia de los modelos lineales que vimos antes, estos algoritmos
pueden trazar fronteras de decisión curvas o complejas en el espacio de
tus datos.

A continuación, te detallo cómo se comporta bajo los mismos criterios y
qué debes considerar para tu problema multiclase de riesgo
cardiovascular:

---

### 1. ¿Acepta valores nulos (missing values)?

* **Tampoco los acepta.** Al igual que la regresión logística y
`LinearSVC`, las implementaciones estándar de `SVC` fallarán si detectan
valores `NaN`.
* **Qué debes hacer:** Necesitas aplicar el mismo preprocesamiento de
imputación previa (rellenar con la media/mediana o usar imputadores
avanzados) antes de alimentar al modelo.
```

```
---

### 2. ¿Es obligatorio escalar las variables?

* **Sí, de forma absoluta e innegociable.** Los SVM basados en kernels
calculan distancias euclidianas y similitudes espaciales entre los datos.
* **Por qué en tu problema importa:** Si no escalas variables como la
*presión arterial* (90-180) frente a la *edad* (30-80), el espacio
vectorial se distorsionará por completo. Las variables con números
grandes dominarán la función de distancia, haciendo que el modelo ignore
las demás. Utiliza siempre `StandardScaler`.

---

### 3. Aspectos clave a tener en cuenta para tu problema de Riesgo
Cardiovascular

* **El poder de los Kernels No Lineales (Ej. RBF):** Esta es la mayor
ventaja del SVM clásico. En medicina, muchos factores de riesgo
cardiovascular no se relacionan de forma lineal; por ejemplo, el impacto
conjunto del tabaquismo y el colesterol alto puede disparar el riesgo de
una manera que una línea recta no logra capturar. Un kernel RBF permite
separar clases con formas circulares, concéntricas o muy complejas.
* **Estrategia Multiclase (One-vs-One - OvO):** Por defecto, `SVC` en
scikit-learn resuelve los problemas multiclase mediante la estrategia
**Uno contra Uno (OvO)**. Si tienes 4 niveles de riesgo cardiovascular,
el algoritmo entrenará $\frac{4 \times (4-1)}{2} = 6$ clasificadores
binarios y el resultado final se decide por un sistema de votación.
* **El gran talón de Aquiles: La interpretabilidad ("Caja Negra"):**
Mientras que en la regresión logística podías extraer *Odds Ratios*
explicables para los médicos, un SVM con kernel RBF **pierde la
interpretabilidad directa**. Es muy difícil explicarle a un comité
clínico *por qué* exacta razón matemática un paciente fue clasificado en
riesgo "alto" más allá de decir que está cerca de un vector de soporte en
un espacio de alta dimensión.
* **Coste computacional:** Los SVMs no escalan bien con bases de datos
enormes. Su tiempo de entrenamiento crece de forma cuadrática o cúbica
respecto al número de muestras ($O(n^2)$ a $O(n^3)$). Si manejas
registros de cientos de miles de pacientes a nivel nacional, el
entrenamiento puede volverse extremadamente lento.
* **Hiperparámetros críticos (`C` y `gamma`):**
* `C`: Controla el equilibrio entre un margen de separación amplio y la
penalización por errores de clasificación.
* `gamma`: Define la influencia de un solo ejemplo de entrenamiento. Un
`gamma` muy alto genera fronteras hiper-complejas que se ajustan
demasiado a los datos, provocando un grave **sobreajuste
(*overfitting*)**, lo cual es peligroso en medicina porque el modelo
fallará con pacientes nuevos.
```

```
* **Estimación de probabilidades:** Al igual que `LinearSVC`, `SVC` no
devuelve probabilidades puras de forma nativa a menos que actives el
parámetro `probability=True` durante su inicialización (lo cual encarece
aún más el tiempo de cómputo porque usa validación cruzada interna para
calibrarlas).

---
```

Análisis

Al no haber un consenso sobre qué dos modelos base elegir, se pregunta por un tercero que se decidió en equipo añadir, el SVM. La explicación ayuda a convencerse de añadir este modelo, puesto que explica las ventajas de un enfoque no lineal frente a los dos modelos lineales. Enfatiza la obligatoriedad del escalado de variables e imputar valores nulos. También indica los hiperparámetros principales, habiendo uno nuevo *gamma* frente a los dos modelos anteriores.

Prompt

```
que valores le pongo?

param_grid_lr = {
    'modelo__C': [0.01, 0.1, 1.0, 10.0],
    'modelo__solver': ['lbfgs'],
    'modelo__max_iter': [1000]
}

pipeline_imputacionMediana_escaladoEstandar_lr =
construir_pipeline_evaluacion(
    pipeline_imputacionMediana_escaladoEstandar,
    LogisticRegression(solver='lbfgs', max_iter=1000, random_state=SEED),
    param_grid_lr
)

# Para dado un pipeline y un modelo, construir un pipeline completo que
incluya el preprocesado y el modelo (y opcionalmente un GridSearchCV si
se proporciona un param_grid).
def construir_pipeline_evaluacion(pipeline_preprocesado, modelo,
param_grid=None):
    pipeline_completo = Pipeline([
        ("preprocesado", pipeline_preprocesado),
        ("modelo", modelo)
    ])
    if param_grid is not None:
        return GridSearchCV(estimator=pipeline_completo,
        param_grid=param_grid, cv=3, scoring='f1_macro', n_jobs=-1)
    return pipeline_completo

O no es recomendable hacer grid con max_iter? es que me va bien ya así.
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

No es recomendable incluir max_iter en el GridSearchCV si el modelo ya converge sin advertencias. Este parámetro solo define el límite de pasos para que el algoritmo encuentre la solución y no cambia la estructura matemática del modelo una vez que alcanza el mínimo.

Qué hacer con los parámetros

• Elimina max_iter y solver del grid: Si ya funcionan bien y no te sale ningún aviso de ConvergenceWarning, déjalos fijos dentro de LogisticRegression(max_iter=1000, solver='lbfgs', random_state=SEED) para ahorrar tiempo de cómputo.

• Amplía el rango de C: Este es el hiperparámetro más importante porque controla la regularización (valores bajos evitan el sobreajuste y valores altos permiten más complejidad). Prueba una escala logarítmica más amplia como [0.001, 0.01, 0.1, 1.0, 10.0, 100.0].

• Considera class_weight: Dado que estás evaluando con f1_macro, es muy probable que tus clases estén desbalanceadas. Añadir 'modelo__class_weight': [None, 'balanced'] suele mejorar bastante esta métrica.

• Evalúa la penalización (opcional): Si quieres probar regularización L1 para descartar variables irrelevantes, puedes cambiar el solver a 'liblinear' o 'saga' y añadir 'modelo__penalty': ['l1', 'l2'].

Código recomendado

python
```
param_grid_lr = {
    'modelo__C': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0],
    'modelo__class_weight': [None, 'balanced']
}

pipe_lr = construir_pipeline(
    'Prep1_Standard',
    LogisticRegression(solver='lbfgs', max_iter=1000, random_state=SEED)
)

grid_lr = GridSearchCV(pipe_lr, param_grid_lr, cv=3, scoring='f1_macro',
n_jobs=-1)
```

Si quieres afinar aún más este modelo base, dime:

• Si tu dataset tiene muchas columnas (por ejemplo, tras aplicar One-Hot Encoding)

• Si tus clases están muy desbalanceadas

```
Con eso puedo decirte si te conviene probar regularización L1 o ajustar
los umbrales de decisión.
```

Análisis

La herramienta no solo resuelve la duda inicial sobre incluir max_iter en el GridSearchCV, indicando que si el modelo ya converge, no aporta valor optimizarlo y conviene fijarlo, sino que identifica C como el hiperparámetro crítico al controlar la regularización y sugiere ampliar sus rangos de búsqueda. Además, al detectar el uso de la métrica f1_macro, propone incorporar el parámetro class_weight, un ajuste que desconocía y que resulta especialmente pertinente dado el desbalanceo de clases del problema. Aprovechando el bajo coste computacional de este modelo base, se decide aceptar ambas sugerencias. Finalmente, esta interacción pone de manifiesto la importancia de revisar en detalle la documentación de los modelos seleccionados para la práctica con el fin de identificar otros hiperparámetros relevantes que puedan pasar desapercibidos.

Prompt

¿Qué valores para el kernel hay?

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
En el contexto de las Máquinas de Vectores de Soporte (SVM) y librerías
como `scikit-learn`, el parámetro `kernel` determina la función
matemática que transforma los datos para encontrar las fronteras de
separación.

Los valores estándar que puedes asignar al parámetro `kernel` en una
función `SVC()` son los siguientes:

---

### 1. `'rbf'` (Radial Basis Function / Gaussiano)

* **Qué hace:** Es el **kernel por defecto**. Proyecta los datos a un
espacio de dimensiones infinitas mediante una función exponencial basada
en la distancia euclidiana. Crea fronteras de decisión circulares,
elípticas o curvas muy flexibles.
* **Cuándo usarlo:** Cuando sospechas que las relaciones entre tus
variables médicas no son lineales (por ejemplo, si el riesgo
cardiovascular no sube de forma recta con la edad y el colesterol, sino
que se dispara de manera exponencial a partir de ciertos umbrales
combinados).
* **Precaución:** Es muy potente, pero si ajustas mal sus hiperparámetros
(`C` y `gamma`), es el que mayor riesgo tiene de sufrir de *overfitting*
(sobreajuste).

### 2. `'linear'` (Lineal)
```

```
* **Qué hace:** Calcula el hiperplano de separación utilizando una línea
recta (en 2D), un plano (en 3D) o un hiperplano (en alta dimensión). Es
matemáticamente equivalente a una regresión logística pero optimizando
márgenes.
* **Cuándo usarlo:** Cuando tus datos son linealmente separables o
quieres **máxima interpretabilidad clínica**. Al ser lineal, puedes
extraer los coeficientes de las variables para saber exactamente cuánto
aporta cada factor de riesgo (ej. tabaquismo, presión arterial) al
resultado final.
* **Ventaja:** Es rápido de entrenar y menos propenso a sobreajuste en
datasets complejos.

### 3. `'poly'` (Polinómico)

* **Qué hace:** Crea fronteras de decisión basadas en funciones
polinómicas. Puedes controlar el grado del polinomio con el parámetro
`degree` (por defecto es 3).
* **Cuándo usarlo:** Cuando quieres modelar interacciones explícitas
entre variables (por ejemplo, cómo interactúa la edad multiplicada por la
presión arterial).
* **Precaución:** Con grados altos ($degree > 3$), el modelo se vuelve
extremadamente complejo, computacionalmente lento y muy propenso a
sobreajuste, por lo que rara vez se prefiere frente al RBF en medicina.

### 4. `'sigmoid'` (Sigmoide)

* **Qué hace:** Utiliza la función sigmoide (similar a la que usa la
regresión logística o una neurona artificial básica de tipo *tanh*).
* **Cuándo usarlo:** En la práctica moderna de machine learning **se
utiliza muy poco**. Fue popular por su conexión con las redes neuronales,
pero los kernels RBF y lineal suelen ofrecer resultados muy superiores en
casi todos los escenarios.

### 5. Una función personalizada (`callable`)

* **Qué hace:** `scikit-learn` te permite pasar tu propia función
matemática personalizada como kernel.
* **Cuándo usarlo:** Si en cardiología tienes una métrica de distancia o
similitud clínica propia y patentada que no encaja en las matemáticas
tradicionales, puedes programarla tú mismo y pasársela al modelo.

---

### ¿Cuál deberías elegir para tu problema de riesgo cardiovascular?

Lo más recomendable en ciencia de datos aplicada a la salud es seguir una
estrategia iterativa:
```

```
1. Empieza probando **`kernel='linear'`**: Te dará una línea base rápida
y una gran interpretabilidad para los médicos.
2. Luego prueba **`kernel='rbf'`**: Evalúa si la flexibilidad del kernel
gaussiano mejora significativamente la capacidad predictiva (por ejemplo,
midiendo el área bajo la curva ROC - AUC) en comparación con el modelo
lineal.

---
```

Análisis

Los primeros ajustes de hiperparámetros en el grid no se estaban realizando correctamente y provocaban un overfitting con un 0.17 de GAP, incluso si se reducía el valor de C, que fue la solución que la IA daba para el tratamiento del sobreajuste. Por lo tanto, se buscaban otras alternativas que pudiesen solucionar este aspecto aparte de ajustar C y gamma, siendo este cambiar el tipo de kernel. La consulta explica correctamente los tipos.

Prompt

si cambio a kernel=linear, deja de ser una SVM? O sería como hacer un Linear SVC?

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
**No, no deja de ser una SVM.** De hecho, un modelo con `kernel='linear'`
es una **Máquina de Vectores de Soporte Lineal**.

Y respecto a tu segunda duda: **sí, conceptualmente hace prácticamente lo
mismo que `LinearSVC**`, ya que ambos buscan encontrar el hiperplano
óptimo para separar las clases mediante la maximización del margen.

Sin embargo, aunque el objetivo matemático es idéntico, **por debajo del
capó (en su implementación en `scikit-learn`) son muy diferentes**. Aquí
te explico las diferencias clave que debes tener en cuenta para tu
proyecto de riesgo cardiovascular:

---

### 1. El motor de cálculo (Librerías distintas)

* **`SVC(kernel='linear')`** utiliza la librería **`libsvm`**. Resuelve
el problema matemático desde su forma **dual**.
* **`LinearSVC()`** está optimizado específicamente para casos lineales y
utiliza la librería **`liblinear`**. Resuelve el problema desde su forma
**primal**.

### 2. Velocidad y escalabilidad (Muy importante)
```

```
* **`LinearSVC` es mucho más rápido** cuando tienes una base de datos
grande (por ejemplo, miles o cientos de miles de registros de pacientes).
Su tiempo de entrenamiento escala de forma lineal respecto al número de
muestras.
* **`SVC(kernel='linear')` es más lento** con datasets grandes porque el
algoritmo basado en `libsvm` escala de forma cuadrática o cúbica
($O(n^2)$ a $O(n^3)$ con respecto al número de filas).

### 3. Opciones de penalización y pérdidas

* **`LinearSVC`** te permite elegir diferentes funciones de pérdida (como
`hinge` o `squared_hinge`) y soporta penalización L1 de forma muy
eficiente (lo que te ayuda a hacer selección automática de variables
médicas descartando las que no sirven).
* **`SVC(kernel='linear')`** se limita a la pérdida estándar de SVM y no
ofrece tanta flexibilidad en las penalizaciones avanzadas.

### 4. Resultados (¿Dan exactamente lo mismo?)

Por lo general, las fronteras de decisión y las métricas de rendimiento
(como el Accuracy o el AUC) serán **casi idénticas**, pero **raramente
100% idénticas**. Esto ocurre porque los optimizadores matemáticos
(`libsvm` frente a `liblinear`) encuentran la solución numérica por
caminos ligeramente distintos.

---

### ¿Cuál deberías usar para tu problema de riesgo cardiovascular?

* **Usa `LinearSVC()**` si tu dataset de pacientes es mediano o grande, o
si quieres experimentar rápidamente con penalizaciones L1 para ver qué
variables clínicas (colesterol, presión, edad) son verdaderamente
relevantes.
* **Usa `SVC(kernel='linear')**` solo si tu dataset es pequeño y quieres
mantener la flexibilidad de cambiar a `kernel='rbf'` o `kernel='poly'`
simplemente cambiando una línea de texto en tu código para comparar
resultados.
```

Análisis

Antes de comprender cómo optimizar adecuadamente los valores del grid, una solución que se encontró fue cambiar kernel a linear. Sin embargo, esto generaba dudas conceptuales sobre si seguía tratándose de una SVM o si se debería de quitar porque equivalía a hacer LinearSVC. Con esta respuesta, se confirma la viabilidad de mantener el modelo como una opción.

Prompt

Para facilitar la inserción del modelo con el pipeline de preprocesado, se ha creado la siguiente función, la cual encapsula las etapas del preprocesado y el modelo. También se añade la posibilidad de proporcionar un diccionario de hiperparámetros para determinar la configuración óptima del modelo. Se trata de un grid de validación cruzada con 3 iteraciones y optimización basada en `f1_macro`. Se ha elegido esta métrica para optimizar frente a `recall_alto` (útil en este problema, ya que al ser uno de medicina en el que se mide el riesgo cardiovascular, no detectar un paciente de alto riesgo supondría el error más grave) porque `f1_macro` ofrece un equilibrio entre sensibilidad y precisión, evitando que los modelos se centren únicamente en maximizar la detección de los casos como de alto riesgo, disparando las falsas alarmas, como harían si se eligiese `recall alto` como métrica.

Esto que expreso aquí es cierto? O debería de incluirse recall_alto como la métrica? Ya que es mejor que se detecte un falso positivo como alto que no detectarse.

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Sí, lo que expresas en tu texto es completamente cierto, riguroso y metodológicamente sólido. No deberías cambiar f1_macro por recall_alto como métrica principal de optimización en el GridSearchCV.
Las razones técnicas que validan tu decisión y desaconsejan el cambio son las siguientes:

• El peligro del colapso del modelo con Recall puro: Si optimizas un GridSearchCV usando únicamente el recall de la clase "alto riesgo", el buscador de hiperparámetros premiará a modelos degenerados que clasifiquen a casi todos los pacientes como "alto riesgo". Un modelo inútil que prediga siempre "alto riesgo" obtiene un recall del 100 %, por lo que ganaría la validación cruzada sin haber aprendido realmente a discriminar entre pacientes.

• La función de control de f1_macro: Al utilizar f1_macro (la media armónica entre precisión y sensibilidad promediada sin ponderar entre todas las clases), obligas al modelo a mantener un rendimiento equilibrado en todos los niveles de riesgo. Penaliza severamente las falsas alarmas masivas y asegura que el modelo tenga capacidad discriminativa real en las clases minoritarias y mayoritarias.

• La separación entre optimización y umbral de decisión: En problemas médicos, la optimización de hiperparámetros sirve para encontrar el modelo que mejor separa las clases (lo que logras con f1_macro). La prioridad clínica de evitar falsos negativos (tu preocupación de que es mejor un falso positivo que no detectar un caso grave) se gestiona después ajustando el umbral de probabilidad (threshold tuning), no sesgando la métrica de entrenamiento.

Tu redacción actual justifica de forma impecable por qué descartas optimizar directamente sobre el recall de la clase alta, demostrando que entiendes tanto el impacto clínico del problema como el comportamiento matemático de los algoritmos de optimización.

Si quieres, puedo ayudarte a revisar el código de tu pipeline o a redactar una breve nota técnica sobre el ajuste del umbral de decisión para reforzar aún más esa sección de tu memoria. ¿Te gustaría que lo viéramos?

Análisis

Aunque se estuvo discutiendo en equipo si usar f1_macro o el recall para el grid, al terminar de escribir este párrafo me volvieron a surgir las dudas, además de que no estaba plenamente justificado. Con esta consulta pude validar que mi razonamiento inicial era correcto, descartar el recall como métrica para evitar que los modelos no aprendar y mejorar la justificación de esta decisión tomada.

Prompt

```
def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    metricas_folds = {
        "train_accuracy": [], "train_f1_macro": [], "train_recall_alto":
[],
        "test_accuracy": [], "test_f1_macro": [], "test_recall_alto": []
    }
    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
{nombre_preprocesado}...")
    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
y), start=1):
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]
        pipeline_fold = clone(pipeline)
        pipeline_fold.fit(X_train_fold, y_train_fold)
        y_pred_train = pipeline_fold.predict(X_train_fold)
        y_pred_test = pipeline_fold.predict(X_test_fold)
        m_train = metricas(y_train_fold, y_pred_train, "train")
        m_test = metricas(y_test_fold, y_pred_test, "test")
        for k, v in {**m_train, **m_test}.items():
            metricas_folds[k].append(v)
        if verbose:
            print(f"--- Iteración {iteracion} ---")
            print(f"  F1 Macro   -> Train: {m_train['train_f1_macro']:.4f}
| Test: {m_test['test_f1_macro']:.4f}")
```

```
            print(f"  Recall Alto -> Train:
{m_train['train_recall_alto']:.4f} | Test:
{m_test['test_recall_alto']:.4f}")
            print(f"  Accuracy   -> Train: {m_train['train_accuracy']:.4f}
| Test: {m_test['test_accuracy']:.4f}")
    calculadas = {}
    for metrica, valores in metricas_folds.items():
        media = np.mean(valores)
        std = np.std(valores, ddof=1)
        calculadas[metrica] = f"{media:.4f} ± {std:.4f}"
    resumen_dict = {
        "preprocesado": nombre_preprocesado,
        "modelo": nombre_modelo,
        "train_f1_macro": calculadas["train_f1_macro"],
        "test_f1_macro": calculadas["test_f1_macro"],
        "train_recall_alto": calculadas["train_recall_alto"],
        "test_recall_alto": calculadas["test_recall_alto"],
        "train_accuracy": calculadas["train_accuracy"],
        "test_accuracy": calculadas["test_accuracy"],
    }
    df_resumen = pd.DataFrame([resumen_dict])
    if verbose:
        print(f"\nResumen global:")
        print(f"  F1 MACRO   -> Train: {resumen_dict['train_f1_macro']} |
Test: {resumen_dict['test_f1_macro']}")
        print(f"  RECALL ALTO-> Train: {resumen_dict['train_recall_alto']}
| Test: {resumen_dict['test_recall_alto']}")
        print(f"  ACCURACY   -> Train: {resumen_dict['train_accuracy']} |
Test: {resumen_dict['test_accuracy']}")
    return df_resumen

Añade el gap entre train y test de f1
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Para calcular y añadir el **gap** (la diferencia) entre train y test para la métrica `f1_macro` dentro de tu función, podemos calcular la media de las diferencias por fold o restar las medias globales. Lo más robusto y estadísticamente correcto en validación cruzada es **calcular el gap en cada fold** y luego obtener su media y desviación estándar (igual que haces con las demás métricas), o bien restarlas directamente.

Aquí tienes el código actualizado con el gap de `f1_macro` incluido tanto en el diccionario por fold, como en el resumen final y en las impresiones en pantalla (`verbose`):

```python
import numpy as np
import pandas as pd
from sklearn.base import clone
```

```
def evaluar(pipeline, X, y, nombre_modelo, nombre_preprocesado,
verbose=False):
    metricas_folds = {
        "train_accuracy": [], "train_f1_macro": [], "train_recall_alto":
        [],
        "test_accuracy": [], "test_f1_macro": [], "test_recall_alto": [],
        "gap_f1_macro": []  # <-- Añadido para almacenar el gap por fold
    }

    if verbose:
        print(f"Evaluando modelo {nombre_modelo} con preprocesado
        {nombre_preprocesado}...")

    for iteracion, (train_idx, test_idx) in enumerate(splitter.split(X,
    y), start=1):
        X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
        y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]

        pipeline_fold = clone(pipeline)
        pipeline_fold.fit(X_train_fold, y_train_fold)

        y_pred_train = pipeline_fold.predict(X_train_fold)
        y_pred_test = pipeline_fold.predict(X_test_fold)

        m_train = metricas(y_train_fold, y_pred_train, "train")
        m_test = metricas(y_test_fold, y_pred_test, "test")

        # Calculamos el gap de f1_macro para este fold (Train - Test)
        # Un valor positivo alto indica sobreajuste (overfitting)
        gap_f1 = m_train['train_f1_macro'] - m_test['test_f1_macro']
        m_fold_extra = {"gap_f1_macro": gap_f1}

        for k, v in {**m_train, **m_test, **m_fold_extra}.items():
            metricas_folds[k].append(v)

        if verbose:
            print(f"--- Iteración {iteracion} ---")
            print(f"   F1 Macro    -> Train:
            {m_train['train_f1_macro']:.4f} | Test:
            {m_test['test_f1_macro']:.4f} | Gap: {gap_f1:.4f}")
            print(f"   Recall Alto -> Train:
            {m_train['train_recall_alto']:.4f} | Test:
            {m_test['test_recall_alto']:.4f}")
            print(f"   Accuracy    -> Train:
            {m_train['train_accuracy']:.4f} | Test:
            {m_test['test_accuracy']:.4f}")

    calculadas = {}
    for metrica, valores in metricas_folds.items():
```

```
        media = np.mean(valores)
        std = np.std(valores, ddof=1)
        calculadas[metrica] = f"{media:.4f} ± {std:.4f}"

    resumen_dict = {
        "preprocesado": nombre_preprocesado,
        "modelo": nombre_modelo,
        "train_f1_macro": calculadas["train_f1_macro"],
        "test_f1_macro": calculadas["test_f1_macro"],
        "gap_f1_macro": calculadas["gap_f1_macro"],  # <-- Añadido al
        diccionario resumen
        "train_recall_alto": calculadas["train_recall_alto"],
        "test_recall_alto": calculadas["test_recall_alto"],
        "train_accuracy": calculadas["train_accuracy"],
        "test_accuracy": calculadas["test_accuracy"],
    }

    df_resumen = pd.DataFrame([resumen_dict])

    if verbose:
        print(f"\nResumen global:")
        print(f"   F1 MACRO    -> Train: {resumen_dict['train_f1_macro']}
        | Test: {resumen_dict['test_f1_macro']} | Gap:
        {resumen_dict['gap_f1_macro']}")
        print(f"   RECALL ALTO -> Train:
        {resumen_dict['train_recall_alto']} | Test:
        {resumen_dict['test_recall_alto']}")
        print(f"   ACCURACY    -> Train: {resumen_dict['train_accuracy']}
        | Test: {resumen_dict['test_accuracy']}")

    return df_resumen

```

Cambios realizados:

1. Se añadió `"gap_f1_macro": []` al diccionario inicial `metricas_folds`.
2. Dentro del bucle, se calcula `m_train['train_f1_macro'] - m_test['test_f1_macro']` por cada iteración. Esto te permite evaluar con precisión estadística (media $\pm$ desviación estándar) cuánto se distancia el rendimiento de entrenamiento frente al de prueba.
3. Se integró `gap_f1_macro` tanto en el DataFrame resultante (`df_resumen`) como en los mensajes informativos en pantalla cuando `verbose=True`.

Análisis

Añade lo pedido para poder visualizar el overfitting facilmente. Se acepta.

Prompt

Añade también la impresión de los mejores parámetros.

Respuesta IA Gemini Flash Lite 3.5 en Gemini

Aquí tienes la actualización en la sección del bucle donde se imprimen los resultados por iteración:

```python
        if verbose:
            print(f"--- Iteración {iteracion} ---")
            if hasattr(pipeline_fold, "best_params_"):
                print(f"  Mejores params ->
                {pipeline_fold.best_params_}")
            print(f"  F1 Macro   -> Train: {m_train['train_f1_macro']:.4f}
            | Test: {m_test['test_f1_macro']:.4f} | Gap: {gap_f1:.4f}")
            print(f"  Recall Alto -> Train:
            {m_train['train_recall_alto']:.4f} | Test:
            {m_test['test_recall_alto']:.4f}")
            print(f"  Accuracy   -> Train: {m_train['train_accuracy']:.4f}
            | Test: {m_test['test_accuracy']:.4f}")

```
¿Por qué es útil el `hasattr` para los parámetros?

Si el estimador que pasa por `evaluar` es un modelo común de Scikit-Learn (como una `LogisticRegression` o un `RandomForestClassifier` estándar), **no** tiene el atributo `best_params_` y lanzaría un error. Gracias a `hasattr(...)`, Python pregunta primero si el objeto contiene esos parámetros; si los tiene los imprime, y si no, simplemente se los salta de forma totalmente transparente y segura.

Análisis

Añade lo pedido correctamente y de forma limpia. Se acepta.

Prompt

```
NUMERICAS = NUMERICAS.remove('colesterol_total') if 'colesterol_total' in
NUMERICAS else NUMERICAS

Me falla esto cuando intento eliminar la columna
```

Respuesta IA Gemini Flash Lite 3.5 en Gemini

```
El error ocurre porque el método `.remove()` de las listas en Python
modifica la lista **directamente en el lugar (in-place)** y **devuelve
`None**`.

Al hacer `NUMERICAS = NUMERICAS.remove(...)`, estás sobrescribiendo la
variable `NUMERICAS` con `None`, lo que provoca que falle en cualquier
operación posterior.

Dependiendo de cómo quieras manejarlo, tienes dos formas correctas de
solucionarlo:

### Opción 1: Modificar la lista directamente (sin reasignar)

Si `NUMERICAS` es una lista y quieres modificarla sin crear una nueva:

```python
if 'colesterol_total' in NUMERICAS:
    NUMERICAS.remove('colesterol_total')

```

Opción 2: Usar comprensión de listas (crea una nueva lista)

Si prefieres mantener una asignación limpia (por ejemplo, si `NUMERICAS`
es una tupla o prefieres un enfoque funcional):

```python
NUMERICAS = [col for col in NUMERICAS if col != 'colesterol_total']

```

*Nota: Si `NUMERICAS` es un índice de pandas (`pd.Index`), puedes usar
`NUMERICAS.drop('colesterol_total')`.*
```

Análisis

La respuesta de la IA aclara por qué estaba fallando y da una solución correcta. Se acepta la primera opción.

Alejandro:

Prompt

```
Revisa el requirements.txt del repositorio. Queremos asegurar la
reproducibilidad estricta de los resultados entre los diferentes entornos
del equipo. Analiza si las versiones fijadas de las librerías científicas
centrales (NumPy, Pandas, Scikit-Learn, SciPy) requieren una versión
específica de Python para compilar sin conflictos de dependencias y sin
alterar las semillas estocásticas.
```

Respuesta IA Claude Code (Claude 3.5 Sonnet / 3.7), CLI en Windows

```
Lectura y análisis de requirements.txt:
Las versiones congeladas en el proyecto utilizan paquetes de última
generación (numpy 2.x, pandas 3.x, scikit-learn 1.9.x, imbalanced-learn
0.13.x), las cuales requieren de forma estricta un intérprete de Python
>= 3.11.

Recomendación técnica:
1. Desplegar un entorno virtual dedicado con Python 3.11 (evitando
versiones 3.10 que romperían el árbol de dependencias).
2. Aislar las dependencias binarias dependientes del sistema operativo
(mkl_fft, pywin32) para asegurar que el comportamiento estocástico
dependa exclusivamente de random_state en scikit-learn y numpy.
```

Análisis

El equipo auditó la coherencia de versiones para cumplir con el apartado de reproducibilidad del enunciado. Se verificó que compilar sobre Python 3.11.18 garantizaba que las particiones pseudoaleatorias de `StratifiedShuffleSplit` produjeran exactamente las mismas matrices numéricas en los diferentes equipos de trabajo.

Prompt

```
Despliega el entorno aislado con Python 3.11, instala el requirements.txt
y ejecuta una auditoría de integración ejecutando los cuadernos del
repositorio en copias temporales. Compara las salidas generadas con los
metadatos almacenados para verificar que no existan discrepancias
numéricas en el EDA ni en las tablas de evaluación preliminar.
```

Respuesta IA Claude Code (Claude 3.5 Sonnet / 3.7), CLI en Windows

```
Se configuró el entorno con Python 3.11.18 y se ejecutaron los cuadernos
en un directorio temporal aislado, preservando el árbol de trabajo:
- 03_test_estadisticos: concordancia del 100% en matrices y contrastes.
- 01_eda: se detectó una discrepancia tipográfica menor en la celda de
detección de baja varianza en dos variables cuyo ratio de frecuencia es
1.0 (empate entre modas), debido al criterio de ordenación interna de
Pandas, sin alteración alguna en las métricas de varianza nula.
- 02_preprocesamiento: concordancia idéntica en las etapas de limpieza
(P0 a P3).
```

Análisis

Se validó la consistencia de los experimentos. Se comprobó manualmente que la divergencia detectada en el EDA se debía a que cuando dos categorías empatan exactamente en frecuencia absoluta, el índice retornado por `.value_counts()` no posee un orden lexicográfico determinista en versiones distintas de Pandas. No obstante, al coincidir el ratio y el porcentaje de varianza, la validez del análisis

exploratorio permanece íntegra.

Prompt

```
A partir de las conclusiones del EDA, donde justificamos la
estratificación etaria clínica [18, 40, 55, 65, 120) y la categorización
de la OMS para el IMC, queremos incorporar la variante P4 que sustituya
edad e IMC por sus códigos ordinales. Además, planifica la incorporación
de preprocesados para desbalanceo (SMOTE y ADASYN) y selección de
variables (RFE), los cuales deben tomar como base P3 o P4 en función de
cuál maximice el F1 macro en test. Integra el código respetando
escrupulosamente los bloques y la arquitectura previa.
```

Respuesta IA Claude Code (Claude 3.5 Sonnet / 3.7), CLI en Windows

```
Implementación inicial:
- Se formuló la función discretizar_edad_imc mapeando a escala ordinal
entera.
- Se calcularon los rendimientos en CV: P3 (F1: 0,6265) frente a P4 (F1:
0,6317), seleccionando P4 como base óptima.
- Se incorporaron celdas de balanceo sintético (SMOTE, ADASYN) y
eliminación recursiva (RFECV).
La IA presentó el código agrupando las nuevas variantes en una sección
separada al final del notebook.
```

Análisis

El equipo rechazó la disposición arquitectónica propuesta por el asistente. Colocar los nuevos preprocesados al final en celdas aisladas rompía la modularidad del proyecto. Se ordenó reestructurar el notebook para que P4 se registrara limpiamente en el diccionario central de transformaciones (capítulo 4), permitiendo evaluar todas las combinaciones bajo el mismo bucle comparativo. Además que al final se utilizó un one-hot encoder para la edad y el IMC.

Prompt

```
por qué se requieren dos tipos de Pipeline distintos? Justifica técnica y
matemáticamente por qué el Pipeline estándar de Scikit-Learn falla al
integrar SMOTE/ADASYN y por qué la solución reside en imblearn.pipeline.
y en la selección de características, RFECV conserva las 30 variables
completas porque la curva de score es estrictamente creciente. Busca cómo
funciona el RFECV para hacer que se eliminen variables para tener un
preprocesamiento diferente.
```

Respuesta IA Claude Code (Claude 3.5 Sonnet / 3.7), CLI en Windows

```
1. Justificación del Pipeline:
```

```
El objeto Pipeline nativo de Scikit-Learn exige que los pasos intermedios
implementen exclusivamente fit() y transform(). Métodos de sobremuestreo
como SMOTE o ADASYN alteran la cardinalidad del vector de etiquetas
modificando el número de filas mediante fit_resample(), lo que genera un
error de interfaz en Scikit-Learn. Se emplea imblearn.pipeline.Pipeline,
que encapsula este método y desactiva el remuestreo de forma automática
durante la fase de inferencia o predict().

2. Implementación de RFECV con la regla 1-SE:
Dado que el score de validación asciende asintóticamente hasta las 30
variables (de 0,493 con 5 atributos a 0,647 con 30), se implementó la
clase RFECVUnErrorEstandar:
- Identifica la puntuación máxima de validación en CV y su error estándar
asociado.
- Selecciona el menor número de variables k tal que:
  score(k) >= score_max - 1 * SE(score_max).
Resultado: reduce la dimensionalidad a 27 atributos (eliminando
trigliceridos, frecuencia_cardiaca_reposo y
antecedentes_familiares_Paterno) preservando idéntico poder predictivo.
```

Análisis

detectamos que la IA había incurrido en un error de cómputo en la primera implementación de la clase `RFECVUnErrorEstandar`, promediando 9 particiones en lugar de 3 debido a un indexado incorrecto en `cv_results_`. Se exigió la depuración del código, logrando aislar un conjunto parsimonioso de 27 variables clínicamente interpretables y con rendimiento estadísticamente indistinguible de la cohorte completa. Aunque luego se arregló ya que se eliminaban variables imputadas sueltas producidas de los one-hot encodings, por lo que ahora ya se elimina todo lo que se tiene que eliminar.

Prompt

```
Realiza una auditoría técnica y metodológica completa sobre el cuaderno
de preprocesamiento y modelado (02_preprocesamiento_y_modelado).
Identifica vulnerabilidades relativas a: integridad de los conjuntos de
evaluación: evalúa si el filtrado de filas previo altera la
comparabilidad de las particiones entre preprocesados. mutabilidad de
estado global en Scikit-Learn: verifica si la manipulación de listas de
columnas compartidas induce fugas o descarte silencioso de variables.
tratamiento de desbalanceo con variables mixtas: analiza si la posición
de SMOTE y ADASYN dentro del pipeline tras el One-Hot Encoding genera
interpolaciones espurias en variables dummy o booleanas. Comprueba si la
eliminación de variables individuales compromete la coherencia de
variables categóricas codificadas con drop="first".
```

Respuesta IA Asistente de IA (Modelo de Razonamiento Avanzado)

1. Fuga y asimetría de evaluación: Al ejecutar eliminar_pcr_alta sobre el DataFrame global previo al StratifiedShuffleSplit, los preprocesados P3-P7 se evalúan sobre 672 muestras de test frente a las 675 de P0-P2, invalidando la comparación directa de métricas al eliminar pacientes atípicos del conjunto de prueba.
2. Corrupción de P0_base por referencia mutable: La instrucción NUMERICAS.remove('colesterol_total') altera la lista interna de ColumnTransformer por referencia. Con remainder='drop', P0 descarta colesterol_total de forma silenciosa, anulando su papel de línea base completa.
3. Corrupción de datos por remuestreo continuo: SMOTE y ADASYN interpolan linealmente en el espacio transformado, asignando valores continuos decimales a dummies One-Hot (violando la exclusión mutua) y a booleanos (fumador_actual con valores fraccionarios). Se requiere SMOTENC antes de la binarización o desacoplar las variables categóricas.
4. Ruptura semántica en RFE: RFE eliminó la dummy Paterno manteniendo Materno y Ambos. Con drop='first', el vector [0, 0] resultante asimila a los pacientes con riesgo paterno con la categoría de referencia Ninguno, distorsionando el riesgo clínico.

Análisis

A partir del diagnóstico técnico, se validó la necesidad de reestructurar el flujo de ejecución para garantizar el rigor experimental. Se modificó el diseño para que los filtros de exclusión clínica actúen estrictamente en la fase de ajuste (`fit`) de cada partición de entrenamiento, preservando intacto y homogéneo el conjunto de prueba para todos los modelos. Asimismo, se blindó la inmutabilidad de las listas de columnas mediante funciones selectoras dinámicas (`numericas_presentes`) y se diseñó una clase de selección por bloques (`RFEGruposUnErrorEstandar`) para impedir que RFE fragmente categorías nominales.

Prompt

Evalúa el diseño técnico del preprocesamiento P4 (discretización de edad e IMC) y el tratamiento de valores anómalos: analiza las implicaciones de escalar códigos ordinales (0, 1, 2, 3) con StandardScaler en modelos lineales cuando los rangos etarios son asimétricos (22, 15, 10 y 55 años). examina la viabilidad matemática de imputar categorías mediante la mediana frente a la moda. evalúa si 177 registros con consumo_alcohol_semanal negativo deben gestionarse mediante un indicador booleano (alcohol_negativo) o si resulta metodológicamente superior truncar a 0.

Respuesta IA Asistente de IA (Modelo de Razonamiento Avanzado)

1. Restricción de linealidad espuria: Mapear intervalos asimétricos a enteros equidistantes y escalarlos en LogisticRegression/LinearSVC fuerza la hipótesis de que el incremento de riesgo es uniforme por paso de categoría, perdiendo la granularidad continua sin ganar flexibilidad no lineal. La solución es codificar los grupos como dummies categóricas (One-Hot).
2. Imputación fraccionaria por mediana: SimpleImputer(strategy='median') sobre muestras con número par de casos centrales promedia valores discretos, generando códigos no existentes (ej. 2.5 entre sobrepeso y obesidad I). Debe emplearse la moda (strategy='most_frequent').
3. Flag vs. Truncamiento a 0 en alcohol: Aunque el subconjunto con alcohol negativo concentra un 82% de riesgo bajo en este dataset, crear un flag induce shortcut learning en despliegue: en producción clínica ningún médico registrará valores negativos, perdiéndose el sesgo del artefacto. Truncar a 0 unidades/semana es clínicamente coherente y evita sobreajustar a un defecto de recogida de datos.

Análisis

Se adoptó la recomendación de modelar los grupos de edad e IMC mediante variables dummy (One-Hot) con imputación modal, evitando sesgos en edades avanzadas y garantizando la coherencia de los modelos lineales. En cuanto al consumo de alcohol, determiné descartar la creación de la columna indicadora `alcohol_negativo` y truncar directamente los valores negativos a 0. Esto previene que el clasificador aprenda atajos espurios derivados de anomalías del formulario y asegura la robustez del modelo ante entradas de datos en entornos clínicos reales.